Replication Code Appendix

ADH China Shock Political Event Study Extension

Rishav Roy

Table of contents

1 00_config.R 2

2 01_helpers.R 3

3 02_build_analysis_data.R 15

4 03_estimate_event_study.R 27

5 04_export_outputs.R 35

6 05_export_diagnostic_maps.R 38

7 06_sixth_extensions.R 45

8 run_pipeline.R 57

9 county_bridge_exceptions.csv 59

10 dependencies.json 60

11 make_replication_code_pdf.R 61

12 README.md 63

This appendix prints the replication source files used to construct the final results. Files are included directly from `../src/` at render time.

1 00_config.R

```r
# extension-adh2013/src/00_config.R

if (!requireNamespace("here", quietly = TRUE)) {
  stop("Package 'here' is required. Install it before sourcing the pipeline config.")
}

REPLICATION_DIR <- here::here()
SRC_DIR <- here::here("src")
OUTPUT_DIR <- here::here("output")
INTERMEDIATE_DIR <- here::here("output", "intermediate")
TABLE_DIR <- here::here("output", "tables")
FIGURE_DIR <- here::here("output", "figures")
DIAGNOSTIC_DIR <- here::here("output", "diagnostics")

CONFIG <- list(
  replication_dir = REPLICATION_DIR,
  src_dir = SRC_DIR,
  output_dir = OUTPUT_DIR,
  intermediate_dir = INTERMEDIATE_DIR,
  table_dir = TABLE_DIR,
  figure_dir = FIGURE_DIR,
  diagnostic_dir = DIAGNOSTIC_DIR,

  first_election_year = 1972L,
  diagnostic_first_election_year = 1952L,
  last_election_year = 2020L,
  reference_year = 1988L,

  outcome_var = "rep_margin",
  outcome_label = "Republican presidential vote margin",
  exposure_var = "exposure",
  exposure_label = "ADH China import exposure, 1990-2007",
  exposure_units = "thousand dollars per worker",
  weight_var = "adh_weight",

  crosswalk_weight = "m5_weight",
  crosswalk_missing_weight_policy = "fallback_m2",
  renormalize_crosswalk_weights = FALSE,
  weight_sum_tolerance = 1e-6,
  min_retained_vote_share = 0.98,
  max_abs_vote_identity_error = 1e-6,
  max_missing_cz_year_share = 0,

  require_balanced_panel = TRUE,
  require_no_duplicate_county_years = TRUE,
  require_no_vcov_repair_for_main = TRUE,
  allow_vcov_repair = FALSE,
  allow_failed_diagnostics = FALSE,

  conley_cutoff_km = 500,
  conley_cutoffs_km = c(250, 500, 750, 1000),
  conley_distance = "spherical",
  nw_lag = 1L,
  dk_lag = 1L,
  main_se_type = "cluster_cz",

  standardize_interacted_controls = TRUE,
  export_bandwidth_diagnostic = TRUE,
  export_crosswalk_maps = TRUE,
  run_crosswalk_sensitivity = FALSE,
  run_sixth_extensions = TRUE,
  extension_crosswalk_weight = "m5_weight",
  extension_crosswalk_missing_weight_policy = "fallback_m2",
  save_single_rds = FALSE,
  rds_chunk_size_mib = 45,
  county1990_shapefile_path = Sys.getenv("COUNTY1990_SHAPEFILE", unset = file.path(REPLICATION_DIR, "spatial-data", "counties-1990")),
  nhgis_extract_dir = Sys.getenv("NHGIS_1990_COUNTY_EXTRACT_DIR", unset = file.path(REPLICATION_DIR, "spatial-data", "nhgis_1990_county"))
)

CONFIG$baseline_controls <- c(
  "l_shind_manuf_cbp",
  "l_sh_popedu_c",
  "l_sh_popfborn",
  "l_sh_empl_f",
  "l_sh_routine33",
  "l_task_outsource"
)

CONFIG$interacted_controls <- CONFIG$baseline_controls

CONFIG$core_interacted_controls <- c(
  "l_shind_manuf_cbp",
  "l_sh_popedu_c",
  "l_sh_popfborn"
)
```

2 01_helpers.R

```r
# extension-adh2013/src/01_helpers.R

required_packages <- function(config = NULL) {
  pkgs <- c(
    "dplyr", "fixest", "ggplot2", "haven", "jsonlite", "kableExtra", "knitr",
    "purrr", "readr", "readxl", "rlang", "stringr", "tibble", "tidyr"
  )
  if (!is.null(config) && isTRUE(config$export_crosswalk_maps)) pkgs <- c(pkgs, "sf")
  unique(pkgs)
}

optional_packages <- function() {
  c("ipumsr")
}

configured_shapefile_defaults <- function(config) {
  c(
    file.path(config$replication_dir, "spatial-data", "counties-1990"),
    file.path(config$replication_dir, "spatial-data", "nhgis_1990_county")
  )
}

load_required_packages <- function(config = NULL) {
  pkgs <- required_packages(config)
  missing <- pkgs[!vapply(pkgs, requireNamespace, FUN.VALUE = logical(1), quietly = TRUE)]
  if (length(missing) > 0) {
    stop(
      "Missing required R packages: ", paste(missing, collapse = ", "),
      ". Install them before running the pipeline, or restore the project dependency manifest.",
      call. = FALSE
    )
  }
  invisible(lapply(pkgs, library, character.only = TRUE))
  options(dplyr.summarise.inform = FALSE)
}

ensure_output_dirs <- function(config) {
  dirs <- c(config$output_dir, config$intermediate_dir, config$table_dir,
            config$figure_dir, config$diagnostic_dir)
  invisible(lapply(dirs, dir.create, recursive = TRUE, showWarnings = FALSE))
}

weight_slug <- function(weight_col) {
  stringr::str_replace(weight_col, "_weight$", "")
}

finalize_config <- function(config) {
  allowed_weights <- c("m2_weight", "m4_weight", "m5_weight", "m6_weight")
  if (!config$crosswalk_weight %in% allowed_weights) {
    stop("CONFIG$crosswalk_weight must be one of: ", paste(allowed_weights, collapse = ", "))
  }

  allowed_policies <- c("fail", "fallback_m4", "fallback_m2", "identity_if_same_fips")
  if (!config$crosswalk_missing_weight_policy %in% allowed_policies) {
    stop(
      "CONFIG$crosswalk_missing_weight_policy must be one of: ",
      paste(allowed_policies, collapse = ", ")
    )
  }

  if (is.null(config$interacted_controls)) {
    config$interacted_controls <- config$baseline_controls
  }
  missing_interacted <- setdiff(config$interacted_controls, config$baseline_controls)
  if (length(missing_interacted) > 0) {
    stop("CONFIG$interacted_controls must be drawn from CONFIG$baseline_controls. Invalid: ",
         paste(missing_interacted, collapse = ", "))
  }

  slug <- weight_slug(config$crosswalk_weight)
  config$crosswalk_weight_slug <- slug
  config$event_years <- seq(config$first_election_year, config$last_election_year, by = 4)
  config$diagnostic_event_years <- seq(config$diagnostic_first_election_year, config$last_election_year, by = 4)
  config$source_decades <- sort(unique(year_to_source_decade(config$diagnostic_event_years)))
  if (is.null(config$main_se_type) || !nzchar(config$main_se_type)) {
    config$main_se_type <- paste0("conley_", config$conley_cutoff_km, "km")
  }

  # Treat common placeholder strings as unset. This avoids silently skipping maps
  # when COUNTY1990_SHAPEFILE was previously set to something like
  # "path/to/nhgis_1990_county.shp".
  if (is.null(config$county1990_shapefile_path) ||
      !nzchar(config$county1990_shapefile_path) ||
      grepl("^path/to", config$county1990_shapefile_path, ignore.case = TRUE)) {
    config$county1990_shapefile_path <- file.path(config$replication_dir, "spatial-data", "counties-1990")
  }
  if (is.null(config$nhgis_extract_dir) || !nzchar(config$nhgis_extract_dir)) {
```

```
  config$nhgis_extract_dir <- file.path(config$replication_dir, "spatial-data", "nhgis_1990_county")
}

config$analysis_panel_rds <- file.path(
  config$intermediate_dir, paste0("analysis_panel_aa2021_rep_margin_", slug, ".rds")
)
config$analysis_panel_full_rds <- file.path(
  config$intermediate_dir, paste0("analysis_panel_full_aa2021_rep_margin_", slug, ".rds")
)
config$diagnostic_panel_rds <- file.path(
  config$intermediate_dir, paste0("analysis_panel_aa2021_rep_margin_1952start_", slug, ".rds")
)
config$diagnostic_panel_full_rds <- file.path(
  config$intermediate_dir, paste0("analysis_panel_full_aa2021_rep_margin_1952start_", slug, ".rds")
)
config$cz_centroids_csv <- file.path(
  config$intermediate_dir, "cz_centroids_1990_population_weighted.csv"
)
config$event_study_rds <- file.path(
  config$intermediate_dir, paste0("event_study_results_aa2021_rep_margin_", slug, ".rds")
)
config$event_study_csv <- file.path(
  config$intermediate_dir, paste0("event_study_coefficients_aa2021_rep_margin_", slug, ".csv")
)
config$all_specs_rds <- file.path(
  config$intermediate_dir, paste0("event_study_results_all_specs_aa2021_rep_margin_", slug, ".rds")
)
config$all_specs_csv <- file.path(
  config$intermediate_dir, paste0("event_study_coefficients_all_specs_aa2021_rep_margin_", slug, ".csv")
)

config$dependency_manifest_json <- file.path(config$diagnostic_dir, "dependency_manifest.json")
config$pipeline_manifest_json <- file.path(config$diagnostic_dir, "pipeline_manifest.json")
config$output_manifest_csv <- file.path(config$diagnostic_dir, "output_manifest.csv")
config$output_manifest_json <- file.path(config$diagnostic_dir, "output_manifest.json")
config$validation_checks_csv <- file.path(config$diagnostic_dir, "pipeline_validation_checks.csv")
config$crosswalk_diagnostics_csv <- file.path(
  config$diagnostic_dir, paste0("ftz_", slug, "_crosswalk_source_diagnostics.csv")
)
config$crosswalk_zero_support_csv <- file.path(
  config$diagnostic_dir, paste0("ftz_", slug, "_undefined_or_zero_support_counties.csv")
)
config$crosswalk_row_diagnostics_csv <- file.path(
  config$diagnostic_dir, paste0("ftz_", slug, "_crosswalk_row_diagnostics.csv")
)
config$crosswalk_issue_vote_report_csv <- file.path(
  config$diagnostic_dir, paste0("ftz_", slug, "_crosswalk_issue_vote_report.csv")
)
config$crosswalk_preflight_summary_csv <- file.path(
  config$diagnostic_dir, "ftz_m5_m6_zero_support_preflight_summary.csv"
)
config$crosswalk_preflight_counties_csv <- file.path(
  config$diagnostic_dir, "ftz_m5_m6_zero_support_preflight_counties.csv"
)
config$all_weight_support_csv <- file.path(
  config$diagnostic_dir, "ftz_all_weight_support_by_source_county.csv"
)
config$target_weight_support_county_csv <- file.path(
  config$diagnostic_dir, "ftz_weight_support_by_1990_county.csv"
)
config$target_weight_support_cz_csv <- file.path(
  config$diagnostic_dir, "ftz_weight_support_by_cz.csv"
)
config$county_bridge_classification_csv <- file.path(
  config$diagnostic_dir, paste0("unmatched_source_county_classification_", slug, ".csv")
)
config$mainland_retention_csv <- file.path(
  config$diagnostic_dir, paste0("county_bridge_retention_by_scope_", slug, ".csv")
)
config$baseline_control_summary_csv <- file.path(
  config$diagnostic_dir, "baseline_control_summary.csv"
)
config$baseline_control_correlation_csv <- file.path(
  config$diagnostic_dir, "baseline_control_correlation_matrix.csv"
)
config$interacted_control_variance_csv <- file.path(
  config$diagnostic_dir, "interacted_control_variance_diagnostics.csv"
)
config$conley_neighbor_counts_csv <- file.path(
  config$diagnostic_dir, "conley_neighbor_count_by_cutoff.csv"
)
config$vcov_rank_csv <- file.path(
  config$diagnostic_dir, "event_study_vcov_rank_by_se_type.csv"
)
config$se_warning_diagnostics_csv <- file.path(
  config$diagnostic_dir, "event_study_se_warning_diagnostics.csv"
)
config$crosswalk_comparison_csv <- file.path(
  config$diagnostic_dir, "crosswalk_specification_comparison_coefficients.csv"
)
```

```
  config$crosswalk_comparison_png <- file.path(
    config$figure_dir, "fig_crosswalk_specification_comparison.png"
  )
  config$crosswalk_comparison_pdf <- file.path(
    config$figure_dir, "fig_crosswalk_specification_comparison.pdf"
  )
  config$crosswalk_comparison_delta_csv <- file.path(
    config$diagnostic_dir, "crosswalk_specification_comparison_deltas_vs_pure_m2.csv"
  )
  config$crosswalk_comparison_delta_png <- file.path(
    config$figure_dir, "fig_crosswalk_specification_comparison_deltas_vs_pure_m2.png"
  )
  config$crosswalk_comparison_delta_pdf <- file.path(
    config$figure_dir, "fig_crosswalk_specification_comparison_deltas_vs_pure_m2.pdf"
  )
  config$crosswalk_support_county_map_png <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties.png"
  )
  config$crosswalk_support_county_map_pdf <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties.pdf"
  )
  config$crosswalk_support_cz_map_png <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs.png"
  )
  config$crosswalk_support_cz_map_pdf <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs.pdf"
  )
  config$crosswalk_support_county_map_simplified_png <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties_simplified.png"
  )
  config$crosswalk_support_county_map_small_png <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties_small.png"
  )
  config$crosswalk_support_county_map_simplified_pdf <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties_simplified.pdf"
  )
  config$crosswalk_support_cz_map_simplified_png <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs_simplified.png"
  )
  config$crosswalk_support_cz_map_small_png <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs_small.png"
  )
  config$crosswalk_support_cz_map_simplified_pdf <- file.path(
    config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs_simplified.pdf"
  )
  config$outcome_duplicate_report_csv <- file.path(
    config$diagnostic_dir, "aa2021_presidential_duplicate_county_years.csv"
  )
  config$outcome_validation_csv <- file.path(
    config$diagnostic_dir, "aa2021_presidential_county_year_validation.csv"
  )
  config$national_vote_totals_csv <- file.path(
    config$diagnostic_dir, "aa2021_presidential_national_vote_totals.csv"
  )
  config$state_vote_totals_csv <- file.path(
    config$diagnostic_dir, "aa2021_presidential_state_vote_totals.csv"
  )
  config$bridge_diagnostics_csv <- file.path(
    config$diagnostic_dir, paste0("county_bridge_diagnostics_by_year_", slug, ".csv")
  )
  config$unmatched_source_counties_csv <- file.path(
    config$diagnostic_dir, paste0("unmatched_source_counties_", slug, ".csv")
  )
  config$missing_cz_years_csv <- file.path(
    config$diagnostic_dir, paste0("missing_cz_year_outcomes_", slug, ".csv")
  )
  config$panel_coverage_csv <- file.path(
    config$diagnostic_dir, paste0("panel_coverage_by_year_", slug, ".csv")
  )
  config$spec_status_csv <- file.path(config$diagnostic_dir, "event_study_spec_status.csv")
  config$model_diagnostics_csv <- file.path(config$diagnostic_dir, "event_study_model_diagnostics.csv")
  config$controls_manifest_csv <- file.path(config$diagnostic_dir, "event_study_controls_manifest.csv")
  config$vcov_diagnostics_csv <- file.path(config$diagnostic_dir, "event_study_vcov_diagnostics.csv")
  config$vcov_eigenvalues_csv <- file.path(config$diagnostic_dir, "event_study_vcov_eigenvalues.csv")
  config$pretrend_tests_csv <- file.path(config$diagnostic_dir, "event_study_pretrend_tests.csv")
  config$pretrend_coefficients_csv <- file.path(config$diagnostic_dir, "event_study_pretrend_coefficients.csv")

  config$table_tex <- file.path(config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, ".tex"))
  config$table_csv <- file.path(config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, ".csv"))
  config$table_pdf <- file.path(config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, ".pdf"))
  config$table_standalone_tex <- file.path(
    config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, "_standalone.tex")
  )
  config$figure_pdf <- file.path(config$figure_dir, paste0("fig_event_study_rep_margin_main_specs_", slug, ".pdf"))
  config$figure_png <- file.path(config$figure_dir, paste0("fig_event_study_rep_margin_main_specs_", slug, ".png"))

  # Sixth-section extension outputs. These are intentionally keyed to the
  # primary crosswalk slug when relevant, but are otherwise single-project
  # diagnostics that should not be re-estimated separately for every
  # crosswalk-sensitivity specification.
```

```
  config$bartik_first_stage_csv <- file.path(config$diagnostic_dir, "bartik_first_stage_diagnostics.csv")
  config$bartik_balance_csv <- file.path(config$diagnostic_dir, "bartik_balance_correlations.csv")
  config$bartik_pretrend_csv <- file.path(config$diagnostic_dir, "bartik_pretrend_placebos.csv")
  config$bartik_data_availability_csv <- file.path(config$diagnostic_dir, "bartik_rotemberg_data_availability.csv")
  config$bartik_identification_memo_md <- file.path(config$diagnostic_dir, "bartik_identification_memo.md")
  config$bartik_first_stage_plot_png <- file.path(config$figure_dir, "fig_bartik_first_stage.png")
  config$bartik_first_stage_plot_pdf <- file.path(config$figure_dir, "fig_bartik_first_stage.pdf")
  config$bartik_industry_shift_summary_csv <- file.path(config$diagnostic_dir, "bartik_industry_shift_summary.csv")
  config$bartik_preperiod_workfile_csv <- file.path(config$diagnostic_dir, "bartik_preperiod_workfile_diagnostics.csv")

  config$alternative_outcomes_coefficients_csv <- file.path(config$diagnostic_dir, "alternative_political_outcome_event_studies.csv")
  config$alternative_outcomes_summary_csv <- file.path(config$diagnostic_dir, "alternative_political_outcome_summary.csv")
  config$alternative_outcomes_plot_png <- file.path(config$figure_dir, "fig_alternative_political_outcomes.png")
  config$alternative_outcomes_plot_pdf <- file.path(config$figure_dir, "fig_alternative_political_outcomes.pdf")
  config$alternative_outcomes_status_csv <- file.path(config$diagnostic_dir, "alternative_political_outcome_status.csv")
  config$alternative_outcomes_decomposition_plot_png <- file.path(config$figure_dir,
   ↪  "fig_alternative_political_outcome_decomposition.png")
  config$alternative_outcomes_decomposition_plot_pdf <- file.path(config$figure_dir,
   ↪  "fig_alternative_political_outcome_decomposition.pdf")
  config$alternative_outcomes_vote_choice_plot_png <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_choice.png")
  config$alternative_outcomes_vote_choice_plot_pdf <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_choice.pdf")
  config$alternative_outcomes_vote_volume_plot_png <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_volume.png")
  config$alternative_outcomes_vote_volume_plot_pdf <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_volume.pdf")

  config$nanda_cz_panel_csv <- file.path(config$intermediate_dir, "nanda_2004_2022_cz_panel.csv")
  config$nanda_outcomes_coefficients_csv <- file.path(config$diagnostic_dir, "nanda_outcome_event_studies.csv")
  config$nanda_outcomes_summary_csv <- file.path(config$diagnostic_dir, "nanda_outcome_summary.csv")
  config$nanda_outcomes_plot_png <- file.path(config$figure_dir, "fig_nanda_outcome_event_studies.png")
  config$nanda_outcomes_plot_pdf <- file.path(config$figure_dir, "fig_nanda_outcome_event_studies.pdf")
  config$nanda_outcomes_status_csv <- file.path(config$diagnostic_dir, "nanda_outcome_event_study_status.csv")
  config$nanda_county_validation_csv <- file.path(config$diagnostic_dir, "nanda_county_year_validation.csv")
  config$nanda_cz_validation_csv <- file.path(config$diagnostic_dir, "nanda_cz_year_validation.csv")
  config$nanda_outlier_rates_csv <- file.path(config$diagnostic_dir, "nanda_outlier_rates.csv")
  config$nanda_outcome_availability_csv <- file.path(config$diagnostic_dir, "nanda_outcome_year_availability.csv")
  config$nanda_turnout_plot_png <- file.path(config$figure_dir, "fig_nanda_turnout_registration.png")
  config$nanda_turnout_plot_pdf <- file.path(config$figure_dir, "fig_nanda_turnout_registration.pdf")
  config$nanda_vote_plot_png <- file.path(config$figure_dir, "fig_nanda_presidential_vote_ratios.png")
  config$nanda_vote_plot_pdf <- file.path(config$figure_dir, "fig_nanda_presidential_vote_ratios.pdf")
  config$nanda_partisanship_plot_png <- file.path(config$figure_dir, "fig_nanda_partisanship_indices.png")
  config$nanda_partisanship_plot_pdf <- file.path(config$figure_dir, "fig_nanda_partisanship_indices.pdf")

  config$adhm2020_summary_csv <- file.path(config$diagnostic_dir, "adhm2020_public_data_summary.csv")
  config$adhm2020_regressions_csv <- file.path(config$diagnostic_dir, "adhm2020_mechanism_regressions.csv")
  config$adhm2020_outcome_dictionary_csv <- file.path(config$diagnostic_dir, "adhm2020_outcome_dictionary.csv")
  config$adhm2020_status_csv <- file.path(config$diagnostic_dir, "adhm2020_mechanism_regression_status.csv")
  config$adhm2020_heterogeneity_availability_csv <- file.path(config$diagnostic_dir, "adhm2020_heterogeneity_data_availability.csv")
  config$adhm2020_plot_png <- file.path(config$figure_dir, "fig_adhm2020_mechanism_regressions.png")
  config$adhm2020_plot_pdf <- file.path(config$figure_dir, "fig_adhm2020_mechanism_regressions.pdf")

  config$subperiod_exposure_csv <- file.path(config$intermediate_dir, "adh_subperiod_exposure_by_cz.csv")
  config$subperiod_exposure_summary_csv <- file.path(config$diagnostic_dir, "adh_subperiod_exposure_summary.csv")
  config$subperiod_event_study_coefficients_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_event_study_coefficients.csv")
  config$subperiod_event_study_status_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_event_study_status.csv")
  config$subperiod_event_study_plot_png <- file.path(config$figure_dir, "fig_subperiod_exposure_event_study.png")
  config$subperiod_event_study_plot_pdf <- file.path(config$figure_dir, "fig_subperiod_exposure_event_study.pdf")
  config$subperiod_first_stage_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_first_stage.csv")
  config$subperiod_equality_tests_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_equality_tests.csv")
  config$subperiod_selected_horizons_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_selected_horizons.csv")

  config
}

fips_to_gisjoin <- function(fips) {
  fips <- stringr::str_pad(as.character(fips), width = 5, side = "left", pad = "0")
  paste0("G", substr(fips, 1, 2), "0", substr(fips, 3, 5), "0")
}

gisjoin_to_fips <- function(gisjoin) {
  gisjoin <- as.character(gisjoin)
  out <- ifelse(is.na(gisjoin), NA_character_, paste0(substr(gisjoin, 2, 3), substr(gisjoin, 5, 7)))
  stringr::str_pad(out, width = 5, side = "left", pad = "0")
}

year_to_source_decade <- function(year) {
  dplyr::case_when(
    year < 1980 ~ 1970L,
    year < 1990 ~ 1980L,
    year < 2000 ~ 1990L,
    year < 2010 ~ 2000L,
    year < 2020 ~ 2010L,
    TRUE ~ 2020L
  )
}

read_aa_rdata_object <- function(path, object_name = NULL) {
  e <- new.env(parent = emptyenv())
  loaded <- load(path, envir = e)
  if (!is.null(object_name)) return(e[[object_name]])
  candidates <- setdiff(loaded, ".Random.seed")
  if (length(candidates) != 1) stop("Expected exactly one non-.Random.seed object in: ", path)
```

```
  e[[candidates]]
}

sum_preserve_all_na <- function(x) {
  if (length(x) == 0 || all(is.na(x))) NA_real_ else sum(x, na.rm = TRUE)
}

read_ftz_crosswalk <- function(source_decade, zip_path, county_fips_1990_identity = NULL,
                               weight_col = "m5_weight",
                               missing_weight_policy = "fail",
                               renormalize_weights = FALSE,
                               tolerance = 1e-8) {
  if (source_decade == 1990L) {
    ids <- unique(as.character(county_fips_1990_identity))
    return(tibble::tibble(
      source_decade = 1990L,
      gisjoin_source = fips_to_gisjoin(ids),
      gisjoin_1990 = fips_to_gisjoin(ids),
      county_fips_source = stringr::str_pad(ids, 5, pad = "0"),
      county_fips_1990 = stringr::str_pad(ids, 5, pad = "0"),
      raw_weight = 1,
      fallback_m4_weight = 1,
      fallback_m2_weight = 1,
      raw_weight_missing_parts = 0L,
      raw_weight_parts = 1L,
      weight_sum_raw = 1,
      selected_weight_sum_raw = 1,
      crosswalk_weight = 1,
      crosswalk_weight_sum = 1,
      raw_weight_undefined = FALSE,
      raw_weight_sum_valid = TRUE,
      raw_weight_requires_policy = FALSE,
      fallback_used = FALSE,
      fallback_policy = "identity_1990",
      weight_source = "identity_1990"
    ))
  }

  file_name <- paste0("Crosswalk_", source_decade, "_1990.csv")
  source_col <- paste0("gisjoin_", source_decade)
  raw <- readr::read_csv(unz(zip_path, file_name), show_col_types = FALSE)
  required <- c(source_col, "gisjoin_1990", weight_col, "m4_weight", "m2_weight")
  missing <- setdiff(required, names(raw))
  if (length(missing) > 0) {
    stop("Missing required FTZ columns in ", file_name, ": ", paste(missing, collapse = ", "))
  }

  parts <- raw %>%
    dplyr::transmute(
      source_decade = as.integer(source_decade),
      gisjoin_source = as.character(.data[[source_col]]),
      gisjoin_1990 = as.character(.data[["gisjoin_1990"]]),
      raw_weight_part = as.numeric(.data[[weight_col]]),
      fallback_m4_part = as.numeric(.data[["m4_weight"]]),
      fallback_m2_part = as.numeric(.data[["m2_weight"]])
    ) %>%
    dplyr::mutate(
      gisjoin_source = dplyr::if_else(
        source_decade == 2020L,
        fips_to_gisjoin(stringr::str_pad(gisjoin_source, 5, pad = "0")),
        gisjoin_source
      )
    )

  out <- parts %>%
    dplyr::group_by(source_decade, gisjoin_source, gisjoin_1990) %>%
    dplyr::summarise(
      raw_weight = sum_preserve_all_na(raw_weight_part),
      fallback_m4_weight = sum_preserve_all_na(fallback_m4_part),
      fallback_m2_weight = sum_preserve_all_na(fallback_m2_part),
      raw_weight_missing_parts = sum(is.na(raw_weight_part)),
      raw_weight_parts = dplyr::n(),
      .groups = "drop"
    ) %>%
    dplyr::mutate(
      county_fips_source = gisjoin_to_fips(gisjoin_source),
      county_fips_1990 = gisjoin_to_fips(gisjoin_1990)
    ) %>%
    dplyr::group_by(source_decade, gisjoin_source) %>%
    dplyr::mutate(
      weight_sum_raw = if (all(is.na(raw_weight))) NA_real_ else sum(raw_weight, na.rm = TRUE),
      raw_weight_undefined = is.na(weight_sum_raw) | weight_sum_raw <= tolerance,
      raw_weight_sum_valid = !is.na(weight_sum_raw) & abs(weight_sum_raw - 1) <= tolerance,
      raw_weight_requires_policy = !raw_weight_sum_valid
    ) %>%
    dplyr::ungroup() %>%
    dplyr::mutate(
      selected_raw_weight = dplyr::case_when(
        !raw_weight_requires_policy ~ raw_weight,
        missing_weight_policy == "fallback_m4" ~ fallback_m4_weight,
```

```
        missing_weight_policy == "fallback_m2" ~ fallback_m2_weight,
        missing_weight_policy == "identity_if_same_fips" & county_fips_source == county_fips_1990 ~ 1,
        missing_weight_policy == "identity_if_same_fips" ~ 0,
        TRUE ~ NA_real_
      ),
      fallback_used = raw_weight_requires_policy & missing_weight_policy != "fail",
      fallback_policy = dplyr::case_when(
        !fallback_used ~ "none",
        missing_weight_policy == "fallback_m4" ~ "fallback_m4",
        missing_weight_policy == "fallback_m2" ~ "fallback_m2",
        missing_weight_policy == "identity_if_same_fips" ~ "identity_if_same_fips",
        TRUE ~ missing_weight_policy
      ),
      weight_source = dplyr::case_when(
        !fallback_used ~ weight_col,
        fallback_policy == "fallback_m4" ~ "m4_weight",
        fallback_policy == "fallback_m2" ~ "m2_weight",
        fallback_policy == "identity_if_same_fips" ~ "identity_if_same_fips",
        TRUE ~ weight_col
      )
    ) %>%
    dplyr::group_by(source_decade, gisjoin_source) %>%
    dplyr::mutate(
      selected_weight_sum_raw = if (all(is.na(selected_raw_weight))) {
        NA_real_
      } else {
        sum(selected_raw_weight, na.rm = TRUE)
      },
      crosswalk_weight = dplyr::if_else(
        !is.na(selected_weight_sum_raw) & selected_weight_sum_raw > tolerance,
        if (renormalize_weights) selected_raw_weight / selected_weight_sum_raw else selected_raw_weight,
        NA_real_
      ),
      crosswalk_weight_sum = if (all(is.na(crosswalk_weight))) NA_real_ else sum(crosswalk_weight, na.rm = TRUE)
    ) %>%
    dplyr::ungroup() %>%
    dplyr::select(
      source_decade, gisjoin_source, gisjoin_1990, county_fips_source, county_fips_1990,
      raw_weight, fallback_m4_weight, fallback_m2_weight, raw_weight_missing_parts,
      raw_weight_parts, weight_sum_raw, selected_weight_sum_raw, crosswalk_weight,
      crosswalk_weight_sum, raw_weight_undefined, raw_weight_sum_valid,
      raw_weight_requires_policy, fallback_used, fallback_policy, weight_source
    )

  out
}

fmt_num <- function(x, digits = 3) {
  ifelse(is.na(x), "", formatC(x, digits = digits, format = "f"))
}

star_code <- function(p) {
  dplyr::case_when(
    is.na(p) ~ "",
    p < 0.01 ~ "***",
    p < 0.05 ~ "**",
    p < 0.10 ~ "*",
    TRUE ~ ""
  )
}

make_interacted_control_vars <- function(data, controls, years, ref_year) {
  years_est <- setdiff(sort(unique(years)), ref_year)
  new_names <- character(0)
  for (ctrl in controls) {
    for (yr in years_est) {
      nm <- paste0("ctrl_", ctrl, "_", yr)
      data[[nm]] <- data[[ctrl]] * as.integer(data$year == yr)
      new_names <- c(new_names, nm)
    }
  }
  list(data = data, vars = new_names)
}

validation_check <- function(name, passed, fatal = TRUE, n_failed = NA_integer_, details = "") {
  tibble::tibble(
    check = name,
    status = if (isTRUE(passed)) "pass" else "fail",
    fatal = isTRUE(fatal),
    n_failed = n_failed,
    details = as.character(details)
  )
}

reported_check <- function(name, n_reported = 0L, details = "") {
  tibble::tibble(
    check = name,
    status = if (is.na(n_reported) || n_reported == 0L) "pass" else "reported",
    fatal = FALSE,
    n_failed = 0L,
```

```
    n_reported = as.integer(dplyr::coalesce(as.integer(n_reported), 0L)),
    details = as.character(details)
  )
}

has_fatal_failures <- function(checks) {
  if (nrow(checks) == 0 || !("status" %in% names(checks))) return(FALSE)
  any(checks$fatal %in% TRUE & checks$status == "fail", na.rm = TRUE)
}

has_any_failures <- function(checks) {
  if (nrow(checks) == 0 || !("status" %in% names(checks))) return(FALSE)
  any(checks$status == "fail", na.rm = TRUE)
}

has_reported_conditions <- function(checks) {
  if (nrow(checks) == 0 || !("status" %in% names(checks))) return(FALSE)
  any(checks$status == "reported", na.rm = TRUE)
}

write_dependency_manifest <- function(config) {
  core <- required_packages(config)
  optional <- optional_packages()
  pkgs <- unique(c(core, optional))
  dep <- tibble::tibble(
    package = pkgs,
    dependency_type = dplyr::case_when(
      package %in% core ~ "core",
      package %in% optional ~ "optional",
      TRUE ~ "other"
    ),
    installed = vapply(pkgs, requireNamespace, FUN.VALUE = logical(1), quietly = TRUE),
    version = vapply(pkgs, function(pkg) {
      if (!requireNamespace(pkg, quietly = TRUE)) return(NA_character_)
      as.character(utils::packageVersion(pkg))
    }, character(1))
  )
  jsonlite::write_json(
    list(
      generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
      r_version = R.version.string,
      packages = dep
    ),
    config$dependency_manifest_json,
    pretty = TRUE,
    auto_unbox = TRUE
  )
  invisible(dep)
}

file_checksum <- function(path) {
  if (!file.exists(path)) return(NA_character_)
  unname(tools::md5sum(path))
}

portable_path <- function(path, config = CONFIG) {
  if (is.null(path) || length(path) == 0 || is.na(path)) return(path)
  path_chr <- as.character(path)
  root <- normalizePath(config$replication_dir, winslash = "/", mustWork = FALSE)
  norm <- normalizePath(path_chr, winslash = "/", mustWork = FALSE)
  prefix <- paste0(root, "/")
  dplyr::case_when(
    norm == root ~ ".",
    startsWith(norm, prefix) ~ substring(norm, nchar(prefix) + 1L),
    TRUE ~ path_chr
  )
}

sanitize_manifest_paths <- function(x, config = CONFIG) {
  if (is.list(x)) {
    nms <- names(x)
    out <- lapply(x, sanitize_manifest_paths, config = config)
    names(out) <- nms
    return(out)
  }
  if (is.character(x) && length(x) == 1 && (grepl("/", x) || grepl("\\\\", x))) {
    return(portable_path(x, config))
  }
  x
}

chunk_dir_for_rds <- function(path) {
  file.path(dirname(path), paste0(tools::file_path_sans_ext(basename(path)), "_rds_chunks"))
}

write_chunked_rds <- function(object, path, config = CONFIG) {
  raw <- serialize(object, connection = NULL, xdr = TRUE)
  compressed <- memCompress(raw, type = "xz")
  chunk_size <- as.integer((config$rds_chunk_size_mib %||% 45) * 1024^2)
  if (!is.finite(chunk_size) || chunk_size <= 0) chunk_size <- 45L * 1024L^2L
```

```
  chunk_dir <- chunk_dir_for_rds(path)
  if (dir.exists(chunk_dir)) unlink(chunk_dir, recursive = TRUE, force = TRUE)
  dir.create(chunk_dir, recursive = TRUE, showWarnings = FALSE)

  n <- length(compressed)
  n_chunks <- max(1L, ceiling(n / chunk_size))
  chunk_files <- character(n_chunks)
  for (i in seq_len(n_chunks)) {
    start <- (i - 1L) * chunk_size + 1L
    end <- min(i * chunk_size, n)
    chunk_files[[i]] <- file.path(chunk_dir, sprintf("part-%03d.rdsbin", i))
    writeBin(compressed[start:end], chunk_files[[i]], useBytes = TRUE)
  }

  manifest <- list(
    format = "serialized-r-object-xdr-memCompress-xz-chunked",
    original_path = portable_path(path, config),
    created_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
    chunk_size_mib = config$rds_chunk_size_mib %||% 45,
    n_chunks = n_chunks,
    uncompressed_bytes = length(raw),
    compressed_bytes = length(compressed),
    chunks = lapply(chunk_files, function(cf) list(
      path = portable_path(cf, config),
      bytes = file.info(cf)$size,
      md5 = file_checksum(cf)
    ))
  )
  jsonlite::write_json(manifest, file.path(chunk_dir, "manifest.json"), pretty = TRUE, auto_unbox = TRUE)
  if (file.exists(path)) unlink(path)
  invisible(list(path = path, chunk_dir = chunk_dir, manifest = manifest))
}

read_chunked_rds <- function(path) {
  chunk_dir <- chunk_dir_for_rds(path)
  manifest_path <- file.path(chunk_dir, "manifest.json")
  if (!file.exists(manifest_path)) stop("Chunked RDS manifest not found: ", manifest_path, call. = FALSE)
  manifest <- jsonlite::read_json(manifest_path, simplifyVector = TRUE)
  chunk_paths <- file.path(chunk_dir, basename(manifest$chunks$path))
  if (!all(file.exists(chunk_paths))) {
    # Fall back to paths relative to the project root when the manifest was moved.
    root <- dirname(dirname(dirname(path)))
    chunk_paths <- file.path(root, manifest$chunks$path)
  }
  if (!all(file.exists(chunk_paths))) {
    missing <- chunk_paths[!file.exists(chunk_paths)]
    stop("Missing chunked RDS files: ", paste(missing, collapse = ", "), call. = FALSE)
  }
  compressed <- unlist(Map(function(pth, nbytes) {
    readBin(pth, what = "raw", n = nbytes)
  }, chunk_paths, file.info(chunk_paths)$size), use.names = FALSE)
  unserialize(memDecompress(compressed, type = "xz"))
}

write_model_object <- function(object, path, config = CONFIG) {
  if (isTRUE(config$save_single_rds)) {
    saveRDS(object, path, compress = "xz")
    chunk_dir <- chunk_dir_for_rds(path)
    if (dir.exists(chunk_dir)) unlink(chunk_dir, recursive = TRUE, force = TRUE)
    invisible(list(path = path, mode = "single_rds"))
  } else {
    out <- write_chunked_rds(object, path, config)
    invisible(c(out, list(mode = "chunked_rds")))
  }
}

read_model_object <- function(path) {
  if (file.exists(path)) return(readRDS(path))
  read_chunked_rds(path)
}

model_object_reference <- function(path, config = CONFIG) {
  if (file.exists(path)) {
    return(list(path = path, md5 = file_checksum(path), storage = "single_rds"))
  }
  chunk_dir <- chunk_dir_for_rds(path)
  manifest_path <- file.path(chunk_dir, "manifest.json")
  if (file.exists(manifest_path)) {
    return(list(
      path = path,
      chunk_manifest = manifest_path,
      chunk_dir = chunk_dir,
      manifest_md5 = file_checksum(manifest_path),
      storage = "chunked_rds"
    ))
  }
  list(path = path, md5 = NA_character_, storage = "missing")
}
```

```
716
717 output_file_category <- function(path, config = CONFIG) {
718   rel <- portable_path(path, config)
719   dplyr::case_when(
720     startsWith(rel, "output/diagnostics/") ~ "diagnostics",
721     startsWith(rel, "output/intermediate/") ~ "intermediate",
722     startsWith(rel, "output/tables/") ~ "tables",
723     startsWith(rel, "output/figures/") ~ "figures",
724     TRUE ~ "other"
725   )
726 }
727
728 infer_crosswalk_slug_from_path <- function(path) {
729   base <- basename(path)
730   hit <- stringr::str_match(base, "_(m2|m4|m5|m6)(\\.|_|$)")[, 2]
731   ifelse(is.na(hit), NA_character_, hit)
732 }
733
734 write_output_manifest <- function(config = CONFIG) {
735   config <- finalize_config(config)
736   if (!dir.exists(config$output_dir)) {
737     empty <- tibble::tibble(
738       path = character(), category = character(), extension = character(), bytes = numeric(),
739       md5 = character(), crosswalk_slug = character(), modified_time = character()
740     )
741     readr::write_csv(empty, config$output_manifest_csv)
742     jsonlite::write_json(list(generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"), files = empty),
743                          config$output_manifest_json, pretty = TRUE, auto_unbox = TRUE)
744     return(invisible(empty))
745   }
746   files <- list.files(config$output_dir, recursive = TRUE, full.names = TRUE, all.files = FALSE, no.. = TRUE)
747   files <- files[file.exists(files) & !dir.exists(files)]
748   # Avoid self-recursive checksum churn by writing the manifest after listing but
749   # excluding the previous copy of itself from checksum-based reproducibility checks.
750   info <- file.info(files)
751   out <- tibble::tibble(
752     path = vapply(files, portable_path, character(1), config = config),
753     category = vapply(files, output_file_category, character(1), config = config),
754     extension = tolower(tools::file_ext(files)),
755     bytes = as.numeric(info$size),
756     md5 = vapply(files, file_checksum, character(1)),
757     crosswalk_slug = vapply(files, infer_crosswalk_slug_from_path, character(1)),
758     modified_time = format(info$mtime, "%Y-%m-%d %H:%M:%S %Z")
759   ) %>%
760     dplyr::arrange(category, path)
761   readr::write_csv(out, config$output_manifest_csv)
762   jsonlite::write_json(
763     list(
764       generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
765       n_files = nrow(out),
766       total_bytes = sum(out$bytes, na.rm = TRUE),
767       files = out
768     ),
769     config$output_manifest_json,
770     pretty = TRUE,
771     auto_unbox = TRUE
772   )
773   invisible(out)
774 }
775
776 write_pipeline_manifest <- function(config, checks, stage, sources = list(), extra = list()) {
777   sources <- sanitize_manifest_paths(sources, config)
778   extra <- sanitize_manifest_paths(extra, config)
779   manifest <- list(
780     stage = stage,
781     generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
782     config = config[c(
783       "first_election_year", "diagnostic_first_election_year", "last_election_year",
784       "reference_year", "crosswalk_weight", "crosswalk_missing_weight_policy",
785       "renormalize_crosswalk_weights", "weight_sum_tolerance", "min_retained_vote_share",
786       "require_balanced_panel", "require_no_duplicate_county_years",
787       "require_no_vcov_repair_for_main", "allow_vcov_repair", "allow_failed_diagnostics",
788       "max_abs_vote_identity_error", "max_missing_cz_year_share",
789       "conley_cutoffs_km", "main_se_type", "nw_lag", "dk_lag", "interacted_controls",
790       "standardize_interacted_controls", "export_crosswalk_maps", "run_crosswalk_sensitivity",
791       "save_single_rds", "rds_chunk_size_mib", "county1990_shapefile_path",
792       "nhgis_extract_dir", "output_manifest_csv", "output_manifest_json"
793     )],
794     sources = sources,
795     checks = checks,
796     fatal_failure = has_fatal_failures(checks),
797     extra = extra
798   )
799   manifest <- sanitize_manifest_paths(manifest, config)
800   jsonlite::write_json(manifest, config$pipeline_manifest_json, pretty = TRUE, auto_unbox = TRUE, null = "null")
801   # Keep the standalone output inventory current after each manifest write.
802   tryCatch(write_output_manifest(config), error = function(e) warning(
803     "Could not update output manifest after writing pipeline manifest: ",
804     conditionMessage(e), call. = FALSE
805   ))
```

```
  invisible(manifest)
}


`%||%` <- function(x, y) if (is.null(x)) y else x

standardized_control_name <- function(ctrl) paste0("z_", ctrl)

standardize_baseline_controls <- function(data, controls) {
  out <- data
  summary <- purrr::map_dfr(controls, function(ctrl) {
    vals <- out[[ctrl]]
    mu <- mean(vals, na.rm = TRUE)
    sig <- stats::sd(vals, na.rm = TRUE)
    zname <- standardized_control_name(ctrl)
    out[[zname]] <<- if (is.finite(sig) && sig > 0) (vals - mu) / sig else NA_real_
    tibble::tibble(
      control = ctrl,
      standardized_control = zname,
      n_nonmissing = sum(!is.na(vals)),
      mean = mu,
      sd = sig,
      min = suppressWarnings(min(vals, na.rm = TRUE)),
      max = suppressWarnings(max(vals, na.rm = TRUE))
    )
  })
  list(data = out, summary = summary)
}


read_county_bridge_exceptions <- function(config) {
  empty <- tibble::tibble(
    year = integer(), county_fips = character(), county_name_pattern = character(),
    override_source_decade = integer(), override_county_fips_source = character(),
    issue_class = character(), reason = character(), apply = logical()
  )
  path <- file.path(config$src_dir, "county_bridge_exceptions.csv")
  if (!file.exists(path)) return(empty)
  out <- readr::read_csv(path, show_col_types = FALSE) %>%
    dplyr::mutate(
      year = as.integer(year),
      county_fips = stringr::str_pad(as.character(county_fips), 5, pad = "0"),
      override_county_fips_source = stringr::str_pad(as.character(override_county_fips_source), 5, pad = "0"),
      override_source_decade = as.integer(override_source_decade),
      apply = as.logical(apply)
    ) %>%
    dplyr::filter(apply %in% TRUE)
  dplyr::bind_rows(empty, out)
}

classify_unmatched_county <- function(state_fips, county_fips, county_name, year) {
  state_fips <- stringr::str_pad(as.character(state_fips), 2, pad = "0")
  county_fips <- stringr::str_pad(as.character(county_fips), 5, pad = "0")
  nm <- tolower(as.character(county_name))
  dplyr::case_when(
    state_fips %in% c("02", "15", "72") ~ "excluded_nonmainland",
    county_fips %in% c("04012", "35006", "08014", "46113", "46102", "51683", "51685", "51735") ~ "county_boundary_change_needs_bridge",
    state_fips == "51" & stringr::str_detect(nm, "south boston|clifton forge|bedford|manassas|poquoson") ~
      "county_boundary_change_needs_bridge",
    year < 1972L ~ "pre_1972_diagnostic_only",
    TRUE ~ "unexpected_unmatched"
  )
}

is_adh_mainland_source <- function(state_fips) {
  !stringr::str_pad(as.character(state_fips), 2, pad = "0") %in% c("02", "15", "72")
}

find_first_shapefile <- function(path) {
  if (is.null(path) || !nzchar(path)) return(NA_character_)
  if (file.exists(path) && grepl("\\.shp$", path, ignore.case = TRUE)) return(path)
  if (dir.exists(path)) {
    shps <- list.files(path, pattern = "\\.shp$", recursive = TRUE, full.names = TRUE, ignore.case = TRUE)
    if (length(shps) > 0) return(shps[[1]])
  }
  NA_character_
}

infer_county_fips_from_sf <- function(sf_obj) {
  nm <- names(sf_obj)
  if ("county_fips_1990" %in% nm) return(stringr::str_pad(as.character(sf_obj$county_fips_1990), 5, pad = "0"))
  if ("FIPS" %in% nm) return(stringr::str_pad(as.character(sf_obj$FIPS), 5, pad = "0"))
  if ("GEOID" %in% nm) return(stringr::str_pad(as.character(sf_obj$GEOID), 5, pad = "0"))
  if ("GISJOIN" %in% nm) return(gisjoin_to_fips(sf_obj$GISJOIN))
  if (all(c("STATEFP", "COUNTYFP") %in% nm)) return(paste0(stringr::str_pad(sf_obj$STATEFP, 2, pad = "0"),
    stringr::str_pad(sf_obj$COUNTYFP, 3, pad = "0")))
  if (all(c("STATEA", "COUNTYA") %in% nm)) return(paste0(stringr::str_pad(sf_obj$STATEA, 2, pad = "0"), stringr::str_pad(sf_obj$COUNTYA,
    3, pad = "0")))
  if (all(c("STATE_FIPS", "FIPS") %in% nm)) return(stringr::str_pad(as.character(sf_obj$FIPS), 5, pad = "0"))
```

```
  stop("Could not infer county FIPS from shapefile. Expected county_fips_1990, FIPS, GEOID, GISJOIN, STATEFP+COUNTYFP, STATEA+COUNTYA, or STATE_FIPS+FIPS.")
}

safe_read_county1990_shapefile <- function(config) {
  if (!requireNamespace("sf", quietly = TRUE)) {
    warning("Package 'sf' is not installed; skipping crosswalk support maps.", call. = FALSE)
    return(NULL)
  }

  candidate_paths <- unique(c(
    config$county1990_shapefile_path,
    config$nhgis_extract_dir,
    configured_shapefile_defaults(config)
  ))
  candidate_paths <- candidate_paths[!is.na(candidate_paths) & nzchar(candidate_paths)]
  candidate_paths <- candidate_paths[!grepl("^path/to", candidate_paths, ignore.case = TRUE)]

  shp <- NA_character_
  for (candidate in candidate_paths) {
    shp <- find_first_shapefile(candidate)
    if (!is.na(shp)) break
  }

  if (is.na(shp)) {
    warning(
      "No 1990 county shapefile found. Put co1990p020.shp (and companion files) under ",
      file.path(config$replication_dir, "spatial-data", "counties-1990"),
      ", or set COUNTY1990_SHAPEFILE to a 1990 county .shp file or folder. ",
      "Skipping map rendering but writing non-spatial diagnostics.",
      call. = FALSE
    )
    return(NULL)
  }

  sf_obj <- sf::st_read(shp, quiet = TRUE)
  sf_obj$county_fips_1990 <- infer_county_fips_from_sf(sf_obj)
  sf_obj <- sf_obj %>%
    dplyr::filter(!is.na(county_fips_1990))

  # Many historical county shapefiles store multipart counties/islands as multiple
  # rows with the same FIPS. Dissolve before any diagnostic joins so maps do not
  # trigger unexpected many-to-many warnings or over-represent multipart counties.
  sf_obj <- sf_obj %>%
    dplyr::group_by(county_fips_1990) %>%
    dplyr::summarise(geometry = sf::st_union(geometry), .groups = "drop")

  attr(sf_obj, "source_shapefile") <- shp
  sf_obj
}

haversine_km <- function(lon1, lat1, lon2, lat2) {
  rad <- pi / 180
  dlon <- (lon2 - lon1) * rad
  dlat <- (lat2 - lat1) * rad
  a <- sin(dlat / 2)^2 + cos(lat1 * rad) * cos(lat2 * rad) * sin(dlon / 2)^2
  6371.0088 * 2 * atan2(sqrt(a), sqrt(pmax(0, 1 - a)))
}

summarise_conley_neighbors <- function(panel, cutoffs_km) {
  coords <- panel %>%
    dplyr::distinct(czone, lon, lat) %>%
    dplyr::filter(is.finite(lon), is.finite(lat))
  if (nrow(coords) == 0) return(tibble::tibble())
  dist_mat <- outer(seq_len(nrow(coords)), seq_len(nrow(coords)), Vectorize(function(i, j) {
    haversine_km(coords$lon[[i]], coords$lat[[i]], coords$lon[[j]], coords$lat[[j]])
  }))
  purrr::map_dfr(cutoffs_km, function(cutoff) {
    counts <- rowSums(dist_mat <= cutoff, na.rm = TRUE) - 1L
    tibble::tibble(
      cutoff_km = cutoff,
      n_cz = nrow(coords),
      min_neighbors = min(counts),
      p25_neighbors = as.numeric(stats::quantile(counts, 0.25)),
      median_neighbors = stats::median(counts),
      mean_neighbors = mean(counts),
      p75_neighbors = as.numeric(stats::quantile(counts, 0.75)),
      max_neighbors = max(counts),
      cz_with_zero_neighbors = sum(counts == 0)
    )
  })
}

render_table_pdf <- function(fragment_tex, standalone_tex, output_pdf) {
  input_name <- basename(fragment_tex)
  lines <- c(
    "\\documentclass[11pt]{article}",
    "\\usepackage[letterpaper,landscape,margin=0.7in]{geometry}",
    "\\usepackage{array}",
    "\\usepackage{booktabs}",
```

```
    "\\usepackage{longtable}",
    "\\usepackage{threeparttablex}",
    "\\usepackage{caption}",
    "\\renewcommand{\\arraystretch}{1.08}",
    "\\newcolumntype{L}[1]{>{\\raggedright\\arraybackslash}p{#1}}",
    "\\newcolumntype{C}[1]{>{\\centering\\arraybackslash}p{#1}}",
    "\\begin{document}",
    paste0("\\input{", input_name, "}"),
    "\\end{document}"
  )
  writeLines(lines, standalone_tex)

  tex_engine <- Sys.which("pdflatex")
  if (!nzchar(tex_engine)) {
    warning("No LaTeX engine found (pdflatex not on PATH). Wrote TeX sources but did not render a PDF.")
    return(invisible(NULL))
  }

  oldwd <- getwd()
  on.exit(setwd(oldwd), add = TRUE)
  setwd(dirname(standalone_tex))

  out1 <- system2(tex_engine, c("-interaction=nonstopmode", basename(standalone_tex)), stdout = TRUE, stderr = TRUE)
  out2 <- system2(tex_engine, c("-interaction=nonstopmode", basename(standalone_tex)), stdout = TRUE, stderr = TRUE)
  pdf_path <- file.path(dirname(standalone_tex), paste0(tools::file_path_sans_ext(basename(standalone_tex)), ".pdf"))

  if (!file.exists(pdf_path)) {
    warning("pdflatex ran but no PDF was produced. Last output:\n", paste(c(out1, out2), collapse = "\n"))
    return(invisible(NULL))
  }

  file.copy(pdf_path, output_pdf, overwrite = TRUE)

  sidecars <- file.path(
    dirname(standalone_tex),
    paste0(tools::file_path_sans_ext(basename(standalone_tex)), c(".aux", ".log", ".out", ".toc"))
  )
  unlink(sidecars[file.exists(sidecars)])

  invisible(output_pdf)
}
```

3 02_build_analysis_data.R

```r
# extension-adh2013/src/02_build_analysis_data.R

preflight_ftz_builtup_support <- function(zip_path, source_decades, tolerance = 1e-6,
                                          weight_types = c("m2_weight", "m4_weight", "m5_weight", "m6_weight")) {
  # 1990 is identity-mapped to itself in this project, so M2/M4/M5/M6 support
  # is not meaningful for that decade. Omitting it avoids the misleading
  # n_source_counties = 1 synthetic row seen in earlier diagnostics.
  source_decades <- setdiff(as.integer(source_decades), 1990L)

  purrr::map_dfr(source_decades, function(source_decade) {
    file_name <- paste0("Crosswalk_", source_decade, "_1990.csv")
    source_col <- paste0("gisjoin_", source_decade)
    raw <- readr::read_csv(
      unz(zip_path, file_name),
      col_select = dplyr::all_of(c(source_col, weight_types)),
      show_col_types = FALSE
    ) %>%
      dplyr::mutate(
        gisjoin_source = as.character(.data[[source_col]])
      ) %>%
      dplyr::select(gisjoin_source, dplyr::all_of(weight_types))
    if (source_decade == 2020L) {
      raw <- raw %>%
        dplyr::mutate(gisjoin_source = fips_to_gisjoin(stringr::str_pad(gisjoin_source, 5, pad = "0")))
    }

    raw %>%
      tidyr::pivot_longer(
        cols = dplyr::all_of(weight_types),
        names_to = "weight_type",
        values_to = "weight"
      ) %>%
      dplyr::group_by(gisjoin_source, weight_type) %>%
      dplyr::summarise(
        weight_sum = sum_preserve_all_na(weight),
        n_parts = dplyr::n(),
        n_missing_parts = sum(is.na(weight)),
        .groups = "drop"
      ) %>%
      dplyr::transmute(
        source_decade = as.integer(source_decade),
        gisjoin_source,
        county_fips_source = gisjoin_to_fips(gisjoin_source),
        weight_type,
        weight_sum,
        n_parts,
        n_missing_parts,
        available = !is.na(weight_sum) & weight_sum > tolerance,
        zero_or_undefined_support = is.na(weight_sum) | weight_sum <= tolerance
      )
  })
}

read_ftz_raw_target_rows <- function(source_decade, zip_path, weight_types = c("m2_weight", "m4_weight", "m5_weight", "m6_weight")) {
  if (source_decade == 1990L) return(tibble::tibble())
  file_name <- paste0("Crosswalk_", source_decade, "_1990.csv")
  source_col <- paste0("gisjoin_", source_decade)
  raw <- readr::read_csv(
    unz(zip_path, file_name),
    col_select = dplyr::all_of(c(source_col, "gisjoin_1990", weight_types)),
    show_col_types = FALSE
  ) %>%
    dplyr::mutate(
      source_decade = as.integer(source_decade),
      gisjoin_source = as.character(.data[[source_col]]),
      gisjoin_source = dplyr::if_else(
        source_decade == 2020L,
        fips_to_gisjoin(stringr::str_pad(gisjoin_source, 5, pad = "0")),
        gisjoin_source
      ),
      county_fips_source = gisjoin_to_fips(gisjoin_source),
      county_fips_1990 = gisjoin_to_fips(gisjoin_1990)
    )
  raw %>%
    dplyr::select(source_decade, gisjoin_source, county_fips_source, gisjoin_1990, county_fips_1990, dplyr::all_of(weight_types))
}

build_target_weight_support_diagnostics <- function(zip_path, source_decades, county_pres, cz_lookup,
                                                    selected_crosswalks, tolerance = 1e-6,
                                                    adh_czones = NULL) {
  weight_types <- c("m2_weight", "m4_weight", "m5_weight", "m6_weight")
  source_support <- preflight_ftz_builtup_support(zip_path, source_decades, tolerance, weight_types)

  raw_targets <- purrr::map_dfr(setdiff(as.integer(source_decades), 1990L), read_ftz_raw_target_rows,
                                zip_path = zip_path, weight_types = weight_types)
  support_long <- source_support %>%
    dplyr::select(source_decade, gisjoin_source, county_fips_source, weight_type, weight_sum, available)
```

```
88
89    # Attach election-year vote mass so the map can indicate which 1990 counties/CZs
90    # actually receive votes from source counties whose M2/M4/M5/M6 support is unavailable.
91    source_votes <- county_pres %>%
92      dplyr::select(year, source_decade, gisjoin_source, county_fips, state_fips, state, county_name, twoparty_votes)
93
94    # Use the selected/fallback crosswalk weights as the vote-mass allocator for the
95    # demonstration map. A target is flagged only if the selected/fallback crosswalk
96    # sends positive vote mass to that 1990 target from a source county where the
97    # diagnostic weight type is unavailable. This avoids over-flagging ordinary
98    # zero-weight target rows in the raw many-to-many FTZ crosswalk.
99    selected_targets <- selected_crosswalks %>%
100     dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > tolerance) %>%
101     dplyr::select(source_decade, gisjoin_source, county_fips_1990,
102                   selected_crosswalk_weight = crosswalk_weight) %>%
103     dplyr::distinct()
104
105   target_long <- raw_targets %>%
106     tidyr::pivot_longer(
107       cols = dplyr::all_of(weight_types),
108       names_to = "weight_type",
109       values_to = "raw_weight_for_type"
110     ) %>%
111     dplyr::left_join(support_long, by = c("source_decade", "gisjoin_source", "county_fips_source", "weight_type")) %>%
112     dplyr::left_join(source_votes, by = c("source_decade", "gisjoin_source"), relationship = "many-to-many") %>%
113     dplyr::left_join(selected_targets, by = c("source_decade", "gisjoin_source", "county_fips_1990"), relationship = "many-to-many") %>%
114     dplyr::mutate(
115       unavailable = !dplyr::coalesce(available, FALSE),
116       selected_crosswalk_weight = dplyr::coalesce(selected_crosswalk_weight, 0),
117       received_vote_mass_proxy = twoparty_votes * selected_crosswalk_weight,
118       receives_positive_vote_mass = !is.na(received_vote_mass_proxy) & received_vote_mass_proxy > 0,
119       affected_received_vote_mass_proxy = dplyr::if_else(
120         unavailable & receives_positive_vote_mass,
121         received_vote_mass_proxy,
122         0
123       )
124     )
125
126   county_diag <- target_long %>%
127     dplyr::filter(!is.na(county_fips_1990), !is.na(year)) %>%
128     dplyr::group_by(county_fips_1990, weight_type) %>%
129     dplyr::summarise(
130       receives_unavailable_source = any(affected_received_vote_mass_proxy > 0, na.rm = TRUE),
131       n_unavailable_source_counties = dplyr::n_distinct(county_fips_source[affected_received_vote_mass_proxy > 0]),
132       unavailable_source_vote_mass_proxy = sum(affected_received_vote_mass_proxy, na.rm = TRUE),
133       total_received_vote_mass_proxy = sum(received_vote_mass_proxy, na.rm = TRUE),
134       share_received_from_unavailable_sources = dplyr::if_else(
135         total_received_vote_mass_proxy > 0,
136         unavailable_source_vote_mass_proxy / total_received_vote_mass_proxy,
137         NA_real_
138       ),
139       .groups = "drop"
140     ) %>%
141     dplyr::left_join(cz_lookup %>% dplyr::select(county_fips_1990, czone), by = "county_fips_1990")
142
143   if (!is.null(adh_czones)) {
144     adh_czones <- unique(as.integer(adh_czones))
145     county_diag <- county_diag %>% dplyr::filter(czone %in% adh_czones)
146   }
147
148   cz_diag <- county_diag %>%
149     dplyr::filter(!is.na(czone)) %>%
150     dplyr::group_by(czone, weight_type) %>%
151     dplyr::summarise(
152       receives_unavailable_source = any(receives_unavailable_source %in% TRUE, na.rm = TRUE),
153       n_1990_counties_receiving_unavailable_source = sum(receives_unavailable_source %in% TRUE, na.rm = TRUE),
154       n_unavailable_source_counties = sum(n_unavailable_source_counties, na.rm = TRUE),
155       unavailable_source_vote_mass_proxy = sum(unavailable_source_vote_mass_proxy, na.rm = TRUE),
156       total_received_vote_mass_proxy = sum(total_received_vote_mass_proxy, na.rm = TRUE),
157       share_received_from_unavailable_sources = dplyr::if_else(
158         total_received_vote_mass_proxy > 0,
159         unavailable_source_vote_mass_proxy / total_received_vote_mass_proxy,
160         NA_real_
161       ),
162       .groups = "drop"
163     )
164
165   list(source_support = source_support, county_diag = county_diag, cz_diag = cz_diag)
166 }
167
168 build_analysis_data <- function(config = CONFIG, stop_on_fatal = TRUE) {
169   load_required_packages(config)
170   config <- finalize_config(config)
171
172   adh_stack_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china.dta")
173   adh_long_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china_long.dta")
174   cz_path <- file.path(config$replication_dir, "cz-data", "cz-198090.xls")
175   centroid_path <- file.path(
176     config$replication_dir, "ftz2024", "crosswalks", "County-CD-centroid-lat-lon",
177     "lat_lon_coordinates_county_csv", "counties_1990_xy.csv"
```

```
  )
  ftz_zip <- file.path(
    config$replication_dir, "ftz2024", "crosswalks", "CountyToCounty", "1990", "1990_csv.zip"
  )
  aa_path <- file.path(
    config$replication_dir, "outcomes-data", "aa2021-nospatial",
    "County_Level_US_Elections_Data",
    "dataverse_shareable_presidential_county_returns_1868_2020.Rdata"
  )

  message("Reading ADH baseline and exposure data...")
  adh_stack <- haven::read_dta(adh_stack_path)
  adh_long <- haven::read_dta(adh_long_path)

  adh_baseline <- adh_stack %>%
    dplyr::filter(yr == 1990) %>%
    dplyr::transmute(
      czone = as.integer(czone),
      statefip = as.integer(statefip),
      dplyr::across(dplyr::all_of(config$baseline_controls), as.numeric)
    ) %>%
    dplyr::distinct()

  adh_exposure <- adh_long %>%
    dplyr::transmute(
      czone = as.integer(czone),
      statefip = as.integer(statefip),
      exposure = as.numeric(d_tradeusch_pw),
      adh_weight = as.numeric(timepwt48)
    ) %>%
    dplyr::distinct()

  adh_sample <- adh_baseline %>%
    dplyr::inner_join(adh_exposure, by = c("czone", "statefip"))

  adh_duplicate_czones <- adh_sample %>%
    dplyr::count(czone, name = "rows_per_czone") %>%
    dplyr::filter(rows_per_czone > 1)

  std_controls <- standardize_baseline_controls(adh_sample, config$baseline_controls)
  adh_sample <- std_controls$data
  readr::write_csv(std_controls$summary, config$baseline_control_summary_csv)

  control_corr <- stats::cor(
    adh_sample[, config$baseline_controls, drop = FALSE],
    use = "pairwise.complete.obs"
  )
  control_corr_long <- as.data.frame(as.table(control_corr), stringsAsFactors = FALSE) %>%
    tibble::as_tibble() %>%
    dplyr::rename(control_1 = Var1, control_2 = Var2, correlation = Freq)
  readr::write_csv(control_corr_long, config$baseline_control_correlation_csv)

  message("Reading 1990 county-to-CZ lookup and county centroids...")
  cz_lookup <- readxl::read_xls(cz_path) %>%
    dplyr::transmute(
      county_fips_1990 = stringr::str_pad(as.character(`County FIPS Code`), width = 5, side = "left", pad = "0"),
      czone = as.integer(CZ90),
      county_name_1990 = as.character(`County name`),
      pop1990 = as.numeric(`Population 1990`)
    )

  cz_lookup_duplicate_count <- cz_lookup %>%
    dplyr::count(county_fips_1990, name = "rows_per_1990_county") %>%
    dplyr::filter(rows_per_1990_county > 1) %>%
    nrow()
  cz_lookup_multi_czone_count <- cz_lookup %>%
    dplyr::distinct(county_fips_1990, czone) %>%
    dplyr::count(county_fips_1990, name = "czones_per_1990_county") %>%
    dplyr::filter(czones_per_1990_county > 1) %>%
    nrow()

  county_centroids <- readr::read_csv(centroid_path, show_col_types = FALSE) %>%
    dplyr::transmute(
      county_fips_1990 = gisjoin_to_fips(gisjoin_1990),
      lon = as.numeric(centroid_x),
      lat = as.numeric(centroid_y)
    )

  cz_centroids <- cz_lookup %>%
    dplyr::inner_join(county_centroids, by = "county_fips_1990") %>%
    dplyr::group_by(czone) %>%
    dplyr::mutate(pop_w = pop1990 / sum(pop1990, na.rm = TRUE)) %>%
    dplyr::summarise(
      lon = stats::weighted.mean(lon, pop_w, na.rm = TRUE),
      lat = stats::weighted.mean(lat, pop_w, na.rm = TRUE),
      pop1990 = sum(pop1990, na.rm = TRUE),
      counties_in_cz = dplyr::n(),
      .groups = "drop"
    ) %>%
    dplyr::inner_join(adh_sample %>% dplyr::distinct(czone), by = "czone")
```

```
  readr::write_csv(cz_centroids, config$cz_centroids_csv)

  preflight_support <- preflight_ftz_builtup_support(
    zip_path = ftz_zip,
    source_decades = config$source_decades,
    tolerance = config$weight_sum_tolerance
  )
  preflight_summary <- preflight_support %>%
    dplyr::group_by(source_decade, weight_type) %>%
    dplyr::summarise(
      n_source_counties = dplyr::n(),
      n_zero_or_undefined_support = sum(zero_or_undefined_support),
      .groups = "drop"
    )
  readr::write_csv(preflight_summary, config$crosswalk_preflight_summary_csv)
  readr::write_csv(
    preflight_support %>% dplyr::filter(zero_or_undefined_support),
    config$crosswalk_preflight_counties_csv
  )

  builtup_preflight <- preflight_support %>%
    dplyr::filter(weight_type %in% c("m5_weight", "m6_weight"))
  if (any(builtup_preflight$zero_or_undefined_support)) {
    summary_msg <- preflight_summary %>%
      dplyr::filter(weight_type %in% c("m5_weight", "m6_weight"), n_zero_or_undefined_support > 0) %>%
      dplyr::mutate(piece = paste0(weight_type, " ", source_decade, ": ", n_zero_or_undefined_support)) %>%
      dplyr::pull(piece) %>%
      paste(collapse = "; ")
    warning(
      "FTZ built-up-support preflight: strict M5/M6 weighting leaves some source counties ",
      "with zero or undefined built-up support, so a strict M5/M6 run cannot produce the balanced ",
      "ADH mainland CZ panel. Current policy is '", config$crosswalk_missing_weight_policy, "'. ",
      "Zero/undefined support counts: ", summary_msg, ". See ",
      config$crosswalk_preflight_summary_csv,
      call. = FALSE
    )
  }

  message("Reading FTZ county crosswalks with ", config$crosswalk_weight, "...")
  crosswalks <- purrr::map_dfr(
    config$source_decades,
    ~ read_ftz_crosswalk(
      source_decade = .x,
      zip_path = ftz_zip,
      county_fips_1990_identity = cz_lookup$county_fips_1990,
      weight_col = config$crosswalk_weight,
      missing_weight_policy = config$crosswalk_missing_weight_policy,
      renormalize_weights = config$renormalize_crosswalk_weights,
      tolerance = config$weight_sum_tolerance
    )
  )

  crosswalk_source_diagnostics <- crosswalks %>%
    dplyr::group_by(source_decade, gisjoin_source, county_fips_source) %>%
    dplyr::summarise(
      weight_sum_raw = dplyr::first(weight_sum_raw),
      selected_weight_sum_raw = dplyr::first(selected_weight_sum_raw),
      crosswalk_weight_sum = dplyr::first(crosswalk_weight_sum),
      raw_weight_undefined = dplyr::first(raw_weight_undefined),
      raw_weight_sum_valid = dplyr::first(raw_weight_sum_valid),
      raw_weight_requires_policy = dplyr::first(raw_weight_requires_policy),
      fallback_used = any(fallback_used),
      fallback_policy = if (any(fallback_policy != "none")) {
        dplyr::first(fallback_policy[fallback_policy != "none"])
      } else {
        "none"
      },
      n_targets = dplyr::n(),
      positive_targets = sum(!is.na(crosswalk_weight) & crosswalk_weight > 0),
      missing_raw_weight_parts = sum(raw_weight_missing_parts),
      final_weight_sum_ok = !is.na(dplyr::first(crosswalk_weight_sum)) &
        abs(dplyr::first(crosswalk_weight_sum) - 1) <= config$weight_sum_tolerance,
      .groups = "drop"
    )

  readr::write_csv(crosswalk_source_diagnostics, config$crosswalk_diagnostics_csv)
  readr::write_csv(
    crosswalk_source_diagnostics %>%
      dplyr::filter(raw_weight_undefined | !final_weight_sum_ok),
    config$crosswalk_zero_support_csv
  )
  readr::write_csv(
    crosswalks %>%
      dplyr::select(
        source_decade, county_fips_source, county_fips_1990, raw_weight,
        weight_sum_raw, crosswalk_weight, crosswalk_weight_sum,
        raw_weight_undefined, raw_weight_sum_valid, raw_weight_requires_policy,
        fallback_used, fallback_policy, weight_source
      ),
    config$crosswalk_row_diagnostics_csv
```

```
)

message("Reading and validating Amlani and Algara (2021) presidential county returns...")
aa_pres <- read_aa_rdata_object(aa_path, object_name = "pres_elections_release")

county_pres_raw <- aa_pres %>%
  dplyr::transmute(
    year = as.integer(election_year),
    county_fips = stringr::str_pad(as.character(fips), width = 5, side = "left", pad = "0"),
    state_fips = stringr::str_pad(as.character(sfips), width = 2, side = "left", pad = "0"),
    state = as.character(state),
    county_name = as.character(county_name),
    rep_votes = as.numeric(republican_raw_votes),
    dem_votes = as.numeric(democratic_raw_votes),
    twoparty_votes = as.numeric(pres_raw_county_vote_totals_two_party),
    total_votes = as.numeric(raw_county_vote_totals),
    complete_case = as.integer(complete_county_cases)
  ) %>%
  dplyr::filter(year %in% config$diagnostic_event_years)

exact_duplicate_rows <- nrow(county_pres_raw) - nrow(dplyr::distinct(county_pres_raw))
county_pres_distinct <- county_pres_raw %>% dplyr::distinct()

duplicate_county_years <- county_pres_distinct %>%
  dplyr::count(year, county_fips, name = "rows_per_county_year") %>%
  dplyr::filter(rows_per_county_year > 1) %>%
  dplyr::left_join(county_pres_distinct, by = c("year", "county_fips"))
readr::write_csv(duplicate_county_years, config$outcome_duplicate_report_csv)

county_pres_validation <- county_pres_distinct %>%
  dplyr::mutate(
    valid_fips = !is.na(county_fips) & stringr::str_detect(county_fips, "^[0-9]{5}$"),
    nonmissing_votes = !is.na(rep_votes) & !is.na(dem_votes) & !is.na(twoparty_votes),
    nonnegative_votes = rep_votes >= 0 & dem_votes >= 0 & twoparty_votes >= 0 &
      (is.na(total_votes) | total_votes >= 0),
    twoparty_matches = abs((rep_votes + dem_votes) - twoparty_votes) <= 1e-6,
    usable_for_bridge = valid_fips & nonmissing_votes & nonnegative_votes &
      twoparty_matches & twoparty_votes > 0 & complete_case == 1L
  )
readr::write_csv(county_pres_validation, config$outcome_validation_csv)

invalid_fips_count <- county_pres_validation %>%
  dplyr::filter(!valid_fips) %>%
  nrow()
max_vote_identity_error <- county_pres_validation %>%
  dplyr::filter(nonmissing_votes) %>%
  dplyr::mutate(abs_vote_identity_error = abs((rep_votes + dem_votes) - twoparty_votes)) %>%
  dplyr::pull(abs_vote_identity_error) %>%
  max(na.rm = TRUE)

bridge_exceptions <- read_county_bridge_exceptions(config)
if (nrow(bridge_exceptions) > 0) {
  readr::write_csv(bridge_exceptions, file.path(config$diagnostic_dir, "county_bridge_exceptions_applied.csv"))
}

county_pres <- county_pres_validation %>%
  dplyr::filter(usable_for_bridge) %>%
  dplyr::mutate(source_decade_default = year_to_source_decade(year)) %>%
  dplyr::left_join(
    bridge_exceptions %>%
      dplyr::select(year, county_fips, override_source_decade, override_county_fips_source, exception_issue_class = issue_class,
        ↪  exception_reason = reason),
    by = c("year", "county_fips")
  ) %>%
  dplyr::mutate(
    source_decade = dplyr::coalesce(override_source_decade, source_decade_default),
    county_fips_for_crosswalk = dplyr::coalesce(override_county_fips_source, county_fips),
    gisjoin_source = fips_to_gisjoin(county_fips_for_crosswalk),
    source_scope = dplyr::if_else(is_adh_mainland_source(state_fips), "adh_mainland_eligible", "excluded_nonmainland"),
    bridge_exception_applied = !is.na(override_county_fips_source) | !is.na(override_source_decade)
  ) %>%
  dplyr::select(
    year, county_fips, county_fips_for_crosswalk, state_fips, state, county_name, rep_votes, dem_votes,
    twoparty_votes, total_votes, source_decade, source_decade_default, gisjoin_source, source_scope,
    bridge_exception_applied, exception_issue_class, exception_reason
  )

target_support <- build_target_weight_support_diagnostics(
  zip_path = ftz_zip,
  source_decades = config$source_decades,
  county_pres = county_pres,
  cz_lookup = cz_lookup,
  selected_crosswalks = crosswalks,
  tolerance = config$weight_sum_tolerance,
  adh_czones = adh_sample$czone
)
readr::write_csv(target_support$source_support, config$all_weight_support_csv)
readr::write_csv(target_support$county_diag, config$target_weight_support_county_csv)
readr::write_csv(target_support$cz_diag, config$target_weight_support_cz_csv)

```

```
source_vote_denominators <- county_pres %>%
  dplyr::group_by(year) %>%
  dplyr::summarise(source_two_party_votes_prejoin = sum(twoparty_votes, na.rm = TRUE), .groups = "drop")

positive_target_summary <- crosswalks %>%
  dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > config$weight_sum_tolerance) %>%
  dplyr::left_join(cz_lookup %>% dplyr::select(county_fips_1990, czone), by = "county_fips_1990") %>%
  dplyr::group_by(source_decade, gisjoin_source) %>%
  dplyr::summarise(
    positive_target_1990_counties = paste(sort(unique(county_fips_1990)), collapse = ";"),
    positive_target_czones = paste(sort(unique(stats::na.omit(czone))), collapse = ";"),
    positive_target_count = dplyr::n_distinct(county_fips_1990),
    positive_target_cz_count = dplyr::n_distinct(czone, na.rm = TRUE),
    .groups = "drop"
  )

# Source-level issue report. Do not filter on row-level crosswalk_weight <= 0:
# valid source counties naturally have many zero-weight target rows in the raw
# many-to-many FTZ crosswalk. A source is an issue only if its selected raw
# weight is undefined/invalid, its final weight sum is invalid, or it has no
# positive final target weight after the chosen fallback policy.
crosswalk_source_issue_rows <- crosswalk_source_diagnostics %>%
  dplyr::mutate(
    no_positive_final_target = positive_targets == 0,
    final_weight_invalid = is.na(crosswalk_weight_sum) |
      abs(crosswalk_weight_sum - 1) > config$weight_sum_tolerance
  ) %>%
  dplyr::filter(raw_weight_undefined | raw_weight_requires_policy | final_weight_invalid | no_positive_final_target) %>%
  dplyr::left_join(positive_target_summary, by = c("source_decade", "gisjoin_source"))

crosswalk_issue_vote_report <- county_pres %>%
  dplyr::left_join(
    crosswalk_source_issue_rows,
    by = c("source_decade", "gisjoin_source")
  ) %>%
  dplyr::filter(
    raw_weight_undefined %in% TRUE |
      raw_weight_requires_policy %in% TRUE |
      final_weight_invalid %in% TRUE |
      no_positive_final_target %in% TRUE
  ) %>%
  dplyr::left_join(source_vote_denominators, by = "year") %>%
  dplyr::mutate(
    vote_share_of_year = twoparty_votes / source_two_party_votes_prejoin,
    source_issue_class = dplyr::case_when(
      no_positive_final_target %in% TRUE & fallback_used %in% TRUE ~ paste0(
        "selected_", weight_slug(config$crosswalk_weight), "_unavailable_", fallback_policy, "_used_but_no_positive_target"
      ),
      no_positive_final_target %in% TRUE & raw_weight_requires_policy %in% TRUE ~ paste0(
        "selected_", weight_slug(config$crosswalk_weight), "_unavailable_no_fallback"
      ),
      no_positive_final_target %in% TRUE ~ classify_unmatched_county(state_fips, county_fips, county_name, year),
      raw_weight_requires_policy %in% TRUE & fallback_used %in% TRUE ~ paste0(
        "selected_", weight_slug(config$crosswalk_weight), "_unavailable_", fallback_policy, "_used"
      ),
      raw_weight_requires_policy %in% TRUE & !(fallback_used %in% TRUE) ~ paste0(
        "selected_", weight_slug(config$crosswalk_weight), "_unavailable_no_fallback"
      ),
      final_weight_invalid %in% TRUE ~ "final_weight_invalid",
      TRUE ~ "reported_crosswalk_issue"
    )
  ) %>%
  dplyr::select(
    year, source_decade, source_decade_default, source_scope, source_issue_class,
    state_fips, state, county_fips, county_fips_for_crosswalk, county_name,
    bridge_exception_applied, exception_issue_class, exception_reason,
    twoparty_votes, source_two_party_votes_prejoin, vote_share_of_year,
    weight_sum_raw, selected_weight_sum_raw, crosswalk_weight_sum,
    raw_weight_undefined, raw_weight_sum_valid, raw_weight_requires_policy,
    fallback_used, fallback_policy, no_positive_final_target, final_weight_invalid,
    n_targets, positive_targets, positive_target_1990_counties, positive_target_czones,
    positive_target_count, positive_target_cz_count
  ) %>%
  dplyr::arrange(year, source_issue_class, state_fips, county_fips)
readr::write_csv(crosswalk_issue_vote_report, config$crosswalk_issue_vote_report_csv)

national_pre <- county_pres %>%
  dplyr::group_by(year) %>%
  dplyr::summarise(
    scope = "pre_bridge",
    rep_votes = sum(rep_votes, na.rm = TRUE),
    dem_votes = sum(dem_votes, na.rm = TRUE),
    twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
    total_votes = sum(total_votes, na.rm = TRUE),
    source_counties = dplyr::n_distinct(county_fips),
    .groups = "drop"
  )

state_pre <- county_pres %>%
  dplyr::group_by(year, state_fips, state) %>%
```

```
    dplyr::summarise(
      scope = "pre_bridge",
      rep_votes = sum(rep_votes, na.rm = TRUE),
      dem_votes = sum(dem_votes, na.rm = TRUE),
      twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
      total_votes = sum(total_votes, na.rm = TRUE),
      source_counties = dplyr::n_distinct(county_fips),
      .groups = "drop"
    )

  message("Bridging county returns to 1990 counties...")
  county_bridged <- county_pres %>%
    dplyr::left_join(
      crosswalks %>%
        dplyr::select(
          source_decade, gisjoin_source, county_fips_1990, raw_weight,
          crosswalk_weight, raw_weight_undefined, raw_weight_sum_valid,
          raw_weight_requires_policy, fallback_used, fallback_policy, weight_source
        ),
      by = c("source_decade", "gisjoin_source"),
      relationship = "many-to-many"
    )

  unmatched_source_counties <- county_bridged %>%
    dplyr::group_by(year, source_decade, source_decade_default, county_fips, county_fips_for_crosswalk, state_fips, state, county_name,
      ↪  source_scope, bridge_exception_applied, exception_issue_class, exception_reason) %>%
    dplyr::summarise(
      twoparty_votes = dplyr::first(twoparty_votes),
      has_positive_crosswalk_weight = any(!is.na(crosswalk_weight) & crosswalk_weight > 0),
      raw_weight_undefined = any(raw_weight_undefined %in% TRUE, na.rm = TRUE),
      fallback_used = any(fallback_used %in% TRUE, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    dplyr::left_join(source_vote_denominators, by = "year") %>%
    dplyr::mutate(
      vote_share_of_year = twoparty_votes / source_two_party_votes_prejoin,
      source_issue_class = classify_unmatched_county(state_fips, county_fips, county_name, year)
    ) %>%
    dplyr::filter(!has_positive_crosswalk_weight)
  readr::write_csv(unmatched_source_counties, config$unmatched_source_counties_csv)
  readr::write_csv(
    unmatched_source_counties %>%
      dplyr::count(year, source_scope, source_issue_class, name = "unmatched_source_counties") %>%
      dplyr::left_join(
        unmatched_source_counties %>%
          dplyr::group_by(year, source_scope, source_issue_class) %>%
          dplyr::summarise(unmatched_two_party_votes = sum(twoparty_votes, na.rm = TRUE), .groups = "drop"),
        by = c("year", "source_scope", "source_issue_class")
      ),
    config$county_bridge_classification_csv
  )

  original_by_year <- county_pres %>%
    dplyr::group_by(year) %>%
    dplyr::summarise(
      source_counties = dplyr::n_distinct(county_fips),
      source_two_party_votes_prejoin = sum(twoparty_votes, na.rm = TRUE),
      source_rep_votes_prejoin = sum(rep_votes, na.rm = TRUE),
      source_dem_votes_prejoin = sum(dem_votes, na.rm = TRUE),
      .groups = "drop"
    )

  naive_postjoin_denominator <- county_bridged %>%
    dplyr::group_by(year) %>%
    dplyr::summarise(
      source_two_party_votes_after_join_naive_for_diagnostic_only = sum(twoparty_votes, na.rm = TRUE),
      .groups = "drop"
    )

  bridged_by_year <- county_bridged %>%
    dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
    dplyr::group_by(year) %>%
    dplyr::summarise(
      matched_source_counties = dplyr::n_distinct(county_fips),
      bridged_two_party_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
      bridged_rep_votes = sum(rep_votes * crosswalk_weight, na.rm = TRUE),
      bridged_dem_votes = sum(dem_votes * crosswalk_weight, na.rm = TRUE),
      fallback_source_counties = dplyr::n_distinct(county_fips[fallback_used %in% TRUE]),
      .groups = "drop"
    )

  original_by_scope <- county_pres %>%
    dplyr::group_by(year, source_scope) %>%
    dplyr::summarise(
      source_counties = dplyr::n_distinct(county_fips),
      source_two_party_votes_prejoin = sum(twoparty_votes, na.rm = TRUE),
      .groups = "drop"
    )
  bridged_by_scope <- county_bridged %>%
    dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
```

```
    dplyr::group_by(year, source_scope) %>%
    dplyr::summarise(
      matched_source_counties = dplyr::n_distinct(county_fips),
      bridged_two_party_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
      fallback_source_counties = dplyr::n_distinct(county_fips[fallback_used %in% TRUE]),
      .groups = "drop"
    )
  retention_by_scope <- original_by_scope %>%
    dplyr::left_join(bridged_by_scope, by = c("year", "source_scope")) %>%
    dplyr::mutate(
      matched_source_counties = dplyr::coalesce(matched_source_counties, 0L),
      bridged_two_party_votes = dplyr::coalesce(bridged_two_party_votes, 0),
      fallback_source_counties = dplyr::coalesce(fallback_source_counties, 0L),
      unmatched_source_counties = source_counties - matched_source_counties,
      retained_vote_share = bridged_two_party_votes / source_two_party_votes_prejoin
    )
  readr::write_csv(retention_by_scope, config$mainland_retention_csv)

  mainland_retention_wide <- retention_by_scope %>%
    dplyr::select(year, source_scope, source_two_party_votes_prejoin, bridged_two_party_votes, retained_vote_share) %>%
    tidyr::pivot_wider(
      names_from = source_scope,
      values_from = c(source_two_party_votes_prejoin, bridged_two_party_votes, retained_vote_share),
      names_sep = "_"
    )

  bridge_diagnostics <- original_by_year %>%
    dplyr::left_join(naive_postjoin_denominator, by = "year") %>%
    dplyr::left_join(bridged_by_year, by = "year") %>%
    dplyr::left_join(mainland_retention_wide, by = "year") %>%
    dplyr::mutate(
      matched_source_counties = dplyr::coalesce(matched_source_counties, 0L),
      unmatched_source_counties = source_counties - matched_source_counties,
      retained_vote_share_correct = bridged_two_party_votes / source_two_party_votes_prejoin,
      retained_vote_share = retained_vote_share_correct,
      retained_vote_share_adh_mainland_eligible = retained_vote_share_adh_mainland_eligible
    )
  readr::write_csv(bridge_diagnostics, config$bridge_diagnostics_csv)

  national_post <- county_bridged %>%
    dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
    dplyr::group_by(year) %>%
    dplyr::summarise(
      scope = "post_bridge",
      rep_votes = sum(rep_votes * crosswalk_weight, na.rm = TRUE),
      dem_votes = sum(dem_votes * crosswalk_weight, na.rm = TRUE),
      twoparty_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
      total_votes = sum(total_votes * crosswalk_weight, na.rm = TRUE),
      source_counties = dplyr::n_distinct(county_fips),
      .groups = "drop"
    )
  readr::write_csv(dplyr::bind_rows(national_pre, national_post), config$national_vote_totals_csv)

  state_post <- county_bridged %>%
    dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
    dplyr::group_by(year, state_fips, state) %>%
    dplyr::summarise(
      scope = "post_bridge",
      rep_votes = sum(rep_votes * crosswalk_weight, na.rm = TRUE),
      dem_votes = sum(dem_votes * crosswalk_weight, na.rm = TRUE),
      twoparty_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
      total_votes = sum(total_votes * crosswalk_weight, na.rm = TRUE),
      source_counties = dplyr::n_distinct(county_fips),
      .groups = "drop"
    )
  readr::write_csv(dplyr::bind_rows(state_pre, state_post), config$state_vote_totals_csv)

  county90_panel <- county_bridged %>%
    dplyr::filter(!is.na(county_fips_1990), !is.na(crosswalk_weight), crosswalk_weight > 0) %>%
    dplyr::mutate(
      rep_votes = rep_votes * crosswalk_weight,
      dem_votes = dem_votes * crosswalk_weight,
      twoparty_votes = twoparty_votes * crosswalk_weight,
      total_votes = total_votes * crosswalk_weight
    ) %>%
    dplyr::group_by(year, county_fips_1990) %>%
    dplyr::summarise(
      rep_votes = sum(rep_votes, na.rm = TRUE),
      dem_votes = sum(dem_votes, na.rm = TRUE),
      twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
      total_votes = sum(total_votes, na.rm = TRUE),
      .groups = "drop"
    )

  message("Aggregating bridged county votes to 1990 commuting zones...")
  cz_panel <- county90_panel %>%
    dplyr::inner_join(cz_lookup, by = "county_fips_1990") %>%
    dplyr::group_by(czone, year) %>%
    dplyr::summarise(
      rep_votes = sum(rep_votes, na.rm = TRUE),
```

```
    dem_votes = sum(dem_votes, na.rm = TRUE),
    twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
    total_votes = sum(total_votes, na.rm = TRUE),
    counties_contributing = dplyr::n(),
    .groups = "drop"
  ) %>%
  dplyr::mutate(rep_margin = (rep_votes - dem_votes) / twoparty_votes)

make_panel <- function(years) {
  tidyr::expand_grid(czone = sort(unique(adh_sample$czone)), year = years) %>%
    dplyr::mutate(election_index = match(year, sort(unique(years)))) %>%
    dplyr::left_join(cz_panel, by = c("czone", "year")) %>%
    dplyr::inner_join(adh_sample, by = "czone") %>%
    dplyr::inner_join(cz_centroids, by = "czone") %>%
    dplyr::arrange(czone, year)
}

analysis_panel_full <- make_panel(config$event_years)
diagnostic_panel_full <- make_panel(config$diagnostic_event_years)

analysis_panel <- analysis_panel_full %>%
  dplyr::filter(!is.na(.data[[config$outcome_var]]))
diagnostic_panel <- diagnostic_panel_full %>%
  dplyr::filter(!is.na(.data[[config$outcome_var]]))

missing_cz_years <- dplyr::bind_rows(
  analysis_panel_full %>%
    dplyr::filter(is.na(.data[[config$outcome_var]])) %>%
    dplyr::transmute(sample = "main_1972_start", czone, year),
  diagnostic_panel_full %>%
    dplyr::filter(is.na(.data[[config$outcome_var]])) %>%
    dplyr::transmute(sample = "diagnostic_1952_start", czone, year)
)
readr::write_csv(missing_cz_years, config$missing_cz_years_csv)

panel_coverage <- dplyr::bind_rows(
  analysis_panel_full %>% dplyr::mutate(sample = "main_1972_start"),
  diagnostic_panel_full %>% dplyr::mutate(sample = "diagnostic_1952_start")
) %>%
  dplyr::mutate(has_outcome = !is.na(.data[[config$outcome_var]])) %>%
  dplyr::group_by(sample, year) %>%
  dplyr::summarise(
    n_cz_total = dplyr::n(),
    n_cz_with_outcome = sum(has_outcome),
    missing_cz_years = n_cz_total - n_cz_with_outcome,
    share_cz_with_outcome = mean(has_outcome),
    .groups = "drop"
  )
readr::write_csv(panel_coverage, config$panel_coverage_csv)

saveRDS(analysis_panel, config$analysis_panel_rds)
saveRDS(analysis_panel_full, config$analysis_panel_full_rds)
saveRDS(diagnostic_panel, config$diagnostic_panel_rds)
saveRDS(diagnostic_panel_full, config$diagnostic_panel_full_rds)

main_expected_n <- length(unique(adh_sample$czone)) * length(config$event_years)
main_missing_n <- nrow(analysis_panel_full) - nrow(analysis_panel)
main_missing_share <- if (main_expected_n > 0) main_missing_n / main_expected_n else NA_real_
adh_duplicate_czone_count <- nrow(adh_duplicate_czones)
min_main_retained <- bridge_diagnostics %>%
  dplyr::filter(year %in% config$event_years) %>%
  dplyr::pull(retained_vote_share) %>%
  min(na.rm = TRUE)
duplicate_count <- duplicate_county_years %>%
  dplyr::distinct(year, county_fips) %>%
  nrow()
twoparty_mismatch_count <- county_pres_validation %>%
  dplyr::filter(complete_case == 1L, valid_fips, nonmissing_votes, !twoparty_matches) %>%
  nrow()
nonnegative_failure_count <- county_pres_validation %>%
  dplyr::filter(complete_case == 1L, valid_fips, nonmissing_votes, !nonnegative_votes) %>%
  nrow()
invalid_final_weight_count <- crosswalk_source_diagnostics %>%
  dplyr::filter(!final_weight_sum_ok) %>%
  nrow()
invalid_positive_final_weight_count <- crosswalk_source_diagnostics %>%
  dplyr::filter(positive_targets > 0, !final_weight_sum_ok) %>%
  nrow()
undefined_selected_weight_count <- crosswalk_source_diagnostics %>%
  dplyr::filter(raw_weight_undefined) %>%
  nrow()

checks <- dplyr::bind_rows(
  validation_check(
    "adh_one_row_per_czone",
    adh_duplicate_czone_count == 0,
    fatal = TRUE,
    n_failed = adh_duplicate_czone_count,
    details = "Requires one ADH baseline/exposure row per commuting zone."
  ),
```

```
    validation_check(
      "cz_lookup_unique_1990_counties",
      cz_lookup_duplicate_count == 0 && cz_lookup_multi_czone_count == 0,
      fatal = TRUE,
      n_failed = cz_lookup_duplicate_count + cz_lookup_multi_czone_count,
      details = "Requires each 1990 county FIPS to appear once and map to one CZ."
    ),
    validation_check(
      "aa_all_county_fips_valid",
      invalid_fips_count == 0,
      fatal = TRUE,
      n_failed = invalid_fips_count,
      details = "Requires all county FIPS to be five numeric digits before filtering."
    ),
    validation_check(
      "aa_vote_identity_error_within_tolerance",
      is.finite(max_vote_identity_error) && max_vote_identity_error <= config$max_abs_vote_identity_error,
      fatal = TRUE,
      n_failed = sum(!county_pres_validation$twoparty_matches, na.rm = TRUE),
      details = paste0("Max absolute vote identity error: ", signif(max_vote_identity_error, 6))
    ),
    validation_check(
      "crosswalk_positive_final_weight_sums_equal_one",
      invalid_positive_final_weight_count == 0,
      fatal = TRUE,
      n_failed = invalid_positive_final_weight_count,
      details = paste0("Tolerance: ", config$weight_sum_tolerance)
    ),
    reported_check(
      "crosswalk_all_source_weight_sums_equal_one_or_reported",
      n_reported = invalid_final_weight_count,
      details = paste0(
        "Global non-ADH or unsupported source-county rows are reported rather than treated as validation failures. ",
        "Estimation is gated by positive final weights, retained vote share, and panel balance. Tolerance: ",
        config$weight_sum_tolerance
      )
    ),
    validation_check(
      "crosswalk_undefined_selected_weights_handled",
      config$crosswalk_missing_weight_policy != "fail" || undefined_selected_weight_count == 0,
      fatal = TRUE,
      n_failed = undefined_selected_weight_count,
      details = paste0("Policy: ", config$crosswalk_missing_weight_policy)
    ),
    validation_check(
      "aa_no_duplicate_county_years_after_exact_distinct",
      duplicate_count == 0 || !isTRUE(config$require_no_duplicate_county_years),
      fatal = isTRUE(config$require_no_duplicate_county_years),
      n_failed = duplicate_count,
      details = paste0("Exact duplicate rows removed: ", exact_duplicate_rows)
    ),
    validation_check(
      "aa_two_party_votes_match_party_votes",
      twoparty_mismatch_count == 0,
      fatal = TRUE,
      n_failed = twoparty_mismatch_count,
      details = "Requires republican + democratic votes to equal two-party votes."
    ),
    validation_check(
      "aa_votes_nonnegative",
      nonnegative_failure_count == 0,
      fatal = TRUE,
      n_failed = nonnegative_failure_count,
      details = "Requires nonnegative Republican, Democratic, two-party, and total votes."
    ),
    validation_check(
      "bridged_vote_retention_above_threshold",
      is.finite(min_main_retained) && min_main_retained >= config$min_retained_vote_share,
      fatal = TRUE,
      n_failed = sum(bridge_diagnostics$retained_vote_share < config$min_retained_vote_share, na.rm = TRUE),
      details = paste0("Minimum retained share in main years: ", signif(min_main_retained, 6))
    ),
    validation_check(
      "main_panel_balanced",
      main_missing_n == 0 || (
        !isTRUE(config$require_balanced_panel) &&
          is.finite(main_missing_share) &&
          main_missing_share <= config$max_missing_cz_year_share
      ),
      fatal = isTRUE(config$require_balanced_panel) || config$max_missing_cz_year_share <= 0,
      n_failed = main_missing_n,
      details = paste0(
        "Observed complete rows: ", nrow(analysis_panel), "; expected rows: ", main_expected_n,
        "; missing share: ", signif(main_missing_share, 6)
      )
    )
  )
  readr::write_csv(checks, config$validation_checks_csv)

  sources <- list(
```

```
    adh_stack = list(path = adh_stack_path, md5 = file_checksum(adh_stack_path)),
    adh_long = list(path = adh_long_path, md5 = file_checksum(adh_long_path)),
    cz_lookup = list(path = cz_path, md5 = file_checksum(cz_path)),
    ftz_zip = list(path = ftz_zip, md5 = file_checksum(ftz_zip)),
    aa_presidential = list(path = aa_path, md5 = file_checksum(aa_path))
  )
  manifest <- write_pipeline_manifest(
    config = config,
    checks = checks,
    stage = "build_analysis_data",
    sources = sources,
    extra = list(
      main_expected_rows = main_expected_n,
      main_observed_rows = nrow(analysis_panel),
      main_missing_rows = main_missing_n,
      main_missing_share = main_missing_share,
      min_main_retained_vote_share = min_main_retained
    )
  )

  failed <- checks %>%
    dplyr::filter(fatal, status == "fail")

  if (nrow(failed) > 0 && isTRUE(stop_on_fatal) && !isTRUE(config$allow_failed_diagnostics)) {
    failed_msg <- paste(
      paste0(
        failed$check,
        " (n_failed = ", failed$n_failed, "): ",
        failed$details
      ),
      collapse = "\n"
    )

    detail_parts <- character(0)
    if ("bridged_vote_retention_above_threshold" %in% failed$check) {
      failing_years <- bridge_diagnostics %>%
        dplyr::filter(year %in% config$event_years, retained_vote_share < config$min_retained_vote_share) %>%
        dplyr::transmute(piece = paste0(year, " retained_vote_share=", signif(retained_vote_share, 6))) %>%
        dplyr::pull(piece)
      if (length(failing_years) > 0) {
        detail_parts <- c(detail_parts, paste0("Failing retention years: ", paste(failing_years, collapse = "; ")))
      }
    }
    if ("main_panel_balanced" %in% failed$check) {
      missing_main <- missing_cz_years %>%
        dplyr::filter(sample == "main_1972_start") %>%
        dplyr::mutate(piece = paste0("CZ ", czone, " in ", year)) %>%
        dplyr::pull(piece)
      if (length(missing_main) > 0) {
        shown <- head(missing_main, 30)
        suffix <- if (length(missing_main) > length(shown)) {
          paste0("; ... +", length(missing_main) - length(shown), " more")
        } else {
          ""
        }
        detail_parts <- c(detail_parts, paste0("Missing main-panel CZ-years: ", paste(shown, collapse = "; "), suffix))
      }
    }
    if ("crosswalk_positive_final_weight_sums_equal_one" %in% failed$check) {
      bad_crosswalks <- crosswalk_source_diagnostics %>%
        dplyr::filter(positive_targets > 0, !final_weight_sum_ok) %>%
        dplyr::mutate(piece = paste0(
          source_decade, " county ", county_fips_source,
          " final_sum=", signif(crosswalk_weight_sum, 6)
        )) %>%
        dplyr::pull(piece)
      if (length(bad_crosswalks) > 0) {
        shown <- head(bad_crosswalks, 30)
        suffix <- if (length(bad_crosswalks) > length(shown)) {
          paste0("; ... +", length(bad_crosswalks) - length(shown), " more")
        } else {
          ""
        }
        detail_parts <- c(detail_parts, paste0("Crosswalk source counties with invalid positive final sums: ", paste(shown, collapse = ";
        ↪  "), suffix))
      }
    }

    detail_msg <- if (length(detail_parts) > 0) {
      paste0("\n\nAdditional failing rows:\n", paste(detail_parts, collapse = "\n"))
    } else {
      ""
    }

    stop(
      "Fatal validation checks failed before estimation:\n",
      failed_msg,
      detail_msg,
      "\nSee ", config$pipeline_manifest_json,
      call. = FALSE
```

```
    )
  }

  invisible(list(
    config = config,
    checks = checks,
    manifest = manifest,
    analysis_panel = analysis_panel,
    analysis_panel_full = analysis_panel_full,
    diagnostic_panel = diagnostic_panel,
    diagnostic_panel_full = diagnostic_panel_full,
    crosswalks = crosswalks
  ))
}
```

4 03_estimate_event_study.R

```
# extension-adh2013/src/03_estimate_event_study.R

add_event_study_terms <- function(data, controls, config) {
  years <- sort(unique(data$year))
  years_est <- setdiff(years, config$reference_year)
  event_vars <- paste0("es_", years_est)
  for (i in seq_along(years_est)) {
    data[[event_vars[[i]]]] <- data[[config$exposure_var]] * as.integer(data$year == years_est[[i]])
  }
  ctrl_build <- make_interacted_control_vars(data, controls, years, config$reference_year)
  list(data = ctrl_build$data, event_vars = event_vars, control_vars = ctrl_build$vars)
}

model_control_names <- function(controls, standardize = FALSE) {
  if (isTRUE(standardize)) standardized_control_name(controls) else controls
}

interacted_control_variance_diagnostics <- function(prepared_data, control_vars, sample_name, spec_name) {
  if (length(control_vars) == 0) return(tibble::tibble())
  purrr::map_dfr(control_vars, function(v) {
    vals <- prepared_data[[v]]
    tibble::tibble(
      sample = sample_name,
      spec = spec_name,
      interacted_control = v,
      n_nonmissing = sum(!is.na(vals)),
      variance = stats::var(vals, na.rm = TRUE),
      min = suppressWarnings(min(vals, na.rm = TRUE)),
      max = suppressWarnings(max(vals, na.rm = TRUE)),
      share_zero = mean(vals == 0, na.rm = TRUE)
    )
  })
}

vcov_rank_summary <- function(vcov_mat, sample_name, spec_name, se_type) {
  if (is.null(vcov_mat) || any(!is.finite(vcov_mat))) {
    return(tibble::tibble(
      sample = sample_name, spec = spec_name, se_type = se_type,
      vcov_dim = NA_integer_, vcov_rank = NA_integer_, vcov_rank_deficiency = NA_integer_,
      min_diag = NA_real_, max_diag = NA_real_, median_diag = NA_real_
    ))
  }
  q <- qr(vcov_mat, tol = 1e-10)
  d <- diag(vcov_mat)
  tibble::tibble(
    sample = sample_name, spec = spec_name, se_type = se_type,
    vcov_dim = nrow(vcov_mat), vcov_rank = q$rank,
    vcov_rank_deficiency = nrow(vcov_mat) - q$rank,
    min_diag = suppressWarnings(min(d, na.rm = TRUE)),
    max_diag = suppressWarnings(max(d, na.rm = TRUE)),
    median_diag = suppressWarnings(stats::median(d, na.rm = TRUE))
  )
}

residualized_design_diagnostics <- function(data, rhs_terms, config, sample_name, spec_name) {
  x <- as.matrix(data[, rhs_terms, drop = FALSE])
  fe <- stats::model.matrix(~ factor(czone) + factor(year) - 1, data = data)
  w <- data[[config$weight_var]]
  w <- ifelse(is.finite(w) & w > 0, sqrt(w), 1)
  xw <- x * w
  few <- fe * w
  fit <- stats::lm.fit(few, xw)
  resid_x <- fit$residuals
  qr_obj <- qr(resid_x, tol = 1e-7)
  singular_values <- tryCatch(svd(resid_x, nu = 0, nv = 0)$d, error = function(e) numeric(0))
  positive_sv <- singular_values[singular_values > 1e-10]
  condition_number <- if (length(positive_sv) == 0) NA_real_ else max(positive_sv) / min(positive_sv)
  tibble::tibble(
    sample = sample_name,
    spec = spec_name,
    n_obs = nrow(data),
    n_cz = dplyr::n_distinct(data$czone),
    n_years = dplyr::n_distinct(data$year),
    n_rhs_terms = length(rhs_terms),
    residualized_rank = qr_obj$rank,
    residualized_rank_deficiency = length(rhs_terms) - qr_obj$rank,
    condition_number = condition_number,
    min_singular_value = if (length(singular_values) == 0) NA_real_ else min(singular_values),
    max_singular_value = if (length(singular_values) == 0) NA_real_ else max(singular_values)
  )
}

vcov_eigen_summary <- function(vcov_mat) {
  if (is.null(vcov_mat) || any(!is.finite(vcov_mat))) {
    return(list(
      eigenvalues = NA_real_, min_eigenvalue = NA_real_,
      max_eigenvalue = NA_real_, repair_required = TRUE,
```

```
      n_negative_eigenvalues = NA_integer_,
      eigenvalue_ratio_abs_min_to_max = NA_real_,
      nonfinite = TRUE
    ))
  }
  ev <- eigen((vcov_mat + t(vcov_mat)) / 2, symmetric = TRUE, only.values = TRUE)$values
  min_ev <- min(ev)
  max_ev <- max(ev)
  list(
    eigenvalues = ev,
    min_eigenvalue = min_ev,
    max_eigenvalue = max_ev,
    repair_required = min_ev < -1e-8,
    n_negative_eigenvalues = sum(ev < 0),
    eigenvalue_ratio_abs_min_to_max = if (is.finite(max_ev) && max_ev > 0) abs(min_ev) / max_ev else NA_real_,
    nonfinite = FALSE
  )
}

extract_event_coefficients <- function(model, vcov_mat, spec_name, sample_name, se_type,
                                       repair_required, vcov_repaired) {
  ct <- fixest::coeftable(model, vcov = vcov_mat)
  ct_df <- data.frame(term = rownames(ct), ct, row.names = NULL, check.names = FALSE)
  names(ct_df)[names(ct_df) == "Estimate"] <- "estimate"
  names(ct_df)[names(ct_df) == "Std. Error"] <- "std.error"
  if ("Pr(>|t|)" %in% names(ct_df)) names(ct_df)[names(ct_df) == "Pr(>|t|)"] <- "p.value"
  if ("Pr(>|z|)" %in% names(ct_df)) names(ct_df)[names(ct_df) == "Pr(>|z|)"] <- "p.value"
  if ("t value" %in% names(ct_df)) names(ct_df)[names(ct_df) == "t value"] <- "statistic"
  if ("z value" %in% names(ct_df)) names(ct_df)[names(ct_df) == "z value"] <- "statistic"
  if (!("p.value" %in% names(ct_df))) ct_df$p.value <- NA_real_
  if (!("statistic" %in% names(ct_df))) ct_df$statistic <- NA_real_

  tibble::as_tibble(ct_df) %>%
    dplyr::filter(grepl("^es_", term)) %>%
    dplyr::mutate(
      sample = sample_name,
      spec = spec_name,
      se_type = se_type,
      year = as.integer(sub("^es_", "", term)),
      conf.low = estimate - stats::qnorm(0.975) * std.error,
      conf.high = estimate + stats::qnorm(0.975) * std.error,
      vcov_repair_required = repair_required,
      vcov_repaired = vcov_repaired
    ) %>%
    dplyr::select(
      sample, spec, se_type, term, year, estimate, std.error, conf.low, conf.high,
      statistic, p.value, vcov_repair_required, vcov_repaired
    ) %>%
    dplyr::arrange(sample, spec, se_type, year)
}

vcov_specifications <- function(config) {
  conley_specs <- purrr::map(config$conley_cutoffs_km, function(cutoff) {
    list(
      se_type = paste0("conley_", cutoff, "km"),
      compute = function(model, repair) {
        fixest::vcov_conley(
          model,
          lat = "lat",
          lon = "lon",
          cutoff = cutoff,
          distance = config$conley_distance,
          vcov_fix = repair
        )
      }
    )
  })
  c(
    list(
      list(
        se_type = "heteroskedastic",
        compute = function(model, repair) fixest::vcov_hetero(model, vcov_fix = repair)
      ),
      list(
        se_type = "cluster_cz",
        compute = function(model, repair) {
          fixest::vcov_cluster(model, cluster = stats::as.formula("~czone"), vcov_fix = repair)
        }
      ),
      list(
        se_type = "cluster_state",
        compute = function(model, repair) {
          fixest::vcov_cluster(model, cluster = stats::as.formula("~statefip"), vcov_fix = repair)
        }
      ),
      list(
        se_type = "cluster_cz_year",
        compute = function(model, repair) {
          fixest::vcov_cluster(model, cluster = stats::as.formula("~czone + year"), vcov_fix = repair)
        }
```

```
      ),
      list(
        se_type = paste0("newey_west_lag", config$nw_lag),
        compute = function(model, repair) {
          fixest::vcov_NW(
            model, unit = "czone", time = "election_index",
            lag = config$nw_lag, vcov_fix = repair
          )
        }
      ),
      list(
        se_type = paste0("driscoll_kraay_lag", config$dk_lag),
        compute = function(model, repair) {
          fixest::vcov_DK(model, time = "election_index", lag = config$dk_lag, vcov_fix = repair)
        }
      )
    ),
    conley_specs
  )
}

compute_vcov_outputs <- function(model, spec_name, sample_name, config) {
  specs <- vcov_specifications(config)
  coefficients <- list()
  diagnostics <- list()
  eigenvalues <- list()
  vcov_mats <- list()

  for (s in specs) {
    se_type <- s$se_type
    raw <- tryCatch(s$compute(model, FALSE), error = function(e) e)
    if (inherits(raw, "error")) {
      diagnostics[[se_type]] <- tibble::tibble(
        sample = sample_name, spec = spec_name, se_type = se_type,
        status = "failed", error_message = conditionMessage(raw),
        min_eigenvalue = NA_real_, max_eigenvalue = NA_real_,
        n_negative_eigenvalues = NA_integer_,
        eigenvalue_ratio_abs_min_to_max = NA_real_,
        repair_required = TRUE, vcov_repaired = FALSE
      )
      next
    }

    eig <- vcov_eigen_summary(raw)
    eigenvalues[[se_type]] <- tibble::tibble(
      sample = sample_name,
      spec = spec_name,
      se_type = se_type,
      eigen_index = seq_along(eig$eigenvalues),
      eigenvalue = eig$eigenvalues
    )

    use_vcov <- raw
    repaired <- FALSE
    status <- "ok"
    err <- NA_character_

    if (isTRUE(eig$repair_required)) {
      status <- "repair_required"
      err <- "Unrepaired VCOV is not positive semidefinite."
      if (isTRUE(config$allow_vcov_repair)) {
        repaired_try <- tryCatch(s$compute(model, TRUE), error = function(e) e)
        if (inherits(repaired_try, "error")) {
          status <- "failed"
          err <- conditionMessage(repaired_try)
        } else {
          use_vcov <- repaired_try
          repaired <- TRUE
          status <- "repaired"
        }
      }
    }

    diagnostics[[se_type]] <- tibble::tibble(
      sample = sample_name, spec = spec_name, se_type = se_type,
      status = status, error_message = err,
      min_eigenvalue = eig$min_eigenvalue, max_eigenvalue = eig$max_eigenvalue,
      n_negative_eigenvalues = eig$n_negative_eigenvalues,
      eigenvalue_ratio_abs_min_to_max = eig$eigenvalue_ratio_abs_min_to_max,
      repair_required = eig$repair_required, vcov_repaired = repaired
    )

    # Write coefficient rows for every successfully computed VCOV, including
    # unrepaired VCOVs flagged as repair_required. This keeps the all-spec CSV
    # complete for diagnostics (especially all Conley cutoffs/specifications),
    # while the vcov_repair_required/vcov_repaired flags make clear which rows
    # should not be treated as publication-ready inference.
    if (status %in% c("ok", "repaired", "repair_required")) {
      coef_tbl <- tryCatch(
        extract_event_coefficients(
```

```
          model = model, vcov_mat = use_vcov, spec_name = spec_name,
          sample_name = sample_name, se_type = se_type,
          repair_required = eig$repair_required, vcov_repaired = repaired
        ),
        error = function(e) e
      )
      if (inherits(coef_tbl, "error")) {
        diagnostics[[se_type]]$status <- "coeftable_failed"
        diagnostics[[se_type]]$error_message <- conditionMessage(coef_tbl)
      } else {
        coefficients[[se_type]] <- coef_tbl
        vcov_mats[[se_type]] <- use_vcov
      }
    }
  }

  list(
    coefficients = dplyr::bind_rows(coefficients),
    diagnostics = dplyr::bind_rows(diagnostics),
    eigenvalues = dplyr::bind_rows(eigenvalues),
    vcov = vcov_mats
  )
}

controls_manifest_for_spec <- function(sample_name, spec_name, controls, model_controls, standardize_controls, config, model) {
  dropped <- model$collin.var %||% character(0)
  tibble::tibble(control = config$baseline_controls) %>%
    dplyr::mutate(
      sample = sample_name,
      spec = spec_name,
      requested = control %in% controls,
      standardize_controls = isTRUE(standardize_controls),
      model_control = dplyr::if_else(requested, model_control_names(control, standardize_controls), NA_character_),
      status = dplyr::case_when(
        !requested ~ "omitted_from_spec",
        vapply(model_control, function(ctrl) !is.na(ctrl) && any(startsWith(dropped, paste0("ctrl_", ctrl, "_"))), logical(1)) ~
          "dropped_by_fixest",
        TRUE ~ "included"
      )
    ) %>%
    dplyr::select(sample, spec, control, requested, standardize_controls, model_control, status)
}

estimate_one_spec <- function(data, sample_name, spec_name, controls, controls_label, config, standardize_controls = FALSE) {
  model_controls <- model_control_names(controls, standardize_controls)
  missing_model_controls <- setdiff(model_controls, names(data))
  if (length(missing_model_controls) > 0) {
    stop("Missing model control variables: ", paste(missing_model_controls, collapse = ", "))
  }
  prepared <- add_event_study_terms(data, model_controls, config)
  rhs_terms <- c(prepared$event_vars, prepared$control_vars)
  formula_chr <- paste0(config$outcome_var, " ~ ", paste(rhs_terms, collapse = " + "), " | czone + year")
  design_diag <- residualized_design_diagnostics(prepared$data, rhs_terms, config, sample_name, spec_name)

  tryCatch({
    message("Estimating ", sample_name, " / ", spec_name)
    model <- fixest::feols(
      fml = stats::as.formula(formula_chr),
      data = prepared$data,
      weights = stats::as.formula(paste0("~", config$weight_var)),
      panel.id = stats::as.formula("~czone + election_index"),
      warn = TRUE,
      notes = FALSE
    )
    vcovs <- compute_vcov_outputs(model, spec_name, sample_name, config)
    main_vcov_diag <- vcovs$diagnostics %>% dplyr::filter(se_type == config$main_se_type)
    main_vcov_failed <- nrow(main_vcov_diag) == 0 ||
      main_vcov_diag$status %in% c("failed", "repair_required") ||
      (isTRUE(config$require_no_vcov_repair_for_main) && isTRUE(main_vcov_diag$repair_required[[1]]))
    status <- if (main_vcov_failed) "failed" else "ok"
    error_message <- if (main_vcov_failed) {
      paste0("Main SE ", config$main_se_type, " failed or required VCOV repair.")
    } else {
      NA_character_
    }

    model_stats <- list(
      "Observations" = format(nrow(prepared$data), big.mark = ","),
      "Commuting zones" = format(dplyr::n_distinct(prepared$data$czone), big.mark = ","),
      "Election years" = paste0(min(prepared$data$year), "--", max(prepared$data$year)),
      "Reference election year" = as.character(config$reference_year),
      "CZ fixed effects" = "Yes",
      "Election-year fixed effects" = "Yes",
      "Controls" = controls_label,
      "Controls standardized" = if (isTRUE(standardize_controls)) "Yes" else "No",
      "Sample" = sample_name,
      "ADH weights" = "timepwt48",
      "Main SE type" = config$main_se_type
    )

```

```
    list(
      sample = sample_name, spec = spec_name, status = status, error_message = error_message,
      formula_chr = formula_chr, model = model, vcov = vcovs$vcov,
      coefficients = vcovs$coefficients, vcov_diagnostics = vcovs$diagnostics,
      vcov_eigenvalues = vcovs$eigenvalues, model_diagnostics = design_diag,
      interacted_control_variance = interacted_control_variance_diagnostics(prepared$data, prepared$control_vars, sample_name, spec_name),
      controls_manifest = controls_manifest_for_spec(sample_name, spec_name, controls, model_controls, standardize_controls, config,
      ↪  model),
      model_stats = model_stats, n_obs = nrow(prepared$data),
      n_cz = dplyr::n_distinct(prepared$data$czone), min_year = min(prepared$data$year),
      max_year = max(prepared$data$year)
    )
  }, error = function(e) {
    list(
      sample = sample_name, spec = spec_name, status = "failed",
      error_message = conditionMessage(e), formula_chr = formula_chr,
      model = NULL, vcov = list(), coefficients = tibble::tibble(),
      vcov_diagnostics = tibble::tibble(), vcov_eigenvalues = tibble::tibble(),
      model_diagnostics = design_diag, interacted_control_variance = tibble::tibble(), controls_manifest = tibble::tibble(),
      model_stats = list(
        "Observations" = format(nrow(prepared$data), big.mark = ","),
        "Commuting zones" = format(dplyr::n_distinct(prepared$data$czone), big.mark = ","),
        "Election years" = paste0(min(prepared$data$year), "--", max(prepared$data$year)),
        "Reference election year" = as.character(config$reference_year),
        "CZ fixed effects" = "Yes",
        "Election-year fixed effects" = "Yes",
        "Controls" = controls_label,
        "Controls standardized" = if (isTRUE(standardize_controls)) "Yes" else "No",
        "Sample" = sample_name,
        "ADH weights" = "timepwt48",
        "Main SE type" = "Failed"
      ),
      n_obs = nrow(prepared$data), n_cz = dplyr::n_distinct(prepared$data$czone),
      min_year = min(prepared$data$year), max_year = max(prepared$data$year)
    )
  })
}

run_pretrend_tests <- function(results, config) {
  purrr::map_dfr(results, function(res) {
    if (is.null(res$model) || res$sample != "main_1972_start") return(tibble::tibble())
    main_vcov <- res$vcov[[config$main_se_type]]
    if (is.null(main_vcov)) return(tibble::tibble())
    pre_years <- sort(unique(res$coefficients$year[res$coefficients$year < config$reference_year]))
    if (length(pre_years) == 0) return(tibble::tibble())
    keep_pattern <- paste0("^es_(", paste(pre_years, collapse = "|"), ")$")
    wt <- tryCatch(
      fixest::wald(res$model, keep = keep_pattern, vcov = main_vcov, print = FALSE),
      error = function(e) e
    )
    if (inherits(wt, "error")) {
      tibble::tibble(
        sample = res$sample, spec = res$spec, se_type = config$main_se_type,
        pre_years = paste(pre_years, collapse = ","),
        statistic = NA_real_, p.value = NA_real_, df1 = NA_real_, df2 = NA_real_,
        status = "failed", error_message = conditionMessage(wt)
      )
    } else {
      tibble::tibble(
        sample = res$sample, spec = res$spec, se_type = config$main_se_type,
        pre_years = paste(pre_years, collapse = ","),
        statistic = wt$stat, p.value = wt$p, df1 = wt$df1, df2 = wt$df2,
        status = "ok", error_message = NA_character_
      )
    }
  })
}

estimate_event_study <- function(config = CONFIG, stop_on_fatal = TRUE) {
  load_required_packages(config)
  config <- finalize_config(config)

  existing_checks <- if (file.exists(config$validation_checks_csv)) {
    readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
  } else {
    tibble::tibble()
  }
  if (nrow(existing_checks) > 0 && has_fatal_failures(existing_checks) && isTRUE(stop_on_fatal)) {
    stop("Fatal build-stage validation checks are present. Rebuild with a nonfatal policy before estimating.", call. = FALSE)
  }

  main_panel <- readRDS(config$analysis_panel_rds)
  diagnostic_panel <- readRDS(config$diagnostic_panel_rds)

  spec_definitions <- list(
    minimal_full_panel = list(
      controls = character(0),
      standardize_controls = FALSE,
      label = "None"
    ),
```

```
  interacted_core_controls_diagnostic = list(
    controls = config$core_interacted_controls,
    standardize_controls = isTRUE(config$standardize_interacted_controls),
    label = "Three 1990 ADH controls interacted with election-year indicators"
  ),
  interacted_controls_full_panel = list(
    controls = config$interacted_controls,
    standardize_controls = isTRUE(config$standardize_interacted_controls),
    label = "All configured 1990 ADH baseline controls interacted with election-year indicators"
  ),
  interacted_controls_full_panel_raw_controls_diagnostic = list(
    controls = config$interacted_controls,
    standardize_controls = FALSE,
    label = "All configured 1990 ADH baseline controls interacted with election-year indicators (raw-control diagnostic)"
  )
)

samples <- list(
  main_1972_start = main_panel,
  diagnostic_1952_start = diagnostic_panel
)

all_results <- list()
for (sample_name in names(samples)) {
  for (spec_name in names(spec_definitions)) {
    def <- spec_definitions[[spec_name]]
    key <- paste(sample_name, spec_name, sep = "__")
    all_results[[key]] <- estimate_one_spec(
      data = samples[[sample_name]],
      sample_name = sample_name,
      spec_name = spec_name,
      controls = def$controls,
      controls_label = def$label,
      config = config,
      standardize_controls = isTRUE(def$standardize_controls)
    )
  }
}

status_tbl <- purrr::map_dfr(all_results, ~ tibble::tibble(
  sample = .x$sample, spec = .x$spec, status = .x$status, error_message = .x$error_message,
  n_obs = .x$n_obs, n_cz = .x$n_cz, min_year = .x$min_year, max_year = .x$max_year,
  formula_chr = .x$formula_chr
))
coef_tbl_all <- purrr::map_dfr(all_results, ~ .x$coefficients)
model_diagnostics <- purrr::map_dfr(all_results, ~ {
  dropped <- if (is.null(.x$model)) character(0) else (.x$model$collin.var %||% character(0))
  .x$model_diagnostics %>%
    dplyr::mutate(
      fixest_dropped_variables = paste(dropped, collapse = "; "),
      fixest_dropped_count = length(dropped)
    )
})
controls_manifest <- purrr::map_dfr(all_results, ~ .x$controls_manifest)
interacted_control_variance <- purrr::map_dfr(all_results, ~ .x$interacted_control_variance %||% tibble::tibble())
vcov_diagnostics <- purrr::map_dfr(all_results, ~ .x$vcov_diagnostics)
vcov_eigenvalues <- purrr::map_dfr(all_results, ~ .x$vcov_eigenvalues)
vcov_rank_diagnostics <- purrr::imap_dfr(all_results, function(res, nm) {
  if (is.null(res$vcov) || length(res$vcov) == 0) return(tibble::tibble())
  purrr::imap_dfr(res$vcov, ~ vcov_rank_summary(.x, res$sample, res$spec, .y))
})
se_warning_diagnostics <- coef_tbl_all %>%
  dplyr::group_by(sample, spec, se_type) %>%
  dplyr::summarise(
    n_event_coefficients = dplyr::n(),
    min_std_error = min(std.error, na.rm = TRUE),
    median_std_error = stats::median(std.error, na.rm = TRUE),
    max_std_error = max(std.error, na.rm = TRUE),
    near_zero_se_count = sum(abs(std.error) < 1e-10, na.rm = TRUE),
    nonfinite_se_count = sum(!is.finite(std.error)),
    .groups = "drop"
  ) %>%
  dplyr::left_join(
    vcov_rank_diagnostics %>%
      dplyr::select(sample, spec, se_type, vcov_dim, vcov_rank, vcov_rank_deficiency),
    by = c("sample", "spec", "se_type")
  ) %>%
  dplyr::left_join(
    vcov_diagnostics %>%
      dplyr::select(sample, spec, se_type, vcov_status = status, repair_required, vcov_repaired, min_eigenvalue),
    by = c("sample", "spec", "se_type")
  ) %>%
  dplyr::mutate(
    diagnostic_note = dplyr::case_when(
      grepl("driscoll", se_type) & near_zero_se_count > 0 & dplyr::coalesce(vcov_rank_deficiency, 0L) > 0 ~
        "Suspicious near-zero Driscoll-Kraay SEs and rank-deficient VCOV; do not report without further diagnosis.",
      grepl("driscoll", se_type) & near_zero_se_count > 0 ~
        "Suspicious near-zero Driscoll-Kraay SEs; do not report without further diagnosis.",
      grepl("conley", se_type) & (dplyr::coalesce(repair_required, FALSE) | dplyr::coalesce(vcov_repaired, FALSE)) ~
        "Conley VCOV is non-positive-definite before repair or requires numerical repair; diagnostic only unless separately justified.",
```

```
      dplyr::coalesce(vcov_repaired, FALSE) ~
        "VCOV required numerical repair; diagnostic only unless separately justified.",
      dplyr::coalesce(vcov_rank_deficiency, 0L) > 0 ~
        "VCOV is rank deficient; use as diagnostic only unless justified.",
      grepl("cluster_cz_year", se_type) ~
        "Two-way clustering includes very few year clusters; use as diagnostic only.",
      TRUE ~ NA_character_
    )
  )
pretrend_tests <- run_pretrend_tests(all_results, config)
pretrend_coefficients <- coef_tbl_all %>%
  dplyr::filter(sample == "main_1972_start", se_type == config$main_se_type, year < config$reference_year)

rank_failures <- model_diagnostics %>%
  dplyr::filter(residualized_rank_deficiency > 0)
main_vcov_problem_count <- vcov_diagnostics %>%
  dplyr::filter(
    sample == "main_1972_start",
    spec %in% c("minimal_full_panel", "interacted_controls_full_panel"),
    se_type == config$main_se_type,
    !status %in% c("ok", "repaired")
  ) %>%
  nrow()
main_vcov_repair_count <- vcov_diagnostics %>%
  dplyr::filter(
    sample == "main_1972_start",
    spec %in% c("minimal_full_panel", "interacted_controls_full_panel"),
    se_type == config$main_se_type,
    repair_required %in% TRUE
  ) %>%
  nrow()
conley_repair_count <- vcov_diagnostics %>%
  dplyr::filter(grepl("^conley_", se_type), repair_required %in% TRUE) %>%
  nrow()

build_checks <- if (file.exists(config$validation_checks_csv)) {
  readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
} else {
  tibble::tibble()
}
estimation_checks <- dplyr::bind_rows(
  validation_check(
    "event_study_residualized_design_full_rank",
    nrow(rank_failures) == 0,
    fatal = TRUE,
    n_failed = nrow(rank_failures),
    details = "Requires no residualized RHS rank deficiency in 1952-start or 1972-start specifications."
  ),
  validation_check(
    "event_study_main_vcov_available_without_repair",
    main_vcov_problem_count == 0 &&
      (!isTRUE(config$require_no_vcov_repair_for_main) || main_vcov_repair_count == 0),
    fatal = TRUE,
    n_failed = main_vcov_problem_count + main_vcov_repair_count,
    details = paste0("Main SE type: ", config$main_se_type)
  ),
  validation_check(
    "event_study_conley_repair_diagnostics_recorded",
    TRUE,
    fatal = FALSE,
    n_failed = conley_repair_count,
    details = paste0("Conley VCOVs requiring repair across all samples/specs/cutoffs: ", conley_repair_count)
  )
)
combined_checks <- dplyr::bind_rows(build_checks, estimation_checks)
readr::write_csv(combined_checks, config$validation_checks_csv)

write_model_object(all_results, config$all_specs_rds, config)
readr::write_csv(status_tbl, config$spec_status_csv)
readr::write_csv(coef_tbl_all, config$all_specs_csv)
readr::write_csv(model_diagnostics, config$model_diagnostics_csv)
readr::write_csv(controls_manifest, config$controls_manifest_csv)
readr::write_csv(interacted_control_variance, config$interacted_control_variance_csv)
readr::write_csv(vcov_diagnostics, config$vcov_diagnostics_csv)
readr::write_csv(vcov_eigenvalues, config$vcov_eigenvalues_csv)
readr::write_csv(vcov_rank_diagnostics, config$vcov_rank_csv)
readr::write_csv(se_warning_diagnostics, config$se_warning_diagnostics_csv)
readr::write_csv(summarise_conley_neighbors(main_panel, config$conley_cutoffs_km), config$conley_neighbor_counts_csv)
readr::write_csv(pretrend_tests, config$pretrend_tests_csv)
readr::write_csv(pretrend_coefficients, config$pretrend_coefficients_csv)

main_display_specs <- c("minimal_full_panel", "interacted_core_controls_diagnostic", "interacted_controls_full_panel")
main_coefficients <- coef_tbl_all %>%
  dplyr::filter(
    sample == "main_1972_start",
    se_type == config$main_se_type,
    spec %in% main_display_specs
  ) %>%
  dplyr::arrange(factor(spec, levels = main_display_specs), year)
main_models <- all_results[grepl("^main_1972_start__", names(all_results))]
```

```
  main_out <- list(
    coefficients = main_coefficients,
    main_se_type = config$main_se_type,
    display_specs = main_display_specs,
    models = main_models
  )
  write_model_object(main_out, config$event_study_rds, config)
  readr::write_csv(main_coefficients, config$event_study_csv)

  failed_main_vcov <- vcov_diagnostics %>%
    dplyr::filter(
      sample == "main_1972_start",
      spec %in% c("minimal_full_panel", "interacted_controls_full_panel"),
      se_type == config$main_se_type,
      !status %in% c("ok", "repaired")
    )

  write_pipeline_manifest(
    config = config,
    checks = combined_checks,
    stage = "estimate_event_study",
    sources = list(
      analysis_panel = list(path = config$analysis_panel_rds, md5 = file_checksum(config$analysis_panel_rds)),
      diagnostic_panel = list(path = config$diagnostic_panel_rds, md5 = file_checksum(config$diagnostic_panel_rds))
    ),
    extra = list(
      specs = status_tbl,
      conley_repair_count = conley_repair_count,
      main_vcov_problem_count = main_vcov_problem_count,
      main_vcov_repair_count = main_vcov_repair_count
    )
  )

  if (nrow(failed_main_vcov) > 0 && isTRUE(stop_on_fatal) && !isTRUE(config$allow_failed_diagnostics)) {
    stop(
      "Main event-study VCOV failed or required repair. See ",
      config$vcov_diagnostics_csv,
      call. = FALSE
    )
  }

  if (has_fatal_failures(combined_checks) && isTRUE(stop_on_fatal) && !isTRUE(config$allow_failed_diagnostics)) {
    stop("Fatal event-study validation checks failed. See ", config$validation_checks_csv, call. = FALSE)
  }

  invisible(list(config = config, results = all_results, status = status_tbl, coefficients = coef_tbl_all))
}
```

5 04_export_outputs.R

```r
# extension-adh2013/src/04_export_outputs.R

export_event_study_outputs <- function(config = CONFIG) {
  load_required_packages(config)
  config <- finalize_config(config)

  checks <- if (file.exists(config$validation_checks_csv)) {
    readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
  } else {
    tibble::tibble()
  }
  if (nrow(checks) > 0 && has_fatal_failures(checks) && !isTRUE(config$allow_failed_diagnostics)) {
    stop("Refusing to export figures/tables because fatal validation checks are present.", call. = FALSE)
  }
  has_validation_failures <- nrow(checks) > 0 && has_any_failures(checks)
  has_reported_nonfatal_conditions <- nrow(checks) > 0 && has_reported_conditions(checks)

  res_all <- read_model_object(config$all_specs_rds)
  spec_order <- c("minimal_full_panel", "interacted_core_controls_diagnostic", "interacted_controls_full_panel")
  pretty_spec_names <- c(
    minimal_full_panel = "Minimal",
    interacted_core_controls_diagnostic = "Core controls",
    interacted_controls_full_panel = "Full controls"
  )

  ok_keys <- names(res_all)[vapply(res_all, function(x) {
    x$sample == "main_1972_start" && x$spec %in% spec_order && x$status == "ok"
  }, logical(1))]
  if (length(ok_keys) == 0) stop("No successful main specifications found in ", config$all_specs_rds)

  coef_tbl_all <- purrr::map_dfr(ok_keys, function(k) {
    res_all[[k]]$coefficients %>%
      dplyr::filter(se_type == config$main_se_type) %>%
      dplyr::mutate(spec_label = pretty_spec_names[spec])
  })

  table_csv <- coef_tbl_all %>%
    dplyr::transmute(
      sample, spec, spec_label, se_type, year, term, estimate, std.error,
      conf.low, conf.high, statistic, p.value, vcov_repair_required, vcov_repaired
    ) %>%
    dplyr::arrange(factor(spec, levels = spec_order), year)
  readr::write_csv(table_csv, config$table_csv)
  uses_repaired_main_vcov <- any(table_csv$vcov_repaired %in% TRUE, na.rm = TRUE)

  event_years_display <- sort(unique(c(coef_tbl_all$year, config$reference_year)))
  coef_cell <- function(df, year_value) {
    row <- df %>% dplyr::filter(year == year_value)
    if (year_value == config$reference_year) return("--")
    if (nrow(row) == 0) return("")
    paste0(fmt_num(row$estimate[[1]]), star_code(row$p.value[[1]]), " (", fmt_num(row$std.error[[1]]), ")")
  }

  table_body <- tibble::tibble(`Election year` = as.character(event_years_display))
  for (s in spec_order) {
    df_s <- coef_tbl_all %>% dplyr::filter(spec == s)
    if (nrow(df_s) > 0) {
      table_body[[pretty_spec_names[[s]]]] <- vapply(event_years_display, function(y) {
        coef_cell(df_s, y)
      }, character(1))
    }
  }

  stat_names <- c(
    "Observations", "Commuting zones", "Election years", "Reference election year",
    "CZ fixed effects", "Election-year fixed effects", "Controls", "Sample",
    "ADH weights", "Controls standardized", "Main SE type"
  )
  get_stat_value <- function(ms, nm) {
    if (is.null(ms) || is.null(ms[[nm]]) || length(ms[[nm]]) == 0) return("")
    as.character(ms[[nm]][1])
  }
  stat_rows <- tibble::tibble(`Election year` = stat_names)
  for (s in spec_order) {
    key <- ok_keys[vapply(ok_keys, function(k) res_all[[k]]$spec == s, logical(1))]
    if (length(key) > 0) {
      vals <- vapply(stat_names, function(nm) get_stat_value(res_all[[key[[1]]]]$model_stats, nm), character(1))
      stat_rows[[pretty_spec_names[[s]]]] <- vals
    }
  }

  table_for_tex <- dplyr::bind_rows(table_body, stat_rows)
  fallback_label <- dplyr::case_when(
    config$crosswalk_missing_weight_policy == "fallback_m2" ~ "M2 fallback",
    config$crosswalk_missing_weight_policy == "fallback_m4" ~ "M4 fallback",
    config$crosswalk_missing_weight_policy == "identity_if_same_fips" ~ "same-FIPS identity fallback",
    TRUE ~ config$crosswalk_missing_weight_policy
```

```
)
crosswalk_note <- if (identical(config$crosswalk_missing_weight_policy, "fail")) {
  paste0(toupper(config$crosswalk_weight_slug), " weights with no fallback")
} else {
  paste0(
    toupper(config$crosswalk_weight_slug),
    " weights where valid, with ", fallback_label,
    " for source counties whose selected weights are undefined or invalid"
  )
}

se_label <- dplyr::case_when(
  config$main_se_type == "cluster_cz" ~ "CZ-clustered",
  config$main_se_type == "heteroskedastic" ~ "heteroskedastic",
  config$main_se_type == "cluster_state" ~ "state-clustered",
  config$main_se_type == "newey_west" ~ "Newey-West",
  TRUE ~ config$main_se_type
)

notes_text <- paste0(
  if (uses_repaired_main_vcov || has_validation_failures) {
    "PROVISIONAL: at least one fatal/non-handled validation check failed or a main numerical repair/warning was recorded; treat these as diagnostic output until reviewed. "
  } else {
    ""
  },
  if (has_reported_nonfatal_conditions) {
    "Handled nonfatal crosswalk-support conditions are reported in the diagnostics and do not block the ADH-mainland estimating sample. "
  } else {
    ""
  },
  "Each coefficient is the interaction between ADH's 1990-2007 import-exposure measure and the indicated presidential-election year, ",
  "with ", config$reference_year, " omitted as the reference year. Exposure is measured in ",
  config$exposure_units, ". The dependent variable is the commuting-zone Republican presidential vote margin, ",
  "defined as Republican minus Democratic votes divided by two-party votes. County presidential returns are from Amlani and Algara (2021), ",
  "bridged to 1990 counties with the Ferrara, Testa, and Zhou (2024) ",
  crosswalk_note, ", then aggregated to 1990 commuting zones. ",
  "Standard errors use ", se_label, ". Significance stars: *** p<0.01, ** p<0.05, * p<0.10."
)

latex_tbl <- knitr::kable(
  table_for_tex,
  format = "latex",
  booktabs = TRUE,
  longtable = TRUE,
  escape = TRUE,
  caption = paste0(
    if (uses_repaired_main_vcov || has_validation_failures) "Provisional " else "",
    "event-study estimates: ADH China exposure and Republican presidential vote margin"
  ),
  align = c("l", rep("c", ncol(table_for_tex) - 1))
) %>%
  kableExtra::kable_styling(
    latex_options = c("repeat_header", "hold_position"),
    position = "center",
    font_size = 9
  ) %>%
  kableExtra::footnote(general = notes_text, threeparttable = TRUE, escape = TRUE)

writeLines(as.character(latex_tbl), config$table_tex)
render_table_pdf(config$table_tex, config$table_standalone_tex, config$table_pdf)

plot_tbl <- coef_tbl_all %>%
  dplyr::mutate(
    spec_label = factor(spec_label, levels = c("Minimal", "Core controls", "Full controls")),
    line_group = ifelse(year < config$reference_year, "pre", "post")
  )
ref_rows <- plot_tbl %>%
  dplyr::distinct(sample, spec, se_type, spec_label) %>%
  dplyr::mutate(
    term = paste0("es_", config$reference_year),
    year = config$reference_year,
    estimate = 0,
    std.error = NA_real_,
    conf.low = 0,
    conf.high = 0,
    statistic = NA_real_,
    p.value = NA_real_,
    vcov_repair_required = FALSE,
    vcov_repaired = FALSE,
    line_group = "reference"
  )
plot_with_ref <- dplyr::bind_rows(plot_tbl, ref_rows)
line_tbl <- dplyr::bind_rows(
  plot_with_ref %>%
    dplyr::filter(year <= config$reference_year) %>%
    dplyr::mutate(line_group = "pre"),
  plot_with_ref %>%
```

```
      dplyr::filter(year >= config$reference_year) %>%
      dplyr::mutate(line_group = "post")
  )

  period_bands <- tibble::tibble(
    xmin = c(1990, 2008, 2016, 2020),
    xmax = c(2007, 2012, 2019.8, 2020.8),
    label = c("ADH exposure", "Great Recession / Obama", "Trump era", "COVID era"),
    fill = c("#9ecae1", "#fdd0a2", "#c7e9c0", "#dadaeb")
  )

  event_plot <- ggplot2::ggplot() +
    ggplot2::geom_rect(
      data = period_bands,
      ggplot2::aes(xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill = label),
      alpha = 0.16,
      inherit.aes = FALSE
    ) +
    ggplot2::geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.35) +
    ggplot2::geom_vline(xintercept = config$reference_year, linetype = "dotted", linewidth = 0.35) +
    ggplot2::geom_errorbar(
      data = plot_tbl,
      ggplot2::aes(x = year, ymin = conf.low, ymax = conf.high),
      width = 0.6,
      linewidth = 0.35
    ) +
    ggplot2::geom_line(
      data = line_tbl,
      ggplot2::aes(x = year, y = estimate, group = interaction(spec_label, line_group)),
      linewidth = 0.4
    ) +
    ggplot2::geom_point(
      data = plot_with_ref,
      ggplot2::aes(x = year, y = estimate),
      size = 1.8
    ) +
    ggplot2::facet_wrap(~ spec_label, ncol = 1) +
    ggplot2::scale_fill_manual(values = stats::setNames(period_bands$fill, period_bands$label), name = NULL) +
    ggplot2::scale_x_continuous(
      breaks = seq(min(plot_with_ref$year), max(plot_with_ref$year), by = 8),
      minor_breaks = seq(min(plot_with_ref$year), max(plot_with_ref$year), by = 4)
    ) +
    ggplot2::labs(
      title = "China exposure and Republican presidential vote margin, 1972-2020",
      subtitle = paste0("Event-study coefficients with 95% ", se_label, " CIs; ", config$reference_year, " normalized to zero."),
      x = "Presidential election year",
      y = paste0("Coefficient on ADH China exposure (per ", config$exposure_units, ")")
    ) +
    ggplot2::theme_bw(base_size = 11) +
    ggplot2::theme(
      plot.title = ggplot2::element_text(face = "bold"),
      panel.grid.minor = ggplot2::element_blank(),
      panel.grid.major.x = ggplot2::element_blank(),
      legend.position = "bottom"
    )

  ggplot2::ggsave(filename = config$figure_pdf, plot = event_plot, width = 8.5, height = 7.5, bg = "white")
  ggplot2::ggsave(filename = config$figure_png, plot = event_plot, width = 8.5, height = 7.5, dpi = 320, bg = "white")

  output_inventory <- write_output_manifest(config)

  write_pipeline_manifest(
    config = config,
    checks = checks,
    stage = "export_event_study_outputs",
    sources = list(
      all_specs = model_object_reference(config$all_specs_rds, config)
    ),
    extra = list(
      outputs = list(
        table_csv = list(path = config$table_csv, md5 = file_checksum(config$table_csv)),
        table_tex = list(path = config$table_tex, md5 = file_checksum(config$table_tex)),
        table_pdf = list(path = config$table_pdf, md5 = file_checksum(config$table_pdf)),
        figure_pdf = list(path = config$figure_pdf, md5 = file_checksum(config$figure_pdf)),
        figure_png = list(path = config$figure_png, md5 = file_checksum(config$figure_png)),
        output_manifest_csv = list(path = config$output_manifest_csv, md5 = file_checksum(config$output_manifest_csv)),
        output_manifest_json = list(path = config$output_manifest_json, md5 = file_checksum(config$output_manifest_json))
      ),
      output_manifest_n_files = nrow(output_inventory),
      validation_failures = has_validation_failures,
      reported_nonfatal_conditions = has_reported_nonfatal_conditions
    )
  )

  invisible(list(table_csv = table_csv, table_tex = config$table_tex, figure = event_plot))
}
```

6 05_export_diagnostic_maps.R

```
# extension-adh2013/src/05_export_diagnostic_maps.R

crosswalk_sensitivity_plan <- function() {
  tibble::tribble(
    ~crosswalk_weight, ~crosswalk_missing_weight_policy, ~label, ~render_support_maps,
    "m5_weight", "fallback_m2", "M5 + fallback M2", TRUE,
    "m6_weight", "fallback_m2", "M6 + fallback M2", FALSE,
    "m2_weight", "fallback_m2", "Pure M2", FALSE,
    "m4_weight", "fallback_m2", "M4 + fallback M2", FALSE
  )
}

label_from_crosswalk_slug <- function(slug) {
  dplyr::case_when(
    slug == "m5" ~ "M5 + fallback M2",
    slug == "m6" ~ "M6 + fallback M2",
    slug == "m2" ~ "Pure M2",
    slug == "m4" ~ "M4 + fallback M2",
    TRUE ~ toupper(slug)
  )
}

export_crosswalk_specification_comparison <- function(config = CONFIG) {
  load_required_packages(config)
  config <- finalize_config(config)

  coef_files <- list.files(
    config$intermediate_dir,
    pattern = "^event_study_coefficients_all_specs_aa2021_rep_margin_.*\\.csv$",
    full.names = TRUE
  )
  if (length(coef_files) == 0) {
    warning("No all-spec coefficient CSVs found; skipping crosswalk specification comparison plot.", call. = FALSE)
    return(invisible(NULL))
  }

  crosswalk_levels <- c("M5 + fallback M2", "M6 + fallback M2", "Pure M2", "M4 + fallback M2")
  display_specs <- c("minimal_full_panel", "interacted_core_controls_diagnostic", "interacted_controls_full_panel")

  coef_long <- purrr::map_dfr(coef_files, function(path) {
    slug <- sub("^event_study_coefficients_all_specs_aa2021_rep_margin_", "", basename(path))
    slug <- sub("\\.csv$", "", slug)
    readr::read_csv(path, show_col_types = FALSE) %>%
      dplyr::mutate(
        crosswalk_specification = label_from_crosswalk_slug(slug),
        crosswalk_slug = slug
      )
  }) %>%
    dplyr::filter(
      sample == "main_1972_start",
      se_type == config$main_se_type,
      spec %in% display_specs
    ) %>%
    dplyr::mutate(
      spec_label = dplyr::recode(
        spec,
        minimal_full_panel = "Minimal",
        interacted_core_controls_diagnostic = "Core controls",
        interacted_controls_full_panel = "Full controls"
      ),
      spec_label = factor(spec_label, levels = c("Minimal", "Core controls", "Full controls")),
      crosswalk_specification = factor(crosswalk_specification, levels = crosswalk_levels)
    ) %>%
    dplyr::filter(!is.na(crosswalk_specification))

  if (nrow(coef_long) == 0) {
    warning("Crosswalk comparison inputs exist but no matching main-spec coefficients were found.", call. = FALSE)
    return(invisible(NULL))
  }

  # Draw thicker baseline/early layers first and thinner layers last. When the
  # four crosswalk estimates are nearly identical, this makes the overlap show
  # up as concentric colored strokes rather than one hidden line.
  draw_order_tbl <- tibble::tibble(
    crosswalk_specification = factor(crosswalk_levels, levels = crosswalk_levels),
    draw_order = c(4L, 3L, 1L, 2L),
    line_width = c(0.75, 0.95, 1.45, 1.20),
    point_size = c(1.25, 1.45, 2.05, 1.75),
    alpha_value = c(0.95, 0.90, 0.70, 0.82)
  )

  coef_long <- coef_long %>%
    dplyr::left_join(draw_order_tbl, by = "crosswalk_specification")

  readr::write_csv(coef_long, config$crosswalk_comparison_csv)

  n_specs <- dplyr::n_distinct(coef_long$crosswalk_specification)
```

```
plot_title <- if (n_specs > 1) {
  "Crosswalk-specification sensitivity"
} else {
  paste0("Event-study coefficients: ", as.character(unique(coef_long$crosswalk_specification)))
}
plot_subtitle <- if (n_specs > 1) {
  paste0("Main 1972-2020 event-study coefficients; colored/width-varied lines reveal nearly overlapping estimates. SE type = ",
    ↪  config$main_se_type)
} else {
  paste0("Only one crosswalk specification was found; rerun sensitivity mode to add M6, M2, and M4. SE type = ", config$main_se_type)
}

# Colors are intentionally specified here because the plot is meant for a
# final appendix figure where four nearly identical series must be visually
# distinguishable.
comparison_colors <- c(
  "M5 + fallback M2" = "#1b9e77",
  "M6 + fallback M2" = "#d95f02",
  "Pure M2" = "#7570b3",
  "M4 + fallback M2" = "#e7298a"
)
comparison_linetypes <- c(
  "M5 + fallback M2" = "solid",
  "M6 + fallback M2" = "22",
  "Pure M2" = "longdash",
  "M4 + fallback M2" = "dotdash"
)
comparison_linewidths <- c(
  "M5 + fallback M2" = 0.75,
  "M6 + fallback M2" = 0.95,
  "Pure M2" = 1.45,
  "M4 + fallback M2" = 1.20
)
comparison_alphas <- c(
  "M5 + fallback M2" = 0.95,
  "M6 + fallback M2" = 0.90,
  "Pure M2" = 0.70,
  "M4 + fallback M2" = 0.82
)
comparison_shapes <- c(
  "M5 + fallback M2" = 16,
  "M6 + fallback M2" = 17,
  "Pure M2" = 15,
  "M4 + fallback M2" = 18
)

coef_plot_data <- coef_long %>%
  dplyr::arrange(spec_label, draw_order, year)

comp_plot <- ggplot2::ggplot(
  coef_plot_data,
  ggplot2::aes(
    x = year, y = estimate,
    color = crosswalk_specification,
    linetype = crosswalk_specification,
    linewidth = crosswalk_specification,
    alpha = crosswalk_specification,
    group = crosswalk_specification
  )
) +
  ggplot2::geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.3, color = "grey45") +
  ggplot2::geom_line(lineend = "round") +
  ggplot2::geom_point(ggplot2::aes(shape = crosswalk_specification, size = crosswalk_specification), stroke = 0.2) +
  ggplot2::scale_color_manual(values = comparison_colors, drop = FALSE) +
  ggplot2::scale_linetype_manual(values = comparison_linetypes, drop = FALSE) +
  ggplot2::scale_linewidth_manual(values = comparison_linewidths, drop = FALSE, guide = "none") +
  ggplot2::scale_alpha_manual(values = comparison_alphas, drop = FALSE, guide = "none") +
  ggplot2::scale_shape_manual(values = comparison_shapes, drop = FALSE) +
  ggplot2::scale_size_manual(values = c(
    "M5 + fallback M2" = 1.25,
    "M6 + fallback M2" = 1.45,
    "Pure M2" = 2.05,
    "M4 + fallback M2" = 1.75
  ), drop = FALSE, guide = "none") +
  ggplot2::facet_wrap(~ spec_label, ncol = 1) +
  ggplot2::labs(
    title = plot_title,
    subtitle = plot_subtitle,
    x = "Presidential election year",
    y = paste0("Coefficient on ADH China exposure (per ", config$exposure_units, ")"),
    color = "Crosswalk",
    linetype = "Crosswalk",
    shape = "Crosswalk"
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(legend.position = "bottom", panel.grid.minor = ggplot2::element_blank())

ggplot2::ggsave(config$crosswalk_comparison_pdf, comp_plot, width = 8.5, height = 7.5, bg = "white")
ggplot2::ggsave(config$crosswalk_comparison_png, comp_plot, width = 8.5, height = 7.5, dpi = 320, bg = "white")

```

```r
  # Appendix companion: differences relative to Pure M2. This makes the actual
  # magnitude of crosswalk sensitivity visible when the level plot overlaps.
  pure_m2 <- coef_long %>%
    dplyr::filter(crosswalk_specification == "Pure M2") %>%
    dplyr::select(spec_label, year, pure_m2_estimate = estimate)

  delta_long <- coef_long %>%
    dplyr::left_join(pure_m2, by = c("spec_label", "year")) %>%
    dplyr::mutate(delta_vs_pure_m2 = estimate - pure_m2_estimate) %>%
    dplyr::arrange(spec_label, draw_order, year)

  readr::write_csv(delta_long, config$crosswalk_comparison_delta_csv)

  if (n_specs > 1 && any(is.finite(delta_long$delta_vs_pure_m2))) {
    delta_plot <- ggplot2::ggplot(
      delta_long,
      ggplot2::aes(
        x = year, y = delta_vs_pure_m2,
        color = crosswalk_specification,
        linetype = crosswalk_specification,
        linewidth = crosswalk_specification,
        alpha = crosswalk_specification,
        group = crosswalk_specification
      )
    ) +
      ggplot2::geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.3, color = "grey45") +
      ggplot2::geom_line(lineend = "round") +
      ggplot2::geom_point(ggplot2::aes(shape = crosswalk_specification, size = crosswalk_specification), stroke = 0.2) +
      ggplot2::scale_color_manual(values = comparison_colors, drop = FALSE) +
      ggplot2::scale_linetype_manual(values = comparison_linetypes, drop = FALSE) +
      ggplot2::scale_linewidth_manual(values = comparison_linewidths, drop = FALSE, guide = "none") +
      ggplot2::scale_alpha_manual(values = comparison_alphas, drop = FALSE, guide = "none") +
      ggplot2::scale_shape_manual(values = comparison_shapes, drop = FALSE) +
      ggplot2::scale_size_manual(values = c(
        "M5 + fallback M2" = 1.25,
        "M6 + fallback M2" = 1.45,
        "Pure M2" = 2.05,
        "M4 + fallback M2" = 1.75
      ), drop = FALSE, guide = "none") +
      ggplot2::facet_wrap(~ spec_label, ncol = 1, scales = "free_y") +
      ggplot2::labs(
        title = "Crosswalk-specification sensitivity relative to Pure M2",
        subtitle = paste0("Main 1972-2020 event-study coefficients; SE type = ", config$main_se_type),
        x = "Presidential election year",
        y = paste0("Coefficient difference vs. Pure M2 (per ", config$exposure_units, ")"),
        color = "Crosswalk",
        linetype = "Crosswalk",
        shape = "Crosswalk"
      ) +
      ggplot2::theme_bw(base_size = 11) +
      ggplot2::theme(legend.position = "bottom", panel.grid.minor = ggplot2::element_blank())

    ggplot2::ggsave(config$crosswalk_comparison_delta_pdf, delta_plot, width = 8.5, height = 7.5, bg = "white")
    ggplot2::ggsave(config$crosswalk_comparison_delta_png, delta_plot, width = 8.5, height = 7.5, dpi = 320, bg = "white")
  }

  invisible(comp_plot)
}

simplify_sf_for_plot <- function(sf_obj, tolerance = 0.03) {
  if (is.null(sf_obj)) return(sf_obj)
  out <- tryCatch(
    sf::st_simplify(sf_obj, dTolerance = tolerance, preserveTopology = TRUE),
    error = function(e) sf_obj
  )
  out
}

save_crosswalk_support_plot_pair <- function(plot, simplified_plot, pdf_path, png_path, simplified_pdf_path, simplified_png_path,
 ↪  small_png_path = NULL) {
  ggplot2::ggsave(pdf_path, plot, width = 10, height = 7.5, bg = "white")
  ggplot2::ggsave(png_path, plot, width = 10, height = 7.5, dpi = 320, bg = "white")
  ggplot2::ggsave(simplified_pdf_path, simplified_plot, width = 10, height = 7.5, bg = "white")
  ggplot2::ggsave(simplified_png_path, simplified_plot, width = 10, height = 7.5, dpi = 260, bg = "white")
  if (!is.null(small_png_path)) {
    ggplot2::ggsave(small_png_path, simplified_plot, width = 10, height = 7.5, dpi = 160, bg = "white")
  }
}

make_support_map_plot <- function(map_data, title, subtitle, caption, boundary_color = NA) {
  ggplot2::ggplot(map_data) +
    ggplot2::geom_sf(ggplot2::aes(fill = map_fill), color = boundary_color, linewidth = 0.05) +
    ggplot2::facet_wrap(~ weight_type, ncol = 2) +
    ggplot2::scale_fill_manual(
      values = c(
        "No positive affected vote mass" = "grey90",
        "Receives positive affected vote mass" = "grey25"
      ),
      name = NULL
    ) +
```

```
    ggplot2::labs(title = title, subtitle = subtitle, caption = caption) +
    ggplot2::theme_void(base_size = 10) +
    ggplot2::theme(legend.position = "bottom", plot.title = ggplot2::element_text(face = "bold"))
}

export_crosswalk_support_maps <- function(config = CONFIG) {
  load_required_packages(config)
  config <- finalize_config(config)
  if (!isTRUE(config$export_crosswalk_maps)) return(invisible(NULL))

  if (!file.exists(config$target_weight_support_county_csv) || !file.exists(config$target_weight_support_cz_csv)) {
    warning("Crosswalk support diagnostics are missing; skipping support maps.", call. = FALSE)
    return(invisible(NULL))
  }
  if (!requireNamespace("sf", quietly = TRUE)) {
    warning("Package 'sf' is not installed; skipping support maps.", call. = FALSE)
    return(invisible(NULL))
  }

  county_sf <- safe_read_county1990_shapefile(config)
  if (is.null(county_sf)) return(invisible(NULL))

  county_diag <- readr::read_csv(config$target_weight_support_county_csv, show_col_types = FALSE) %>%
    dplyr::mutate(
      weight_type = factor(
        weight_type,
        levels = c("m2_weight", "m4_weight", "m5_weight", "m6_weight"),
        labels = c("M2 unavailable", "M4 unavailable", "M5 unavailable", "M6 unavailable")
      )
    )
  county_map <- county_sf %>%
    dplyr::left_join(county_diag, by = "county_fips_1990") %>%
    dplyr::filter(!is.na(weight_type)) %>%
    dplyr::mutate(
      receives_unavailable_source = dplyr::coalesce(receives_unavailable_source, FALSE),
      map_fill = dplyr::if_else(receives_unavailable_source, "Receives positive affected vote mass", "No positive affected vote mass")
    )

  county_plot <- make_support_map_plot(
    county_map,
    title = "ADH-mainland 1990 counties receiving affected bridged vote mass",
    subtitle = "Demonstration map: flags counties receiving positive vote mass from a source county where the indicated FTZ weight type
    ↪  has zero/undefined support.",
    caption = "Uses the selected/fallback crosswalk as the vote-mass allocator and the 1990 county shapefile under
    ↪  spatial-data/counties-1990 unless overridden.",
    boundary_color = NA
  )
  county_map_simple <- simplify_sf_for_plot(county_map, tolerance = 0.03)
  county_plot_simple <- make_support_map_plot(
    county_map_simple,
    title = "ADH-mainland 1990 counties receiving affected bridged vote mass",
    subtitle = "Simplified-geometry version for smaller appendix files.",
    caption = "Uses the selected/fallback crosswalk as the vote-mass allocator and simplified 1990 county geometry.",
    boundary_color = NA
  )
  save_crosswalk_support_plot_pair(
    county_plot, county_plot_simple,
    config$crosswalk_support_county_map_pdf, config$crosswalk_support_county_map_png,
    config$crosswalk_support_county_map_simplified_pdf, config$crosswalk_support_county_map_simplified_png,
    config$crosswalk_support_county_map_small_png
  )

  cz_lookup <- readxl::read_xls(file.path(config$replication_dir, "cz-data", "cz-198090.xls")) %>%
    dplyr::transmute(
      county_fips_1990 = stringr::str_pad(as.character(`County FIPS Code`), width = 5, side = "left", pad = "0"),
      czone = as.integer(CZ90)
    )

  cz_diag <- readr::read_csv(config$target_weight_support_cz_csv, show_col_types = FALSE) %>%
    dplyr::mutate(
      weight_type = factor(
        weight_type,
        levels = c("m2_weight", "m4_weight", "m5_weight", "m6_weight"),
        labels = c("M2 unavailable", "M4 unavailable", "M5 unavailable", "M6 unavailable")
      )
    )

  cz_sf <- county_sf %>%
    dplyr::inner_join(cz_lookup, by = "county_fips_1990") %>%
    dplyr::filter(czone %in% unique(cz_diag$czone)) %>%
    dplyr::group_by(czone) %>%
    dplyr::summarise(geometry = sf::st_union(geometry), .groups = "drop")

  cz_map <- cz_sf %>%
    dplyr::left_join(cz_diag, by = "czone") %>%
    dplyr::filter(!is.na(weight_type)) %>%
    dplyr::mutate(
      receives_unavailable_source = dplyr::coalesce(receives_unavailable_source, FALSE),
      map_fill = dplyr::if_else(receives_unavailable_source, "Receives positive affected vote mass", "No positive affected vote mass")
    )
```

```
  cz_plot <- make_support_map_plot(
    cz_map,
    title = "ADH-mainland 1990 CZs receiving affected bridged vote mass",
    subtitle = "Demonstration map: flags CZs receiving positive vote mass from a source county where the indicated FTZ weight type has zero/undefined support.",
    caption = "CZ shapes are dissolved from 1990 counties using cz-198090.xls and limited to the ADH diagnostic CZ universe.",
    boundary_color = "white"
  )
  cz_map_simple <- simplify_sf_for_plot(cz_map, tolerance = 0.05)
  cz_plot_simple <- make_support_map_plot(
    cz_map_simple,
    title = "ADH-mainland 1990 CZs receiving affected bridged vote mass",
    subtitle = "Simplified-geometry version for smaller appendix files.",
    caption = "CZ shapes are dissolved from simplified 1990 counties and limited to the ADH diagnostic CZ universe.",
    boundary_color = "white"
  )
  save_crosswalk_support_plot_pair(
    cz_plot, cz_plot_simple,
    config$crosswalk_support_cz_map_pdf, config$crosswalk_support_cz_map_png,
    config$crosswalk_support_cz_map_simplified_pdf, config$crosswalk_support_cz_map_simplified_png,
    config$crosswalk_support_cz_map_small_png
  )

  invisible(list(county_plot = county_plot, cz_plot = cz_plot))
}

copy_if_exists <- function(from, to_dir) {
  if (is.null(from) || length(from) == 0 || is.na(from) || !file.exists(from)) return(invisible(FALSE))
  dir.create(to_dir, recursive = TRUE, showWarnings = FALSE)
  file.copy(from, file.path(to_dir, basename(from)), overwrite = TRUE)
  invisible(TRUE)
}

archive_crosswalk_run_diagnostics <- function(config, label, status = "completed", error_message = NA_character_) {
  config <- finalize_config(config)
  slug <- config$crosswalk_weight_slug
  out_dir <- file.path(config$diagnostic_dir, "by_crosswalk", slug)
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  files <- unique(stats::na.omit(c(
    config$validation_checks_csv, config$pipeline_manifest_json, config$crosswalk_diagnostics_csv,
    config$crosswalk_zero_support_csv, config$crosswalk_row_diagnostics_csv,
    config$crosswalk_issue_vote_report_csv, config$bridge_diagnostics_csv,
    config$mainland_retention_csv, config$unmatched_source_counties_csv,
    config$county_bridge_classification_csv, config$missing_cz_years_csv,
    config$panel_coverage_csv, config$spec_status_csv, config$model_diagnostics_csv,
    config$vcov_diagnostics_csv, config$se_warning_diagnostics_csv, config$event_study_csv,
    config$all_specs_csv, config$table_csv
  )))
  invisible(lapply(files, copy_if_exists, to_dir = out_dir))

  tibble::tibble(
    crosswalk_slug = slug,
    crosswalk_specification = label,
    crosswalk_weight = config$crosswalk_weight,
    crosswalk_missing_weight_policy = config$crosswalk_missing_weight_policy,
    status = status,
    error_message = error_message,
    diagnostics_dir = portable_path(out_dir, config),
    archived_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")
  ) %>%
    readr::write_csv(file.path(out_dir, "run_status.csv"))

  invisible(out_dir)
}

write_crosswalk_sensitivity_status <- function(status_rows, config) {
  config <- finalize_config(config)
  status_tbl <- dplyr::bind_rows(status_rows)
  path <- file.path(config$diagnostic_dir, "crosswalk_sensitivity_run_status.csv")
  readr::write_csv(status_tbl, path)
  invisible(path)
}

export_diagnostic_maps_and_comparisons <- function(config = CONFIG) {
  config <- finalize_config(config)
  export_crosswalk_specification_comparison(config)
  export_crosswalk_support_maps(config)

  checks <- if (file.exists(config$validation_checks_csv)) {
    readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
  } else {
    tibble::tibble()
  }
  output_inventory <- write_output_manifest(config)

  write_pipeline_manifest(
    config = config,
    checks = checks,
```

```
    stage = "export_diagnostic_maps_and_comparisons",
    sources = list(
      county_support = list(path = config$target_weight_support_county_csv, md5 = file_checksum(config$target_weight_support_county_csv)),
      cz_support = list(path = config$target_weight_support_cz_csv, md5 = file_checksum(config$target_weight_support_cz_csv))
    ),
    extra = list(
      outputs = list(
        crosswalk_comparison_png = list(path = config$crosswalk_comparison_png, md5 = file_checksum(config$crosswalk_comparison_png)),
        crosswalk_comparison_delta_png = list(path = config$crosswalk_comparison_delta_png, md5 =
        ↪  file_checksum(config$crosswalk_comparison_delta_png)),
        crosswalk_comparison_delta_csv = list(path = config$crosswalk_comparison_delta_csv, md5 =
        ↪  file_checksum(config$crosswalk_comparison_delta_csv)),
        crosswalk_support_county_map_png = list(path = config$crosswalk_support_county_map_png, md5 =
        ↪  file_checksum(config$crosswalk_support_county_map_png)),
        crosswalk_support_county_map_simplified_png = list(path = config$crosswalk_support_county_map_simplified_png, md5 =
        ↪  file_checksum(config$crosswalk_support_county_map_simplified_png)),
        crosswalk_support_county_map_small_png = list(path = config$crosswalk_support_county_map_small_png, md5 =
        ↪  file_checksum(config$crosswalk_support_county_map_small_png)),
        crosswalk_support_cz_map_png = list(path = config$crosswalk_support_cz_map_png, md5 =
        ↪  file_checksum(config$crosswalk_support_cz_map_png)),
        crosswalk_support_cz_map_simplified_png = list(path = config$crosswalk_support_cz_map_simplified_png, md5 =
        ↪  file_checksum(config$crosswalk_support_cz_map_simplified_png)),
        crosswalk_support_cz_map_small_png = list(path = config$crosswalk_support_cz_map_small_png, md5 =
        ↪  file_checksum(config$crosswalk_support_cz_map_small_png)),
        output_manifest_csv = list(path = config$output_manifest_csv, md5 = file_checksum(config$output_manifest_csv)),
        output_manifest_json = list(path = config$output_manifest_json, md5 = file_checksum(config$output_manifest_json))
      ),
      output_manifest_n_files = nrow(output_inventory)
    )
  )
  invisible(NULL)
}

run_crosswalk_sensitivity_pipeline <- function(config = CONFIG) {
  base_config <- finalize_config(config)
  plan <- crosswalk_sensitivity_plan()
  ensure_output_dirs(base_config)
  write_dependency_manifest(base_config)

  status_rows <- list()
  for (i in seq_len(nrow(plan))) {
    local_config <- base_config
    local_config$crosswalk_weight <- plan$crosswalk_weight[[i]]
    local_config$crosswalk_missing_weight_policy <- plan$crosswalk_missing_weight_policy[[i]]
    local_config$export_crosswalk_maps <- isTRUE(base_config$export_crosswalk_maps) && isTRUE(plan$render_support_maps[[i]])
    local_config <- finalize_config(local_config)

    label <- plan$label[[i]]
    message("Running crosswalk sensitivity specification: ", label, " (", local_config$crosswalk_weight, ", ",
    ↪  local_config$crosswalk_missing_weight_policy, ")")

    step_status <- "completed"
    step_error <- NA_character_
    started_at <- Sys.time()
    tryCatch({
      build_analysis_data(local_config, stop_on_fatal = TRUE)
      estimate_event_study(local_config, stop_on_fatal = TRUE)
      export_event_study_outputs(local_config)
      if (isTRUE(local_config$export_crosswalk_maps)) export_crosswalk_support_maps(local_config)
    }, error = function(e) {
      step_status <<- "failed"
      step_error <<- conditionMessage(e)
      message("Crosswalk sensitivity specification failed: ", label, ": ", step_error)
    })

    archive_crosswalk_run_diagnostics(local_config, label, status = step_status, error_message = step_error)
    status_rows[[length(status_rows) + 1L]] <- tibble::tibble(
      crosswalk_slug = local_config$crosswalk_weight_slug,
      crosswalk_specification = label,
      crosswalk_weight = local_config$crosswalk_weight,
      crosswalk_missing_weight_policy = local_config$crosswalk_missing_weight_policy,
      status = step_status,
      error_message = step_error,
      started_at = format(started_at, "%Y-%m-%d %H:%M:%S %Z"),
      finished_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")
    )
    write_crosswalk_sensitivity_status(status_rows, base_config)
  }

  # After all slug-specific coefficient files exist, create the appendix-ready comparison figure.
  comparison_config <- finalize_config(base_config)
  export_crosswalk_specification_comparison(comparison_config)

  if (isTRUE(comparison_config$run_sixth_extensions)) {
    # Sixth-section extensions are estimated only once for the preferred M5 + fallback-M2 design.
    run_sixth_extensions(comparison_config)
  }

  checks <- if (file.exists(comparison_config$validation_checks_csv)) {
```

```
    readr::read_csv(comparison_config$validation_checks_csv, show_col_types = FALSE)
  } else {
    tibble::tibble()
  }
  status_tbl <- dplyr::bind_rows(status_rows)
  write_crosswalk_sensitivity_status(status_rows, comparison_config)
  output_inventory <- write_output_manifest(comparison_config)
  write_pipeline_manifest(
    config = comparison_config,
    checks = checks,
    stage = "crosswalk_sensitivity_complete",
    sources = list(
      sensitivity_plan = list(specifications = plan),
      sensitivity_status = list(path = file.path(comparison_config$diagnostic_dir, "crosswalk_sensitivity_run_status.csv"), md5 =
        ↪ file_checksum(file.path(comparison_config$diagnostic_dir, "crosswalk_sensitivity_run_status.csv")))
    ),
    extra = list(
      any_failed_specifications = any(status_tbl$status != "completed"),
      completed_specifications = status_tbl %>% dplyr::filter(status == "completed") %>% dplyr::pull(crosswalk_specification),
      failed_specifications = status_tbl %>% dplyr::filter(status != "completed") %>% dplyr::pull(crosswalk_specification),
      comparison_csv = list(path = comparison_config$crosswalk_comparison_csv, md5 =
        ↪ file_checksum(comparison_config$crosswalk_comparison_csv)),
      comparison_png = list(path = comparison_config$crosswalk_comparison_png, md5 =
        ↪ file_checksum(comparison_config$crosswalk_comparison_png)),
      comparison_delta_csv = list(path = comparison_config$crosswalk_comparison_delta_csv, md5 =
        ↪ file_checksum(comparison_config$crosswalk_comparison_delta_csv)),
      comparison_delta_png = list(path = comparison_config$crosswalk_comparison_delta_png, md5 =
        ↪ file_checksum(comparison_config$crosswalk_comparison_delta_png)),
      output_manifest_csv = list(path = comparison_config$output_manifest_csv, md5 =
        ↪ file_checksum(comparison_config$output_manifest_csv)),
      output_manifest_json = list(path = comparison_config$output_manifest_json, md5 =
        ↪ file_checksum(comparison_config$output_manifest_json)),
      output_manifest_n_files = nrow(output_inventory)
    )
  )
  invisible(status_tbl)
}
```

7 06_sixth_extensions.R

```
# extension-adh2013/src/06_sixth_extensions.R

# Highest-priority Sixth-section extensions. These modules intentionally run once
# for the preferred M5 + fallback-M2 design, even when the crosswalk-sensitivity
# wrapper is active. The mechanism/identification diagnostics are meant to be
# interpreted for the preferred analysis sample rather than repeated for every
# crosswalk robustness specification.

extension_primary_config <- function(config = CONFIG) {
  out <- config
  out$crosswalk_weight <- config$extension_crosswalk_weight %||% "m5_weight"
  out$crosswalk_missing_weight_policy <- config$extension_crosswalk_missing_weight_policy %||% "fallback_m2"
  out <- finalize_config(out)
  current <- finalize_config(config)
  if (!file.exists(out$analysis_panel_rds) && file.exists(current$analysis_panel_rds)) out <- current
  out
}

safe_num <- function(x) suppressWarnings(as.numeric(x))

safe_divide <- function(num, den) {
  num <- safe_num(num); den <- safe_num(den)
  out <- ifelse(is.na(num) | is.na(den) | den <= 0, NA_real_, num / den)
  out[!is.finite(out)] <- NA_real_
  out
}

scalar_or_na <- function(x) {
  if (is.null(x) || length(x) == 0) return(NA_real_)
  as.numeric(x[[1]])
}

weighted_mean_safe <- function(x, w = NULL) {
  x <- safe_num(x)
  if (is.null(w)) w <- rep(1, length(x)) else w <- safe_num(w)
  ok <- !is.na(x) & !is.na(w) & w > 0
  if (!any(ok)) return(NA_real_)
  stats::weighted.mean(x[ok], w[ok], na.rm = TRUE)
}

extension_spec_label <- function(x) {
  dplyr::recode(as.character(x),
    minimal = "Minimal",
    core_controls = "Core controls",
    full_controls = "Full controls",
    .default = as.character(x)
  )
}

extension_outcome_label <- function(x) {
  dplyr::recode(as.character(x),
    rep_margin = "Republican margin",
    rep_2party_share = "Republican two-party share",
    dem_2party_share = "Democratic two-party share",
    two_party_votes_per_1990_pop = "Two-party votes / 1990 population",
    total_votes_per_1990_pop = "Total votes / 1990 population",
    rep_votes_per_1990_pop = "Republican votes / 1990 population",
    dem_votes_per_1990_pop = "Democratic votes / 1990 population",
    rep_margin_swing = "Republican-margin swing",
    rep_2party_share_swing = "Republican-share swing",
    reg_voters_pct = "Registered voters / CVAP",
    voter_turnout_pct = "Ballots cast / CVAP",
    reg_voter_turnout_pct = "Ballots cast / registered voters",
    pres_rep_ratio = "Presidential Republican ratio",
    pres_dem_ratio = "Presidential Democratic ratio",
    partisan_index_dem = "Democratic partisanship index",
    partisan_index_rep = "Republican partisanship index",
    .default = as.character(x)
  )
}

adhm_outcome_label <- function(x) {
  dplyr::recode(as.character(x),
    d2_rwin_2002_2016 = "Republican House win share, 2002-2016",
    d2_cfavg_repcon_2002_2016 = "Conservative Republican CF-score share",
    d2_cfavg_repmod_2002_2016 = "Moderate Republican CF-score share",
    d2_cfavg_demmod_2002_2016 = "Moderate Democratic CF-score share",
    d2_cfavg_demlib_2002_2016 = "Liberal Democratic CF-score share",
    d2_teaparty_2002_2010 = "Tea Party / Freedom Caucus outcome",
    d_shnr_pres_2000_2016 = "Republican presidential share, 2000-2016",
    d_shnr_pres_2000_2008 = "Republican presidential share, 2000-2008",
    .default = as.character(x)
  )
}

plot_pointrange_event <- function(df, title, subtitle, y_lab = "Coefficient on ADH exposure × year") {
  ggplot2::ggplot(df, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 * std.error)) +
```

```
    ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) +
    ggplot2::geom_pointrange(linewidth = 0.25) +
    ggplot2::labs(title = title, subtitle = subtitle, x = NULL, y = y_lab) +
    ggplot2::theme_minimal(base_size = 10)
}

first_existing_col <- function(df, candidates) {
  nm <- names(df)
  hit <- candidates[candidates %in% nm]
  if (length(hit) == 0) NA_character_ else hit[[1]]
}

read_first_rdata_frame <- function(path) {
  e <- new.env(parent = emptyenv())
  loaded <- load(path, envir = e)
  candidates <- loaded[vapply(loaded, function(nm) is.data.frame(e[[nm]]), logical(1))]
  if (length(candidates) == 0) stop("No data.frame object found in ", path)
  e[[candidates[[1]]]]
}

coeftable_to_tibble <- function(model, vcov = NULL, term_prefix = NULL) {
  ct <- tryCatch({
    if (is.null(vcov)) fixest::coeftable(model) else fixest::coeftable(model, vcov = vcov)
  }, error = function(e) NULL)
  if (is.null(ct)) return(tibble::tibble())
  out <- as.data.frame(ct) %>% tibble::rownames_to_column("term") %>% tibble::as_tibble()
  names(out) <- c("term", "estimate", "std.error", "statistic", "p.value")
  if (!is.null(term_prefix)) out <- out %>% dplyr::filter(startsWith(term, term_prefix))
  out
}

fit_weighted_feols <- function(formula, data, weight_var = "adh_weight", cluster_var = "czone") {
  weights_fml <- if (!is.null(weight_var) && weight_var %in% names(data)) stats::as.formula(paste0("~", weight_var)) else NULL
  cluster_fml <- if (!is.null(cluster_var) && cluster_var %in% names(data)) stats::as.formula(paste0("~", cluster_var)) else NULL
  fixest::feols(formula, data = data, weights = weights_fml, cluster = cluster_fml, warn = FALSE, notes = FALSE)
}

write_plot_safely <- function(plot, png_path, pdf_path, width = 10, height = 6) {
  try(ggplot2::ggsave(png_path, plot, width = width, height = height, dpi = 300, bg = "white"), silent = TRUE)
  try(ggplot2::ggsave(pdf_path, plot, width = width, height = height, bg = "white"), silent = TRUE)
  invisible(NULL)
}

load_adh_extension_data <- function(config = CONFIG) {
  adh_stack_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china.dta")
  adh_long_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china_long.dta")
  stack <- haven::read_dta(adh_stack_path)
  long <- haven::read_dta(adh_long_path)
  baseline <- stack %>%
    dplyr::filter(yr == 1990) %>%
    dplyr::transmute(
      czone = as.integer(czone),
      statefip = as.integer(statefip),
      dplyr::across(dplyr::all_of(config$baseline_controls), safe_num)
    ) %>%
    dplyr::distinct()
  exposure <- long %>%
    dplyr::transmute(
      czone = as.integer(czone),
      statefip = as.integer(statefip),
      exposure_1990_2007 = safe_num(d_tradeusch_pw),
      instrument_1990_2007 = safe_num(d_tradeotch_pw_lag),
      adh_weight = safe_num(timepwt48)
    ) %>%
    dplyr::distinct()
  baseline %>%
    dplyr::inner_join(exposure, by = c("czone", "statefip")) %>%
    standardize_baseline_controls(config$baseline_controls) %>%
    purrr::pluck("data")
}

make_single_exposure_event_terms <- function(data, exposure_var, years, ref_year, prefix = "es") {
  years <- sort(unique(years))
  for (yr in setdiff(years, ref_year)) {
    nm <- paste0(prefix, "_", yr)
    data[[nm]] <- data[[exposure_var]] * as.integer(data$year == yr)
  }
  data
}

apply_control_interactions <- function(data, controls, years, ref_year, standardize = TRUE) {
  controls_use <- model_control_names(controls, standardize = standardize)
  controls_use <- controls_use[controls_use %in% names(data)]
  if (length(controls_use) == 0) return(list(data = data, controls = controls_use, terms = character(0)))
  built <- make_interacted_control_vars(data, controls_use, years, ref_year)
  list(data = built$data, controls = controls_use, terms = built$vars)
}

fit_single_outcome_event_study <- function(panel, outcome_var, controls, spec_name, config,
                                           ref_year = config$reference_year,
```

```
                                        standardize_controls = TRUE,
                                        sample_label = "main_1972_start") {
  yrs <- sort(unique(panel$year[!is.na(panel[[outcome_var]])]))
  if (length(yrs) == 0) return(list(status = "empty", coefficients = tibble::tibble()))
  if (!ref_year %in% yrs) ref_year <- min(yrs, na.rm = TRUE)
  dat <- panel %>% dplyr::filter(year %in% yrs, !is.na(.data[[outcome_var]]), !is.na(exposure))
  if (nrow(dat) == 0) return(list(status = "empty", coefficients = tibble::tibble()))
  dat <- make_single_exposure_event_terms(dat, "exposure", yrs, ref_year, prefix = "es")
  event_terms <- paste0("es_", setdiff(yrs, ref_year))
  ctrl_build <- apply_control_interactions(dat, controls, yrs, ref_year, standardize = standardize_controls)
  dat <- ctrl_build$data
  rhs <- c(event_terms, ctrl_build$terms)
  if (length(rhs) == 0) return(list(status = "no_terms", coefficients = tibble::tibble()))
  fml <- stats::as.formula(paste(outcome_var, "~", paste(rhs, collapse = " + "), "| czone + year"))
  tryCatch({
    model <- fixest::feols(fml, data = dat, weights = ~adh_weight, cluster = ~czone, panel.id = ~czone + election_index, warn = FALSE,
     ↪  notes = FALSE)
    coef <- coeftable_to_tibble(model) %>%
      dplyr::filter(term %in% event_terms) %>%
      dplyr::mutate(
        outcome = outcome_var,
        spec = spec_name,
        sample = sample_label,
        se_type = "cluster_cz",
        reference_year = ref_year,
        year = as.integer(stringr::str_extract(term, "[0-9]{4}")),
        n_obs = stats::nobs(model),
        n_cz = dplyr::n_distinct(dat$czone),
        controls = if (length(ctrl_build$controls) == 0) "none" else paste(ctrl_build$controls, collapse = "; ")
      ) %>%
      dplyr::select(outcome, spec, sample, se_type, term, year, reference_year, estimate, std.error, statistic, p.value, n_obs, n_cz,
       ↪  controls)
    list(status = "ok", coefficients = coef)
  }, error = function(e) {
    list(status = paste0("error: ", conditionMessage(e)), coefficients = tibble::tibble())
  })
}

# ------------------------------------------------------------------------------
# 1. Bartik identification diagnostics
# ------------------------------------------------------------------------------

extract_trade_shift_summary <- function(config) {
  trade_path <- file.path(config$replication_dir, "adh2013", "dta", "sic87dd_trade_data.dta")
  if (!file.exists(trade_path)) return(tibble::tibble(status = "missing", path = trade_path))
  tr <- haven::read_dta(trade_path)
  names(tr) <- tolower(names(tr))
  req <- c("year", "importer", "exporter", "imports", "sic87dd")
  if (!all(req %in% names(tr))) {
    return(tibble::tibble(status = "missing_required_columns", columns_found = paste(names(tr), collapse = ";")))
  }
  tr <- tr %>%
    dplyr::mutate(
      year = as.integer(year),
      importer = toupper(trimws(as.character(importer))),
      exporter = toupper(trimws(as.character(exporter))),
      imports = safe_num(imports),
      sic87dd = as.character(sic87dd)
    )
  periods <- tibble::tibble(start_year = c(1991L, 2000L, 1991L), end_year = c(1999L, 2007L, 2007L), period = c("1991-1999", "2000-2007",
   ↪  "1991-2007"))
  purrr::pmap_dfr(periods, function(start_year, end_year, period) {
    base <- tr %>% dplyr::filter(year %in% c(start_year, end_year), exporter %in% c("CHN", "CHINA"), importer %in% c("USA", "OTH",
     ↪  "OTHER", "OTHCAN"))
    if (nrow(base) == 0) return(tibble::tibble(period = period, status = "no_matching_trade_rows"))
    wide <- base %>%
      dplyr::group_by(sic87dd, importer, year) %>%
      dplyr::summarise(imports = sum(imports, na.rm = TRUE), .groups = "drop") %>%
      tidyr::pivot_wider(names_from = year, values_from = imports, names_prefix = "imports_")
    start_col <- paste0("imports_", start_year)
    end_col <- paste0("imports_", end_year)
    if (!all(c(start_col, end_col) %in% names(wide))) return(tibble::tibble(period = period, status = "missing_period_endpoints"))
    wide %>%
      dplyr::mutate(
        start_imports = .data[[start_col]],
        end_imports = .data[[end_col]],
        import_growth = end_imports - start_imports
      ) %>%
      dplyr::group_by(period = period, importer) %>%
      dplyr::slice_max(order_by = abs(import_growth), n = 15, with_ties = FALSE) %>%
      dplyr::ungroup() %>%
      dplyr::mutate(status = "ok") %>%
      dplyr::select(status, period, importer, sic87dd, start_imports, end_imports, import_growth)
  })
}

run_bartik_identification_diagnostics <- function(config = CONFIG) {
  config <- finalize_config(config)
  adh <- load_adh_extension_data(config)
  controls_core <- model_control_names(config$core_interacted_controls, standardize = TRUE)
```

```
controls_full <- model_control_names(config$interacted_controls, standardize = TRUE)
controls_core <- controls_core[controls_core %in% names(adh)]
controls_full <- controls_full[controls_full %in% names(adh)]
fs_specs <- list(minimal = character(0), core_controls = controls_core, full_controls = controls_full)
fs_tbl <- purrr::imap_dfr(fs_specs, function(ctrls, spec) {
  rhs <- c("instrument_1990_2007", ctrls)
  fml <- stats::as.formula(paste("exposure_1990_2007 ~", paste(rhs, collapse = " + ")))
  model <- fit_weighted_feols(fml, adh, weight_var = "adh_weight", cluster_var = "statefip")
  ct <- coeftable_to_tibble(model) %>% dplyr::filter(term == "instrument_1990_2007")
  tibble::tibble(
    spec = spec,
    endogenous_variable = "ADH exposure, 1990-2007",
    instrument = "ADH other-high-income-country import growth instrument",
    estimate = scalar_or_na(ct$estimate),
    std.error = scalar_or_na(ct$std.error),
    statistic = scalar_or_na(ct$statistic),
    p.value = scalar_or_na(ct$p.value),
    first_stage_F_t2 = (scalar_or_na(ct$statistic))^2,
    n_obs = stats::nobs(model),
    r2 = tryCatch(fixest::r2(model, type = "r2"), error = function(e) NA_real_),
    controls = if (length(ctrls) == 0) "none" else paste(ctrls, collapse = "; ")
  )
})
readr::write_csv(fs_tbl, config$bartik_first_stage_csv)
p_fs <- ggplot2::ggplot(adh, ggplot2::aes(x = instrument_1990_2007, y = exposure_1990_2007)) +
  ggplot2::geom_point(ggplot2::aes(size = adh_weight), alpha = 0.35) +
  ggplot2::geom_smooth(method = "lm", se = TRUE, linewidth = 0.5) +
  ggplot2::guides(size = "none") +
  ggplot2::labs(title = "ADH exposure first stage", subtitle = "CZ-level China exposure and ADH other-country instrument", x =
    ↪  "Instrument", y = "Exposure") +
  ggplot2::theme_minimal(base_size = 11)
write_plot_safely(p_fs, config$bartik_first_stage_plot_png, config$bartik_first_stage_plot_pdf, width = 8, height = 5)

balance_vars <- unique(c(config$baseline_controls, controls_full, "statefip"))
balance_vars <- balance_vars[balance_vars %in% names(adh)]
balance_tbl <- purrr::map_dfr(balance_vars, function(v) {
  x <- adh[[v]]
  tibble::tibble(
    variable = v,
    correlation_with_exposure = suppressWarnings(stats::cor(x, adh$exposure_1990_2007, use = "pairwise.complete.obs")),
    correlation_with_instrument = suppressWarnings(stats::cor(x, adh$instrument_1990_2007, use = "pairwise.complete.obs")),
    weighted_mean = weighted_mean_safe(x, adh$adh_weight),
    sd = stats::sd(safe_num(x), na.rm = TRUE)
  )
})
readr::write_csv(balance_tbl, config$bartik_balance_csv)

pretrend_tbl <- tibble::tibble()
if (file.exists(config$diagnostic_panel_rds)) {
  panel <- readRDS(config$diagnostic_panel_rds) %>% dplyr::filter(year < config$reference_year) %>% dplyr::arrange(czone, year)
  if (nrow(panel) > 0) {
    slope_tbl <- panel %>%
      dplyr::group_by(czone) %>%
      dplyr::summarise(
        pretrend_slope_rep_margin = tryCatch(stats::coef(stats::lm(rep_margin ~ year))[2], error = function(e) NA_real_),
        pretrend_change_first_to_last = rep_margin[which.max(year)] - rep_margin[which.min(year)],
        n_pre_years = sum(!is.na(rep_margin)),
        .groups = "drop"
      ) %>% dplyr::left_join(adh, by = "czone")
    for (outcome in c("pretrend_slope_rep_margin", "pretrend_change_first_to_last")) {
      for (rhs_var in c("exposure_1990_2007", "instrument_1990_2007")) {
        for (spec_name in names(fs_specs)) {
          ctrls <- fs_specs[[spec_name]]
          dat <- slope_tbl %>% dplyr::filter(!is.na(.data[[outcome]]), !is.na(.data[[rhs_var]]), n_pre_years >= 2)
          if (nrow(dat) > 0) {
            rhs <- c(rhs_var, ctrls)
            fml <- stats::as.formula(paste(outcome, "~", paste(rhs, collapse = " + ")))
            model <- tryCatch(fit_weighted_feols(fml, dat, weight_var = "adh_weight", cluster_var = "statefip"), error = function(e)
              ↪  NULL)
            if (!is.null(model)) {
              ct <- coeftable_to_tibble(model) %>% dplyr::filter(term == rhs_var)
              pretrend_tbl <- dplyr::bind_rows(pretrend_tbl, tibble::tibble(
                outcome = outcome, spec = spec_name, regressor = rhs_var,
                estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error),
                statistic = scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value),
                n_obs = stats::nobs(model), note = "CZ-level pre-1988 presidential Republican-margin pretrend diagnostic; not a causal
                  ↪  test."
              ))
            }
          }
        }
      }
    }
  }
}
readr::write_csv(pretrend_tbl, config$bartik_pretrend_csv)

preperiod_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china_preperiod.dta")
preperiod_tbl <- tibble::tibble()
if (file.exists(preperiod_path)) {
```

```
  pre_raw <- haven::read_dta(preperiod_path)
  names(pre_raw) <- make.names(names(pre_raw), unique = TRUE)
  names(pre_raw) <- gsub("\\.", "_", names(pre_raw))
  pre_raw <- pre_raw %>% dplyr::mutate(czone = as.integer(czone))
  candidate_outcomes <- names(pre_raw)[grepl("^d_", names(pre_raw))]
  candidate_outcomes <- setdiff(candidate_outcomes, c("d_tradeusch_pw", "d_tradeotch_pw_lag", "d_tradeusch_pw_future",
   ↪  "d_tradeotch_pw_lag_future"))
  # The preperiod workfile can contain multiple rows per CZ. Aggregate to one
  # row per CZ before running diagnostics so the output is comparable to the
  # main CZ-level exposure regressions.
  pre <- pre_raw %>%
    dplyr::group_by(czone) %>%
    dplyr::summarise(
      dplyr::across(dplyr::all_of(candidate_outcomes), ~ mean(safe_num(.x), na.rm = TRUE)),
      preperiod_rows_per_cz = dplyr::n(),
      .groups = "drop"
    ) %>%
    dplyr::mutate(dplyr::across(dplyr::all_of(candidate_outcomes), ~ dplyr::if_else(is.nan(.x), NA_real_, .x))) %>%
    dplyr::left_join(adh, by = "czone")
  for (outcome in candidate_outcomes) {
    for (rhs_var in c("exposure_1990_2007", "instrument_1990_2007")) {
      dat <- pre %>% dplyr::filter(!is.na(.data[[outcome]]), !is.na(.data[[rhs_var]]))
      if (nrow(dat) >= 50) {
        rhs <- c(rhs_var, controls_core)
        fml <- stats::as.formula(paste(outcome, "~", paste(rhs, collapse = " + ")))
        mod <- tryCatch(fit_weighted_feols(fml, dat, weight_var = "adh_weight", cluster_var = "statefip"), error = function(e) NULL)
        if (!is.null(mod)) {
          ct <- coeftable_to_tibble(mod) %>% dplyr::filter(term == rhs_var)
          preperiod_tbl <- dplyr::bind_rows(preperiod_tbl, tibble::tibble(
            outcome = outcome, regressor = rhs_var, estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error),
            statistic = scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value), n_obs = stats::nobs(mod),
            median_source_rows_per_cz = stats::median(dat$preperiod_rows_per_cz, na.rm = TRUE),
            note = "ADH preperiod-workfile diagnostic: source rows aggregated to one CZ row before regression; future
             ↪  exposure/instrument predicting preperiod/local-labor-market changes."
          ))
        }
      }
    }
  }
}
readr::write_csv(preperiod_tbl, config$bartik_preperiod_workfile_csv)

trade_summary <- tryCatch(extract_trade_shift_summary(config), error = function(e) tibble::tibble(status = "error", message =
 ↪  conditionMessage(e)))
readr::write_csv(trade_summary, config$bartik_industry_shift_summary_csv)

rot_tbl <- tibble::tibble(
  diagnostic = c("rotemberg_weights_exact", "industry_by_cz_shares_status", "industry_shift_summary_status", "next_step"),
  status = c("not_yet_computed", "requires_reconstructing_CZ_by_industry_shares", ifelse(nrow(trade_summary) > 0, "written",
   ↪  "not_written"), "feasible_with_ADH_raw_CBP_scripts_but_not_a_one-file_join"),
  details = c(
    "Exact GPSS Rotemberg weights require a CZ-by-industry baseline share matrix matched to industry-level shifts. This patch adds
     ↪  industry-shift summaries and documents the reconstruction path, but does not yet run the full ADH CBP-to-CZ industry-share
     ↪  reconstruction.",
    "Relevant public ADH files include other/cbp1980_to_cz.do, cbp1990_to_cz.do, cbp2000_to_cz.do, cbp*_imputations.do, cw_n97_s87.dta,
     ↪  and sic87dd_trade_data.dta.",
    "Industry-level China import-growth summaries are exported from sic87dd_trade_data.dta where columns are available.",
    "To compute exact GPSS weights: reconstruct 1990 CZ x SIC87DD employment shares, create shift vectors from Chinese import growth in
     ↪  the instrument country group, residualize by controls, and compute Rotemberg weights."
  )
)
readr::write_csv(rot_tbl, config$bartik_data_availability_csv)

memo <- c(
  "# Bartik / shift-share identification memo", "",
  "This project estimates reduced-form event-study relationships between CZ-level ADH China import exposure and political outcomes. The
   ↪  ADH exposure variable is a shift-share object: local baseline industry composition is combined with national or foreign
   ↪  import-growth shocks.", "",
  "## Required assumptions", "",
  "A causal reading requires high-exposure CZs not to have been on different political trajectories for reasons unrelated to the China
   ↪  shock after conditioning on fixed effects and baseline-control-by-year interactions. For the ADH-style instrument, one can
   ↪  emphasize exogeneity of foreign import-growth shifts or exogeneity of baseline industry shares; both routes require substantive
   ↪  defense.", "",
  "## Project-specific threats", "",
  "- Initial industry shares may be correlated with prior political realignment, union decline, automation, racial composition,
   ↪  religiosity, education, or local media markets.",
  "- If only a few high-shock industries drive exposure, the estimates may capture politics of those industries rather than a generic
   ↪  China-shock effect.",
  "- Migration and turnout can make CZ-level political outcomes change even if individual beliefs do not change.",
  "- Candidate supply and elite rhetoric may translate material shocks into political outcomes; the ADHM 2020 mechanism outputs are
   ↪  diagnostic for this channel.", "",
  "## Diagnostics written by the pipeline", "",
  "- `bartik_first_stage_diagnostics.csv` and `fig_bartik_first_stage.png`: first-stage strength of the ADH other-country instrument.",
  "- `bartik_balance_correlations.csv`: exposure/instrument correlations with baseline controls.",
  "- `bartik_pretrend_placebos.csv`: whether exposure/instrument predict pre-1988 Republican-margin trends.",
  "- `bartik_preperiod_workfile_diagnostics.csv`: diagnostics using ADH's preperiod workfile.",
  "- `bartik_industry_shift_summary.csv`: industry-level import-shift summaries from ADH public trade data.",
  "- `bartik_rotemberg_data_availability.csv`: exact Rotemberg weights are a feasible future reconstruction, not a completed diagnostic
   ↪  here.", ""
```

```
  )
  writeLines(memo, config$bartik_identification_memo_md)
  invisible(list(first_stage = fs_tbl, balance = balance_tbl, pretrend = pretrend_tbl, preperiod = preperiod_tbl, trade = trade_summary))
}

# ------------------------------------------------------------------------------
# 2. Alternative political outcomes
# ------------------------------------------------------------------------------

add_alternative_political_outcomes <- function(panel) {
  panel %>%
    dplyr::arrange(czone, year) %>%
    dplyr::group_by(czone) %>%
    dplyr::mutate(
      rep_2party_share = safe_divide(rep_votes, twoparty_votes),
      dem_2party_share = safe_divide(dem_votes, twoparty_votes),
      two_party_votes_per_1990_pop = safe_divide(twoparty_votes, pop1990),
      total_votes_per_1990_pop = safe_divide(total_votes, pop1990),
      rep_votes_per_1990_pop = safe_divide(rep_votes, pop1990),
      dem_votes_per_1990_pop = safe_divide(dem_votes, pop1990),
      rep_margin_swing = rep_margin - dplyr::lag(rep_margin),
      rep_2party_share_swing = rep_2party_share - dplyr::lag(rep_2party_share)
    ) %>%
    dplyr::ungroup()
}

run_alternative_political_outcome_event_studies <- function(config = CONFIG) {
  config <- finalize_config(config)
  if (!file.exists(config$analysis_panel_rds)) {
    readr::write_csv(tibble::tibble(module = "alternative_political_outcomes", status = "missing_panel", detail =
      ↪ config$analysis_panel_rds), config$alternative_outcomes_status_csv)
    return(invisible(NULL))
  }
  panel <- readRDS(config$analysis_panel_rds) %>% add_alternative_political_outcomes()
  outcomes <- c("rep_margin", "rep_2party_share", "dem_2party_share", "two_party_votes_per_1990_pop", "total_votes_per_1990_pop",
   ↪ "rep_votes_per_1990_pop", "dem_votes_per_1990_pop", "rep_margin_swing", "rep_2party_share_swing")
  outcome_labels <- tibble::tibble(outcome = outcomes, label = c("Republican margin", "Republican two-party share", "Democratic two-party
   ↪ share", "Two-party votes / 1990 population", "Total votes / 1990 population", "Republican votes / 1990 population", "Democratic
   ↪ votes / 1990 population", "Republican-margin swing since previous election", "Republican-share swing since previous election"))
  specs <- list(minimal = list(controls = character(0), standardize = FALSE), core_controls = list(controls =
   ↪ config$core_interacted_controls, standardize = TRUE), full_controls = list(controls = config$interacted_controls, standardize =
   ↪ TRUE))
  coef_tbl <- tibble::tibble(); status_tbl <- tibble::tibble()
  for (outcome in outcomes) {
    for (spec_name in names(specs)) {
      spec <- specs[[spec_name]]
      fit <- fit_single_outcome_event_study(panel, outcome, spec$controls, spec_name, config, standardize_controls = spec$standardize)
      coef_tbl <- dplyr::bind_rows(coef_tbl, fit$coefficients)
      status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(module = "alternative_political_outcomes", outcome = outcome, spec =
        ↪ spec_name, status = fit$status, n_coefficients = nrow(fit$coefficients)))
    }
  }
  coef_tbl <- coef_tbl %>% dplyr::left_join(outcome_labels, by = "outcome") %>% dplyr::mutate(spec_label = extension_spec_label(spec))
  readr::write_csv(coef_tbl, config$alternative_outcomes_coefficients_csv)
  readr::write_csv(status_tbl, config$alternative_outcomes_status_csv)
  summary_tbl <- panel %>%
    dplyr::summarise(dplyr::across(dplyr::all_of(outcomes), list(n = ~sum(!is.na(.x)), mean = ~mean(.x, na.rm = TRUE), sd =
     ↪ ~stats::sd(.x, na.rm = TRUE), min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE)), .names = "{.col}_{.fn}")) %>%
    tidyr::pivot_longer(dplyr::everything(), names_to = "stat", values_to = "value")
  readr::write_csv(summary_tbl, config$alternative_outcomes_summary_csv)
  if (nrow(coef_tbl) > 0) {
    plot_df <- coef_tbl %>% dplyr::filter(spec %in% c("minimal", "core_controls", "full_controls"), outcome %in% outcomes[1:7])
    p <- ggplot2::ggplot(plot_df, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 *
     ↪ std.error)) +
      ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(linewidth = 0.25) +
      ggplot2::facet_grid(label ~ spec_label, scales = "free_y") +
      ggplot2::labs(title = "Alternative political outcome event studies", subtitle = "CZ-clustered 95% intervals; 1988 reference where
       ↪ available", x = NULL, y = "Coefficient on ADH exposure × election year") +
      ggplot2::theme_minimal(base_size = 10)
    write_plot_safely(p, config$alternative_outcomes_plot_png, config$alternative_outcomes_plot_pdf, width = 12, height = 10)
    vote_choice <- coef_tbl %>% dplyr::filter(spec == "full_controls", outcome %in% c("rep_margin", "rep_2party_share",
     ↪ "dem_2party_share"))
    if (nrow(vote_choice) > 0) {
      p_choice <- plot_pointrange_event(vote_choice, "Vote-choice outcome diagnostics", "Full-control specification; CZ-clustered 95%
       ↪ intervals") +
        ggplot2::facet_wrap(~label, scales = "free_y", ncol = 1)
      write_plot_safely(p_choice, config$alternative_outcomes_vote_choice_plot_png, config$alternative_outcomes_vote_choice_plot_pdf,
       ↪ width = 8.5, height = 7)
    }
    vote_volume <- coef_tbl %>% dplyr::filter(spec == "full_controls", outcome %in% c("two_party_votes_per_1990_pop",
     ↪ "total_votes_per_1990_pop", "rep_votes_per_1990_pop", "dem_votes_per_1990_pop"))
    if (nrow(vote_volume) > 0) {
      p_volume <- plot_pointrange_event(vote_volume, "Vote-volume outcome diagnostics", "Full-control specification; CZ-clustered 95%
       ↪ intervals") +
        ggplot2::facet_wrap(~label, scales = "free_y", ncol = 2)
      write_plot_safely(p_volume, config$alternative_outcomes_vote_volume_plot_png, config$alternative_outcomes_vote_volume_plot_pdf,
       ↪ width = 9.5, height = 7)
    }
```

```
    decomp <- coef_tbl %>% dplyr::filter(spec == "full_controls", outcome %in% c("rep_margin", "rep_votes_per_1990_pop",
    ↪  "dem_votes_per_1990_pop", "two_party_votes_per_1990_pop"))
    p2 <- ggplot2::ggplot(decomp, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 *
    ↪  std.error)) +
      ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(linewidth = 0.25) +
      ggplot2::facet_wrap(~label, scales = "free_y", ncol = 2) +
      ggplot2::labs(title = "Vote-margin decomposition diagnostics", subtitle = "Full-control specification; CZ-clustered 95% intervals",
      ↪  x = NULL, y = "Coefficient") +
      ggplot2::theme_minimal(base_size = 10)
    write_plot_safely(p2, config$alternative_outcomes_decomposition_plot_png, config$alternative_outcomes_decomposition_plot_pdf, width =
    ↪  10, height = 7)
  }
  invisible(coef_tbl)
}

# ------------------------------------------------------------------------------
# Shared county-to-CZ bridge for county-year auxiliary outcomes
# ------------------------------------------------------------------------------

build_generic_county_to_cz_panel <- function(config, county_year_data, value_cols) {
  config <- finalize_config(config)
  cz_path <- file.path(config$replication_dir, "cz-data", "cz-198090.xls")
  ftz_zip <- file.path(config$replication_dir, "ftz2024", "crosswalks", "CountyToCounty", "1990", "1990_csv.zip")
  cz_lookup <- readxl::read_xls(cz_path) %>%
    dplyr::transmute(county_fips_1990 = stringr::str_pad(as.character(`County FIPS Code`), width = 5, side = "left", pad = "0"), czone =
    ↪  as.integer(CZ90), pop1990 = safe_num(`Population 1990`))
  source_decades <- sort(unique(year_to_source_decade(county_year_data$year)))
  xwalk <- purrr::map_dfr(source_decades, ~ read_ftz_crosswalk(.x, ftz_zip, cz_lookup$county_fips_1990, config$crosswalk_weight,
  ↪  config$crosswalk_missing_weight_policy, config$renormalize_crosswalk_weights, config$weight_sum_tolerance)) %>%
    dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0)
  bridge_exceptions <- read_county_bridge_exceptions(config)
  county_year_data %>%
    dplyr::mutate(source_decade_default = year_to_source_decade(year)) %>%
    dplyr::left_join(
      bridge_exceptions %>% dplyr::select(year, county_fips, override_source_decade, override_county_fips_source),
      by = c("year", "county_fips")
    ) %>%
    dplyr::mutate(
      source_decade = dplyr::coalesce(override_source_decade, source_decade_default),
      county_fips_source = dplyr::coalesce(override_county_fips_source, county_fips)
    ) %>%
    dplyr::left_join(xwalk, by = c("source_decade", "county_fips_source"), relationship = "many-to-many") %>%
    dplyr::filter(!is.na(county_fips_1990), !is.na(crosswalk_weight), crosswalk_weight > 0) %>%
    dplyr::mutate(dplyr::across(dplyr::all_of(value_cols), ~ safe_num(.x) * crosswalk_weight)) %>%
    dplyr::group_by(year, county_fips_1990) %>%
    dplyr::summarise(dplyr::across(dplyr::all_of(value_cols), ~ sum(.x, na.rm = TRUE)), .groups = "drop") %>%
    dplyr::inner_join(cz_lookup, by = "county_fips_1990") %>%
    dplyr::group_by(czone, year) %>%
    dplyr::summarise(dplyr::across(dplyr::all_of(value_cols), ~ sum(.x, na.rm = TRUE)), pop1990 = sum(pop1990, na.rm = TRUE), .groups =
    ↪  "drop")
}

# ------------------------------------------------------------------------------
# 3. NaNDA turnout and partisanship
# ------------------------------------------------------------------------------

run_nanda_outcome_event_studies <- function(config = CONFIG) {
  config <- finalize_config(config)
  nanda_path <- file.path(config$replication_dir, "outcomes-data", "nanda-2004-2022", "DS0001", "38506-0001-Data.rda")
  if (!file.exists(nanda_path)) {
    readr::write_csv(tibble::tibble(status = "missing", path = nanda_path), config$nanda_outcomes_summary_csv)
    return(invisible(NULL))
  }
  raw <- read_first_rdata_frame(nanda_path)
  names(raw) <- toupper(names(raw))
  fips_col <- first_existing_col(raw, c("STCOFIPS10", "STCOFIPS20", "FIPS", "COUNTY_FIPS"))
  if (is.na(fips_col) || !"YEAR" %in% names(raw)) stop("NaNDA data must contain county FIPS and YEAR columns.")
  fips_source <- if (all(c("STCOFIPS10", "STCOFIPS20") %in% names(raw))) {
    dplyr::if_else(as.integer(raw$YEAR) >= 2020L, as.character(raw$STCOFIPS20), as.character(raw$STCOFIPS10))
  } else {
    as.character(raw[[fips_col]])
  }
  numeric_candidates <- c("REG_VOTERS", "BALLOTS_CAST", "CVAP", "PRES_DEM_VOTES", "PRES_REP_VOTES", "SEN_DEM_VOTES", "SEN_REP_VOTES")
  count_cols <- numeric_candidates[numeric_candidates %in% names(raw)]
  index_cols <- c("PARTISAN_INDEX_DEM", "PARTISAN_INDEX_REP")
  index_cols <- index_cols[index_cols %in% names(raw)]
  raw$COUNTY_FIPS_SELECTED_FOR_BRIDGE <- fips_source
  nanda <- raw %>%
    dplyr::transmute(county_fips = stringr::str_pad(as.character(COUNTY_FIPS_SELECTED_FOR_BRIDGE), 5, pad = "0"), year =
    ↪  as.integer(YEAR), dplyr::across(dplyr::all_of(c(count_cols, index_cols)), safe_num)) %>%
    dplyr::mutate(dplyr::across(dplyr::all_of(c(count_cols, index_cols)), ~ dplyr::na_if(.x, -1)))
  # Restrict to documented NaNDA availability; avoid legacy/structural zeros.
  nanda <- nanda %>% dplyr::filter(year >= 2004, year <= 2022)
  county_validation <- nanda %>%
    dplyr::mutate(
      reg_voters_pct_raw = if (all(c("REG_VOTERS", "CVAP") %in% names(nanda))) safe_divide(REG_VOTERS, CVAP) else NA_real_,
      voter_turnout_pct_raw = if (all(c("BALLOTS_CAST", "CVAP") %in% names(nanda))) safe_divide(BALLOTS_CAST, CVAP) else NA_real_,
      reg_voter_turnout_pct_raw = if (all(c("BALLOTS_CAST", "REG_VOTERS") %in% names(nanda))) safe_divide(BALLOTS_CAST, REG_VOTERS) else
      ↪  NA_real_,
```

```
    county_rate_flag = dplyr::if_else(reg_voters_pct_raw > 1.5 | voter_turnout_pct_raw > 1.5 | reg_voter_turnout_pct_raw > 1.5, TRUE, FALSE, missing = FALSE)
  )
readr::write_csv(county_validation %>% dplyr::select(county_fips, year, dplyr::ends_with("_raw"), county_rate_flag), config$nanda_county_validation_csv)
# Convert index shares to CVAP-weighted pseudo-counts before bridging.
for (idx in index_cols) nanda[[paste0(idx, "_NUM")]] <- nanda[[idx]] * if ("CVAP" %in% count_cols) nanda$CVAP else 1
bridge_cols <- unique(c(count_cols, paste0(index_cols, "_NUM")))
bridge_cols <- bridge_cols[bridge_cols %in% names(nanda)]
cz_panel <- build_generic_county_to_cz_panel(config, nanda, bridge_cols)
if ("CVAP" %in% names(cz_panel)) {
  if ("REG_VOTERS" %in% names(cz_panel)) cz_panel$reg_voters_pct <- safe_divide(cz_panel$REG_VOTERS, cz_panel$CVAP)
  if ("BALLOTS_CAST" %in% names(cz_panel)) cz_panel$voter_turnout_pct <- safe_divide(cz_panel$BALLOTS_CAST, cz_panel$CVAP)
  if (all(c("BALLOTS_CAST", "REG_VOTERS") %in% names(cz_panel))) cz_panel$reg_voter_turnout_pct <- safe_divide(cz_panel$BALLOTS_CAST, cz_panel$REG_VOTERS)
  if (all(c("PRES_REP_VOTES", "PRES_DEM_VOTES") %in% names(cz_panel))) {
    denom <- cz_panel$PRES_REP_VOTES + cz_panel$PRES_DEM_VOTES
    cz_panel$pres_rep_ratio <- safe_divide(cz_panel$PRES_REP_VOTES, denom)
    cz_panel$pres_dem_ratio <- safe_divide(cz_panel$PRES_DEM_VOTES, denom)
  }
  for (idx in index_cols) {
    num <- paste0(idx, "_NUM")
    if (num %in% names(cz_panel)) cz_panel[[tolower(idx)]] <- safe_divide(cz_panel[[num]], cz_panel$CVAP)
  }
}
rate_cols <- c("reg_voters_pct", "voter_turnout_pct", "reg_voter_turnout_pct", "pres_rep_ratio", "pres_dem_ratio", "partisan_index_dem", "partisan_index_rep")
rate_cols <- rate_cols[rate_cols %in% names(cz_panel)]
cz_initial <- cz_panel %>%
  dplyr::mutate(dplyr::across(dplyr::all_of(rate_cols), ~ dplyr::if_else(.x < 0 | .x > 1.5 | !is.finite(.x), NA_real_, .x)))
outlier_rows <- cz_panel %>%
  dplyr::filter(dplyr::if_any(dplyr::all_of(rate_cols), ~ !is.na(.x) & (!is.finite(.x) | .x < 0 | .x > 1.5))) %>%
  dplyr::select(czone, year, dplyr::all_of(rate_cols))
readr::write_csv(outlier_rows, config$nanda_outlier_rates_csv)

availability_long <- cz_initial %>%
  dplyr::select(czone, year, dplyr::all_of(rate_cols)) %>%
  tidyr::pivot_longer(dplyr::all_of(rate_cols), names_to = "outcome", values_to = "value") %>%
  dplyr::group_by(outcome, year) %>%
  dplyr::summarise(
    n_cz = dplyr::n(),
    n_nonmissing = sum(!is.na(value)),
    n_zero = sum(!is.na(value) & abs(value) < 1e-12),
    n_nonzero = sum(!is.na(value) & abs(value) >= 1e-12),
    all_nonmissing_values_zero = n_nonmissing > 0 && n_nonzero == 0,
    min = if (n_nonmissing > 0) min(value, na.rm = TRUE) else NA_real_,
    max = if (n_nonmissing > 0) max(value, na.rm = TRUE) else NA_real_,
    mean = if (n_nonmissing > 0) mean(value, na.rm = TRUE) else NA_real_,
    .groups = "drop"
  ) %>%
  dplyr::mutate(available_for_event_study = n_nonmissing >= 50 & !all_nonmissing_values_zero)
readr::write_csv(availability_long, config$nanda_outcome_availability_csv)

unavailable_years <- availability_long %>%
  dplyr::filter(!available_for_event_study) %>%
  dplyr::select(outcome, year)
cz_val <- cz_initial
if (nrow(unavailable_years) > 0) {
  for (i in seq_len(nrow(unavailable_years))) {
    outcome_i <- unavailable_years$outcome[[i]]
    year_i <- unavailable_years$year[[i]]
    cz_val[[outcome_i]][cz_val$year == year_i] <- NA_real_
  }
}

cz_summary <- cz_val %>%
  dplyr::summarise(n_cz = dplyr::n_distinct(czone), min_year = min(year, na.rm = TRUE), max_year = max(year, na.rm = TRUE), dplyr::across(dplyr::all_of(rate_cols), list(n_nonmissing = ~sum(!is.na(.x)), min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE)), .names = "{.col}_{.fn}"))
readr::write_csv(cz_summary, config$nanda_cz_validation_csv)
readr::write_csv(cz_val, config$nanda_cz_panel_csv)
adh <- load_adh_extension_data(config) %>% dplyr::select(czone, statefip, exposure = exposure_1990_2007, adh_weight, dplyr::all_of(config$baseline_controls), dplyr::starts_with("z_"))
nanda_panel <- cz_val %>% dplyr::inner_join(adh, by = "czone") %>% dplyr::arrange(czone, year) %>% dplyr::group_by(czone) %>% dplyr::mutate(election_index = dplyr::dense_rank(year)) %>% dplyr::ungroup()
outcomes <- rate_cols
specs <- list(minimal = list(controls = character(0), standardize = FALSE), core_controls = list(controls = config$core_interacted_controls, standardize = TRUE), full_controls = list(controls = config$interacted_controls, standardize = TRUE))
coef_tbl <- tibble::tibble(); status_tbl <- tibble::tibble()
for (outcome in outcomes) {
  outcome_i <- outcome
  valid_years <- availability_long %>% dplyr::filter(.data$outcome == outcome_i, available_for_event_study) %>% dplyr::pull(year) %>% sort()
  ref <- if (outcome_i %in% c("partisan_index_dem", "partisan_index_rep")) {
    if (2006L %in% valid_years) 2006L else if (length(valid_years) > 0) min(valid_years) else NA_integer_
  } else if (outcome_i %in% c("pres_rep_ratio", "pres_dem_ratio")) {
    if (2004L %in% valid_years) 2004L else if (length(valid_years) > 0) min(valid_years) else NA_integer_
  } else {
```

```
      if (2004L %in% valid_years) 2004L else if (length(valid_years) > 0) min(valid_years) else NA_integer_
    }
    for (spec_name in names(specs)) {
      spec <- specs[[spec_name]]
      fit <- if (length(valid_years) >= 2 && !is.na(ref)) {
        panel_subset <- nanda_panel %>% dplyr::filter(year %in% valid_years)
        fit_single_outcome_event_study(panel_subset, outcome_i, spec$controls, spec_name, config, ref_year = ref, standardize_controls = spec$standardize, sample_label = "nanda_2004_2022")
      } else {
        list(status = "insufficient_valid_years", coefficients = tibble::tibble())
      }
      coef_tbl <- dplyr::bind_rows(coef_tbl, fit$coefficients)
      status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(module = "nanda_turnout_partisanship", outcome = outcome_i, outcome_label = extension_outcome_label(outcome_i), spec = spec_name, spec_label = extension_spec_label(spec_name), reference_year = ref, valid_years = paste(valid_years, collapse = ";"), status = fit$status, n_coefficients = nrow(fit$coefficients)))
    }
  }
  coef_tbl <- coef_tbl %>% dplyr::mutate(outcome_label = extension_outcome_label(.data$outcome), spec_label = extension_spec_label(.data$spec))
  readr::write_csv(coef_tbl, config$nanda_outcomes_coefficients_csv)
  readr::write_csv(status_tbl, config$nanda_outcomes_status_csv)
  readr::write_csv(availability_long, config$nanda_outcomes_summary_csv)
  if (nrow(coef_tbl) > 0) {
    p <- plot_pointrange_event(coef_tbl, "NaNDA turnout and partisanship outcomes", "Outcome-specific valid years and reference years; rates outside [0, 1.5] and all-zero structural years set missing; CZ-clustered intervals") +
      ggplot2::facet_grid(outcome_label ~ spec_label, scales = "free_y")
    write_plot_safely(p, config$nanda_outcomes_plot_png, config$nanda_outcomes_plot_pdf, width = 12, height = 10)
    plot_family <- function(family_outcomes, title, png_path, pdf_path, height = 6) {
      df <- coef_tbl %>% dplyr::filter(outcome %in% family_outcomes, spec == "full_controls")
      if (nrow(df) > 0) {
        pp <- plot_pointrange_event(df, title, "Full-control specification; outcome-specific reference years; CZ-clustered intervals") +
          ggplot2::facet_wrap(~outcome_label, scales = "free_y", ncol = 1)
        write_plot_safely(pp, png_path, pdf_path, width = 8.5, height = height)
      }
    }
    plot_family(c("reg_voters_pct", "voter_turnout_pct", "reg_voter_turnout_pct"), "NaNDA turnout and registration diagnostics", config$nanda_turnout_plot_png, config$nanda_turnout_plot_pdf, height = 7)
    plot_family(c("pres_rep_ratio", "pres_dem_ratio"), "NaNDA presidential vote-ratio diagnostics", config$nanda_vote_plot_png, config$nanda_vote_plot_pdf, height = 5.5)
    plot_family(c("partisan_index_dem", "partisan_index_rep"), "NaNDA partisanship-index diagnostics", config$nanda_partisanship_plot_png, config$nanda_partisanship_plot_pdf, height = 5.5)
  }
  invisible(coef_tbl)
}

# ------------------------------------------------------------------------------
# 4. ADHM 2020 political-supply/mechanism diagnostics
# ------------------------------------------------------------------------------

run_adhm2020_mechanism_diagnostics <- function(config = CONFIG) {
  config <- finalize_config(config)
  adhm_dir <- file.path(config$replication_dir, "adhm2020", "dta")
  house_path <- file.path(adhm_dir, "house_2002_2016.dta")
  pres_path <- file.path(adhm_dir, "president_2000_2016.dta")
  if (!file.exists(house_path) && !file.exists(pres_path)) {
    readr::write_csv(tibble::tibble(status = "missing", path = adhm_dir), config$adhm2020_summary_csv)
    return(invisible(NULL))
  }
  adh <- load_adh_extension_data(config)
  spec_controls <- list(minimal = character(0), core_controls = model_control_names(config$core_interacted_controls, standardize = TRUE), full_controls = model_control_names(config$interacted_controls, standardize = TRUE))
  spec_controls <- purrr::map(spec_controls, ~ .x[.x %in% names(adh)])
  regressions <- tibble::tibble(); status <- tibble::tibble()
  outcome_dict <- tibble::tibble(
    outcome = c("d2_rwin_2002_2016", "d2_cfavg_repcon_2002_2016", "d2_cfavg_repmod_2002_2016", "d2_cfavg_demmod_2002_2016", "d2_cfavg_demlib_2002_2016", "d2_teaparty_2002_2010", "d_shnr_pres_2000_2016", "d_shnr_pres_2000_2008"),
    outcome_label = adhm_outcome_label(outcome),
    interpretation = c("Change in Republican House win indicator/share", "Change in conservative Republican CF-score share", "Change in moderate Republican CF-score share", "Change in moderate Democratic CF-score share", "Change in liberal Democratic CF-score share", "Change in Tea Party outcome", "Change in Republican presidential vote share, 2000-2016", "Change in Republican presidential vote share, 2000-2008"),
    report_family = c("House control", "House ideology", "House ideology", "House ideology", "House ideology", "Tea Party", "Presidential", "Presidential")
  )
  readr::write_csv(outcome_dict, config$adhm2020_outcome_dictionary_csv)
  fit_cross_section <- function(df, outcomes, source, weight_var = NULL) {
    purrr::map_dfr(outcomes[outcomes %in% names(df)], function(outcome) {
      purrr::imap_dfr(spec_controls, function(ctrls, spec_name) {
        dat <- df %>% dplyr::inner_join(adh, by = "czone") %>% dplyr::filter(!is.na(.data[[outcome]]), !is.na(exposure_1990_2007))
        if (nrow(dat) == 0) {
          status <<- dplyr::bind_rows(status, tibble::tibble(source = source, outcome = outcome, spec = spec_name, status = "empty", n_obs = 0L, note = NA_character_))
          return(tibble::tibble())
        }
        rhs <- c("exposure_1990_2007", ctrls)
        fml <- stats::as.formula(paste(outcome, "~", paste(rhs, collapse = " + ")))
        mod <- tryCatch(fit_weighted_feols(fml, dat, weight_var = if (!is.null(weight_var) && weight_var %in% names(dat)) weight_var else "adh_weight", cluster_var = "statefip"), error = function(e) e)
        if (inherits(mod, "error")) {
```

```
          status <<- dplyr::bind_rows(status, tibble::tibble(source = source, outcome = outcome, spec = spec_name, status = "error", n_obs = nrow(dat), note = conditionMessage(mod)))
          return(tibble::tibble())
        }
        ct <- coeftable_to_tibble(mod) %>% dplyr::filter(term == "exposure_1990_2007")
        status <<- dplyr::bind_rows(status, tibble::tibble(source = source, outcome = outcome, spec = spec_name, status = "ok", n_obs = stats::nobs(mod), note = NA_character_))
        tibble::tibble(source = source, outcome = outcome, outcome_label = adhm_outcome_label(outcome), spec = spec_name, spec_label = extension_spec_label(spec_name), estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error), statistic = scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value), n_obs = stats::nobs(mod), controls = if (length(ctrls) == 0) "none" else paste(ctrls, collapse = "; "))
      })
    })
  }
  if (file.exists(house_path)) {
    house <- haven::read_dta(house_path) %>% dplyr::mutate(czone = as.integer(czone), sh_district_2002 = dplyr::coalesce(safe_num(sh_district_2002), 1))
    house_outcomes <- c("d2_rwin_2002_2016", "d2_cfavg_repcon_2002_2016", "d2_cfavg_repmod_2002_2016", "d2_cfavg_demmod_2002_2016", "d2_cfavg_demlib_2002_2016", "d2_teaparty_2002_2010")
    house_cz <- house %>% dplyr::group_by(czone) %>% dplyr::summarise(dplyr::across(dplyr::all_of(house_outcomes[house_outcomes %in% names(house)]), ~ weighted_mean_safe(.x, sh_district_2002)), .groups = "drop")
    regressions <- dplyr::bind_rows(regressions, fit_cross_section(house_cz, house_outcomes, "adhm2020_house_2002_2016"))
  }
  if (file.exists(pres_path)) {
    pres <- haven::read_dta(pres_path) %>% dplyr::mutate(czone = as.integer(czone), totvote_2000pres = dplyr::coalesce(safe_num(totvote_2000pres), 1))
    pres_cz <- pres %>%
      dplyr::group_by(czone) %>%
      dplyr::summarise(shnr_pres2000 = weighted_mean_safe(shnr_pres2000, totvote_2000pres), shnr_pres2008 = weighted_mean_safe(shnr_pres2008, totvote_2000pres), shnr_pres2016 = weighted_mean_safe(shnr_pres2016, totvote_2000pres), .groups = "drop") %>%
      dplyr::mutate(d_shnr_pres_2000_2016 = shnr_pres2016 - shnr_pres2000, d_shnr_pres_2000_2008 = shnr_pres2008 - shnr_pres2000)
    regressions <- dplyr::bind_rows(regressions, fit_cross_section(pres_cz, c("d_shnr_pres_2000_2016", "d_shnr_pres_2000_2008"), "adhm2020_president_2000_2016"))
  }
  readr::write_csv(regressions, config$adhm2020_regressions_csv)
  readr::write_csv(status, config$adhm2020_status_csv)
  hetero_vars <- c("l_sh_pop_white", "l_sh_popwht", "sh_pop_white", "l_sh_pop_black", "l_sh_pop_hispanic")
  avail <- tibble::tibble(candidate_variable = hetero_vars, available_in_current_ADH_extension_data = hetero_vars %in% names(adh), note = "If a majority-white or race-composition variable is available, add exposure × baseline-race heterogeneity to compare with ADHM 2020 heterogeneity results.")
  readr::write_csv(avail, config$adhm2020_heterogeneity_availability_csv)
  summary <- tibble::tibble(source = c("house_2002_2016", "president_2000_2016"), file_present = c(file.exists(house_path), file.exists(pres_path)), purpose = c("Political-supply and House ideology outcomes from ADHM public replication data", "Presidential-vote benchmark outcomes from ADHM public replication data"))
  readr::write_csv(summary, config$adhm2020_summary_csv)
  if (nrow(regressions) > 0) {
    plot_dat <- regressions %>%
      dplyr::mutate(outcome_label = factor(outcome_label, levels = rev(unique(outcome_label))))
    p <- ggplot2::ggplot(plot_dat, ggplot2::aes(x = estimate, y = outcome_label, xmin = estimate - 1.96 * std.error, xmax = estimate + 1.96 * std.error)) +
      ggplot2::geom_vline(xintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(linewidth = 0.25) +
      ggplot2::facet_grid(source ~ spec_label, scales = "free_y", space = "free_y") +
      ggplot2::labs(title = "ADHM 2020 political-supply diagnostics", subtitle = "Cross-sectional CZ regressions on ADH exposure; public ADHM outcomes aggregated to CZs", x = "Coefficient on ADH exposure", y = NULL) +
      ggplot2::theme_minimal(base_size = 10)
    write_plot_safely(p, config$adhm2020_plot_png, config$adhm2020_plot_pdf, width = 12, height = 7)
  }
  invisible(regressions)
}

# ---------------------------------------------------------------------------
# 6. Subperiod exposure
# ---------------------------------------------------------------------------

run_subperiod_exposure_event_studies <- function(config = CONFIG) {
  config <- finalize_config(config)
  adh_stack_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china.dta")
  if (!file.exists(adh_stack_path) || !file.exists(config$analysis_panel_rds)) return(invisible(NULL))
  stack <- haven::read_dta(adh_stack_path)
  exp_period <- stack %>%
    dplyr::filter(yr %in% c(1990, 2000)) %>%
    dplyr::transmute(czone = as.integer(czone), period = dplyr::case_when(yr == 1990 ~ "1990_2000", yr == 2000 ~ "2000_2007", TRUE ~ as.character(yr)), exposure = safe_num(d_tradeusch_pw), instrument = safe_num(d_tradeotch_pw_lag)) %>%
    tidyr::pivot_wider(names_from = period, values_from = c(exposure, instrument), names_sep = "_")
  readr::write_csv(exp_period, config$subperiod_exposure_csv)
  readr::write_csv(exp_period %>% dplyr::summarise(dplyr::across(-czone, list(mean = ~mean(.x, na.rm = TRUE), sd = ~stats::sd(.x, na.rm = TRUE), min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE)), .names = "{.col}_{.fn}")), config$subperiod_exposure_summary_csv)
  adh <- load_adh_extension_data(config)
  fs <- exp_period %>% dplyr::inner_join(adh %>% dplyr::select(czone, adh_weight, statefip, dplyr::starts_with("z_")), by = "czone")
  fs_tbl <- tibble::tibble()
  for (pair in list(c("exposure_1990_2000", "instrument_1990_2000"), c("exposure_2000_2007", "instrument_2000_2007"))) {
    if (all(pair %in% names(fs))) {
      for (spec_name in c("minimal", "core_controls", "full_controls")) {
        ctrls <- if (spec_name == "minimal") character(0) else if (spec_name == "core_controls") model_control_names(config$core_interacted_controls, TRUE) else model_control_names(config$interacted_controls, TRUE)
        ctrls <- ctrls[ctrls %in% names(fs)]
        fml <- stats::as.formula(paste(pair[1], "~", paste(c(pair[2], ctrls), collapse = " + ")))
```

```
      mod <- tryCatch(fit_weighted_feols(fml, fs, weight_var = "adh_weight", cluster_var = "statefip"), error = function(e) NULL)
      if (!is.null(mod)) {
        ct <- coeftable_to_tibble(mod) %>% dplyr::filter(term == pair[2])
        fs_tbl <- dplyr::bind_rows(fs_tbl, tibble::tibble(exposure_period = gsub("exposure_", "", pair[1]), spec = spec_name,
          ↪ instrument = pair[2], estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error), statistic =
          ↪ scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value), first_stage_F_t2 = scalar_or_na(ct$statistic)^2, n_obs =
          ↪ stats::nobs(mod)))
      }
    }
  }
}
readr::write_csv(fs_tbl, config$subperiod_first_stage_csv)

panel <- readRDS(config$analysis_panel_rds) %>% dplyr::left_join(exp_period, by = "czone")
yrs <- config$event_years; ref <- config$reference_year
for (ev in c("exposure_1990_2000", "exposure_2000_2007")) for (yr in setdiff(yrs, ref)) panel[[paste0(ev, "_x_", yr)]] <- panel[[ev]] *
 ↪ as.integer(panel$year == yr)
specs <- list(minimal = list(controls = character(0), standardize = FALSE), core_controls = list(controls =
 ↪ config$core_interacted_controls, standardize = TRUE), full_controls = list(controls = config$interacted_controls, standardize =
 ↪ TRUE))
coef_tbl <- tibble::tibble(); status_tbl <- tibble::tibble()
for (spec_name in names(specs)) {
  spec <- specs[[spec_name]]
  dat <- panel
  ctrl_build <- apply_control_interactions(dat, spec$controls, yrs, ref, standardize = spec$standardize)
  dat <- ctrl_build$data
  event_terms <- unlist(lapply(c("exposure_1990_2000", "exposure_2000_2007"), function(ev) paste0(ev, "_x_", setdiff(yrs, ref))))
  fml <- stats::as.formula(paste("rep_margin ~", paste(c(event_terms, ctrl_build$terms), collapse = " + "), "| czone + year"))
  fit <- tryCatch(fixest::feols(fml, data = dat, weights = ~adh_weight, cluster = ~czone, panel.id = ~czone + election_index, warn =
   ↪ FALSE, notes = FALSE), error = function(e) e)
  if (inherits(fit, "error")) {
    status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(spec = spec_name, status = "error", error_message =
     ↪ conditionMessage(fit), n_coefficients = 0L))
  } else {
    ct <- coeftable_to_tibble(fit) %>%
      dplyr::filter(term %in% event_terms) %>%
      dplyr::mutate(
        spec = spec_name,
        exposure_period = dplyr::case_when(startsWith(term, "exposure_1990_2000") ~ "1990-2000", startsWith(term, "exposure_2000_2007")
         ↪ ~ "2000-2007", TRUE ~ NA_character_),
        year = as.integer(stringr::str_extract(term, "[0-9]{4}$")),
        period_type = dplyr::case_when(year < 1990 ~ "placebo_pre_shock", year <= 2008 ~ "during_or_immediate_post_shock", year <= 2016
         ↪ ~ "medium_run", TRUE ~ "long_run"),
        reference_year = ref,
        n_obs = stats::nobs(fit), n_cz = dplyr::n_distinct(dat$czone)
      ) %>%
      dplyr::select(spec, exposure_period, period_type, term, year, reference_year, estimate, std.error, statistic, p.value, n_obs,
       ↪ n_cz)
    coef_tbl <- dplyr::bind_rows(coef_tbl, ct)
    status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(spec = spec_name, status = "ok", error_message = NA_character_,
     ↪ n_coefficients = nrow(ct)))
  }
}
readr::write_csv(coef_tbl, config$subperiod_event_study_coefficients_csv)
readr::write_csv(status_tbl, config$subperiod_event_study_status_csv)
equality <- tibble::tibble()
if (nrow(coef_tbl) > 0) {
  wide <- coef_tbl %>% dplyr::select(spec, year, exposure_period, estimate, std.error) %>% tidyr::pivot_wider(names_from =
   ↪ exposure_period, values_from = c(estimate, std.error), names_sep = "_")
  if (all(c("estimate_1990-2000", "estimate_2000-2007") %in% names(wide))) {
    equality <- wide %>% dplyr::mutate(difference_early_minus_late = `estimate_1990-2000` - `estimate_2000-2007`, approx_se_difference =
     ↪ sqrt((`std.error_1990-2000`)^2 + (`std.error_2000-2007`)^2), approx_t_difference = difference_early_minus_late /
     ↪ approx_se_difference, approx_p_difference = 2 * stats::pnorm(-abs(approx_t_difference)), note = "Approximate test ignores
     ↪ covariance between early and late exposure coefficients; use as descriptive diagnostic.")
  }
}
readr::write_csv(equality, config$subperiod_equality_tests_csv)
selected_horizons <- tibble::tibble()
if (nrow(equality) > 0) {
  selected_horizons <- equality %>%
    dplyr::filter(year %in% c(1996L, 2000L, 2008L, 2016L, 2020L)) %>%
    dplyr::transmute(
      spec, year,
      early_exposure_coefficient = `estimate_1990-2000`,
      late_exposure_coefficient = `estimate_2000-2007`,
      early_minus_late = difference_early_minus_late,
      approx_se_difference, approx_t_difference, approx_p_difference,
      interpretation_note = "Approximate early-vs-late comparison; covariance ignored. Use for descriptive horizon interpretation."
    )
}
readr::write_csv(selected_horizons, config$subperiod_selected_horizons_csv)
if (nrow(coef_tbl) > 0) {
  p <- ggplot2::ggplot(coef_tbl, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 *
   ↪ std.error, color = exposure_period)) +
    ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(position = ggplot2::position_dodge(width = 0.6),
     ↪ linewidth = 0.25) +
    ggplot2::facet_wrap(~spec, ncol = 1, scales = "free_y") +
    ggplot2::labs(title = "Subperiod ADH exposure event study", subtitle = "1990-2000 and 2000-2007 exposure terms entered jointly", x =
     ↪ NULL, y = "Coefficient", color = "Exposure period") +
```

```
      ggplot2::theme_minimal(base_size = 11)
    write_plot_safely(p, config$subperiod_event_study_plot_png, config$subperiod_event_study_plot_pdf, width = 10, height = 8)
  }
  invisible(coef_tbl)
}

run_sixth_extensions <- function(config = CONFIG) {
  config <- extension_primary_config(config)
  ensure_output_dirs(config)
  load_required_packages(config)
  message("Running Sixth-section extension diagnostics for primary design: ", toupper(config$crosswalk_weight_slug), " + ",
   ↪  config$crosswalk_missing_weight_policy)
  results <- list()
  results$bartik <- tryCatch(run_bartik_identification_diagnostics(config), error = function(e) { warning("Bartik diagnostics failed: ",
   ↪  conditionMessage(e), call. = FALSE); NULL })
  results$alternative_outcomes <- tryCatch(run_alternative_political_outcome_event_studies(config), error = function(e) {
   ↪  warning("Alternative political outcomes failed: ", conditionMessage(e), call. = FALSE); NULL })
  results$nanda <- tryCatch(run_nanda_outcome_event_studies(config), error = function(e) { warning("NaNDA outcomes failed: ",
   ↪  conditionMessage(e), call. = FALSE); NULL })
  results$adhm2020 <- tryCatch(run_adhm2020_mechanism_diagnostics(config), error = function(e) { warning("ADHM 2020 diagnostics failed:
   ↪  ", conditionMessage(e), call. = FALSE); NULL })
  results$subperiod <- tryCatch(run_subperiod_exposure_event_studies(config), error = function(e) { warning("Subperiod exposure
   ↪  diagnostics failed: ", conditionMessage(e), call. = FALSE); NULL })
  checks <- if (file.exists(config$validation_checks_csv)) readr::read_csv(config$validation_checks_csv, show_col_types = FALSE) else
   ↪  tibble::tibble()
  write_pipeline_manifest(config = config, checks = checks, stage = "sixth_extensions", sources = list(adh2013 = list(path =
   ↪  file.path(config$replication_dir, "adh2013")), nanda = list(path = file.path(config$replication_dir, "outcomes-data",
   ↪  "nanda-2004-2022")), adhm2020 = list(path = file.path(config$replication_dir, "adhm2020"))), extra = list(extension_modules =
   ↪  c("bartik_identification", "alternative_political_outcomes", "nanda_turnout_partisanship", "adhm2020_political_supply",
   ↪  "subperiod_exposure"), crosswalk_used_for_extensions = paste0(config$crosswalk_weight, " / ",
   ↪  config$crosswalk_missing_weight_policy)))
  invisible(results)
}
```

8 run_pipeline.R

```
# extension-adh2013/src/run_pipeline.R

if (!requireNamespace("here", quietly = TRUE)) {
  stop("Package 'here' is required. Install it before running the pipeline.", call. = FALSE)
}
library(here)
here::i_am("src/run_pipeline.R")

source(here::here("src", "00_config.R"))
source(here::here("src", "01_helpers.R"))
source(here::here("src", "02_build_analysis_data.R"))
source(here::here("src", "03_estimate_event_study.R"))
source(here::here("src", "04_export_outputs.R"))
source(here::here("src", "05_export_diagnostic_maps.R"))
source(here::here("src", "06_sixth_extensions.R"))

read_bool_env <- function(name, default) {
  value <- Sys.getenv(name, unset = "")
  if (!nzchar(value)) return(default)
  tolower(value) %in% c("1", "true", "t", "yes", "y")
}

run_single_pipeline <- function(config) {
  config <- finalize_config(config)
  ensure_output_dirs(config)
  write_dependency_manifest(config)

  message("Building analysis data...")
  build_analysis_data(config, stop_on_fatal = TRUE)

  message("Estimating event-study specifications...")
  estimate_event_study(config, stop_on_fatal = TRUE)

  message("Exporting tables and figures...")
  export_event_study_outputs(config)

  message("Exporting diagnostic maps and crosswalk comparisons when inputs are available...")
  export_diagnostic_maps_and_comparisons(config)

  if (isTRUE(config$run_sixth_extensions)) {
    message("Running Sixth-section extensions...")
    run_sixth_extensions(config)
  }

  invisible(config)
}

policy_override <- Sys.getenv("CROSSWALK_MISSING_WEIGHT_POLICY", unset = "")
if (nzchar(policy_override)) CONFIG$crosswalk_missing_weight_policy <- policy_override

weight_override <- Sys.getenv("CROSSWALK_WEIGHT", unset = "")
if (nzchar(weight_override)) CONFIG$crosswalk_weight <- weight_override

ext_weight_override <- Sys.getenv("EXTENSION_CROSSWALK_WEIGHT", unset = "")
if (nzchar(ext_weight_override)) CONFIG$extension_crosswalk_weight <- ext_weight_override

ext_policy_override <- Sys.getenv("EXTENSION_CROSSWALK_MISSING_WEIGHT_POLICY", unset = "")
if (nzchar(ext_policy_override)) CONFIG$extension_crosswalk_missing_weight_policy <- ext_policy_override

CONFIG$renormalize_crosswalk_weights <- read_bool_env(
  "RENORMALIZE_CROSSWALK_WEIGHTS", CONFIG$renormalize_crosswalk_weights
)
CONFIG$require_balanced_panel <- read_bool_env(
  "REQUIRE_BALANCED_PANEL", CONFIG$require_balanced_panel
)
CONFIG$require_no_vcov_repair_for_main <- read_bool_env(
  "REQUIRE_NO_VCOV_REPAIR_FOR_MAIN", CONFIG$require_no_vcov_repair_for_main
)
CONFIG$allow_vcov_repair <- read_bool_env("ALLOW_VCOV_REPAIR", CONFIG$allow_vcov_repair)
CONFIG$allow_failed_diagnostics <- read_bool_env(
  "ALLOW_FAILED_DIAGNOSTICS", CONFIG$allow_failed_diagnostics
)
CONFIG$standardize_interacted_controls <- read_bool_env(
  "STANDARDIZE_INTERACTED_CONTROLS", CONFIG$standardize_interacted_controls
)
CONFIG$export_crosswalk_maps <- read_bool_env(
  "EXPORT_CROSSWALK_MAPS", CONFIG$export_crosswalk_maps
)
CONFIG$run_crosswalk_sensitivity <- read_bool_env(
  "RUN_CROSSWALK_SENSITIVITY", CONFIG$run_crosswalk_sensitivity
)
CONFIG$run_sixth_extensions <- read_bool_env(
  "RUN_SIXTH_EXTENSIONS", CONFIG$run_sixth_extensions
)
CONFIG$save_single_rds <- read_bool_env(
  "SAVE_SINGLE_RDS", CONFIG$save_single_rds
)
```

```
88
89  if (isTRUE(CONFIG$run_crosswalk_sensitivity)) {
90    message("Running the four-specification crosswalk sensitivity pipeline...")
91    run_crosswalk_sensitivity_pipeline(CONFIG)
92  } else {
93    run_single_pipeline(CONFIG)
94  }
95
96  message("Pipeline complete.")
```

9 county_bridge_exceptions.csv

```
year,county_fips,county_name_pattern,override_source_decade,override_county_fips_source,issue_class,reason,apply
1984,04012,La Paz,1990,04012,county_boundary_change_needs_bridge,"La Paz County, AZ was created after 1980 but exists in 1990 target geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
1988,04012,La Paz,1990,04012,county_boundary_change_needs_bridge,"La Paz County, AZ was created after 1980 but exists in 1990 target geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
1984,35006,Cibola,1990,35006,county_boundary_change_needs_bridge,"Cibola County, NM was created after 1980 but exists in 1990 target geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
1988,35006,Cibola,1990,35006,county_boundary_change_needs_bridge,"Cibola County, NM was created after 1980 but exists in 1990 target geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
2016,46102,Oglala Lakota,2010,46113,county_boundary_change_needs_bridge,"Oglala Lakota County, SD is the renamed Shannon County; map 2016 returns to the 2010/1990 Shannon FIPS where needed.",TRUE
2020,46102,Oglala Lakota,2010,46113,county_boundary_change_needs_bridge,"Oglala Lakota County, SD is the renamed Shannon County; map 2020 returns to the 2010/1990 Shannon FIPS where needed.",FALSE
2004,08014,Broomfield,2010,08014,county_boundary_change_needs_bridge,"Broomfield County, CO was created after the 2000 Census; use the 2010 source crosswalk for 2004 returns so its vote mass is allocated to 1990 counties.",TRUE
2008,08014,Broomfield,2010,08014,county_boundary_change_needs_bridge,"Broomfield County, CO was created after the 2000 Census; use the 2010 source crosswalk for 2008 returns so its vote mass is allocated to 1990 counties.",TRUE
1976,51683,Manassas,1980,51683,county_boundary_change_needs_bridge,"Manassas city appears in 1976 returns but not the 1970 source crosswalk used by the decade rule; use the 1980 source crosswalk.",TRUE
1976,51685,Manassas Park,1980,51685,county_boundary_change_needs_bridge,"Manassas Park city appears in 1976 returns but not the 1970 source crosswalk used by the decade rule; use the 1980 source crosswalk.",TRUE
1976,51735,Poquoson,1980,51735,county_boundary_change_needs_bridge,"Poquoson city appears in 1976 returns but not the 1970 source crosswalk used by the decade rule; use the 1980 source crosswalk.",TRUE
```

10 dependencies.json

```
{
  "description": "Dependency manifest for the extension-adh2013 source pipeline. Install core packages before running; the runner does not install packages. The sf package is needed for routine diagnostic map rendering; ipumsr is optional for retrieving NHGIS shapefiles. Large model outputs are saved as compressed chunks by default to avoid GitHub's 100 MiB file limit.",
  "core_packages": [
    "here",
    "dplyr",
    "fixest",
    "ggplot2",
    "haven",
    "jsonlite",
    "kableExtra",
    "knitr",
    "purrr",
    "readr",
    "readxl",
    "rlang",
    "stringr",
    "tibble",
    "tidyr",
    "sf"
  ],
  "optional_packages": [
    "ipumsr"
  ]
}
```

11 make_replication_code_pdf.R

```
# src/make_replication_code_pdf.R
# Generate a Quarto document that prints all replication source files.

library(fs)
library(glue)
library(stringr)

project_root <- rprojroot::find_root(rprojroot::has_file("src/run_pipeline.R"))
src_dir <- file.path(project_root, "src")
out_dir <- file.path(project_root, "report")
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

qmd_path <- file.path(out_dir, "replication-code.qmd")

source_files <- dir_ls(
  src_dir,
  recurse = FALSE,
  type = "file",
  regexp = "\\.(R|r|csv|json|md|qmd|yml|yaml)$"
)

# Put pipeline files first, then auxiliary data/config files.
preferred_order <- c(
  "00_config.R",
  "01_helpers.R",
  "02_build_analysis_data.R",
  "03_estimate_event_study.R",
  "04_export_outputs.R",
  "05_export_diagnostic_maps.R",
  "06_sixth_extensions.R",
  "run_pipeline.R"
)

source_files <- source_files[order(
  match(path_file(source_files), preferred_order, nomatch = 999),
  path_file(source_files)
)]

rel_for_latex <- function(path) {
  # QMD lives in report/, so paths are relative to report/.
  rel <- fs::path_rel(path, start = out_dir)
  # LaTeX on Windows is happier with forward slashes.
  rel <- gsub("\\\\", "/", rel)
  rel
}

escape_md_inline_code <- function(x) {
  # File names here should not contain backticks, but be defensive.
  gsub("`", "\\\\`", x)
}

header <- c(
  "---",
  'title: "Replication Code Appendix"',
  'subtitle: "ADH China Shock Political Event Study Extension"',
  'author: "Rishav Roy"',
  "format:",
  "  pdf:",
  "    documentclass: article",
  "    geometry:",
  "      - margin=0.65in",
  "    fontsize: 9pt",
  "    toc: true",
  "    number-sections: true",
  "    colorlinks: true",
  "    keep-tex: true",
  "    include-in-header:",
  "      text: |",
  "        \\usepackage{fvextra}",
  "        \\usepackage{xcolor}",
  "        \\DefineVerbatimEnvironment{CodeFile}{Verbatim}{%",
  "          breaklines=true,",
  "          breakanywhere=true,",
  "          breaksymbolleft={},",
  "          breaksymbolright={},",
  "          fontsize=\\scriptsize,",
  "          numbers=left,",
  "          numbersep=5pt,",
  "          frame=single,",
  "          framerule=0.2pt,",
  "          framesep=2mm,",
  "          tabsize=2,",
  "          commandchars=\\\\\\{\\}",
  "        }",
  "execute:",
  "  echo: false",
  "  warning: false",
```

```
  "  message: false",
  "---",
  "",
  "This appendix prints the replication source files used to construct the final results.",
  "",
  "The files are included directly from `src/` at render time.",
  ""
)

body <- unlist(lapply(source_files, function(path) {
  nm <- path_file(path)
  rel <- rel_for_latex(path)

  c(
    glue("\\newpage"),
    "",
    glue("# `{escape_md_inline_code(nm)}`"),
    "",
    "```{=latex}",
    glue("\\VerbatimInput[breaklines=true,breakanywhere=true,fontsize=\\scriptsize,numbers=left,frame=single,tabsize=2]{{{rel}}}"),
    "```",
    ""
  )
}))

writeLines(c(header, body), qmd_path)

message("Wrote ", qmd_path)

if (nzchar(Sys.which("quarto"))) {
  old <- setwd(out_dir)
  on.exit(setwd(old), add = TRUE)
  system2("quarto", c("render", basename(qmd_path)))
} else {
  message("Quarto not found on PATH. Render manually with: quarto render report/replication-code.qmd")
}
```

12 README.md

````
# ADH politics event-study pipeline (`extension-adh2013/src`)

The pipeline builds CZ-level presidential outcomes from Amlani and Algara (2021), bridges counties to 1990 geography with the FTZ county crosswalk, joins ADH China-shock exposure and baseline controls, validates the panel, estimates event-study specifications, and exports diagnostics, tables, figures, crosswalk support diagnostics, and demonstration maps.

Run from the project root:

```r
source(here::here("src", "run_pipeline.R"))
```

## Default run

The default run is the practical M5 + M2-fallback specification used for diagnostics and drafting:

- `crosswalk_weight = "m5_weight"`
- `crosswalk_missing_weight_policy = "fallback_m2"`
- `require_balanced_panel = TRUE`
- `min_retained_vote_share = 0.98`
- `main_se_type = "cluster_cz"`
- `allow_vcov_repair = FALSE`
- `require_no_vcov_repair_for_main = TRUE`
- `standardize_interacted_controls = TRUE`
- `save_single_rds = FALSE`

Set `CROSSWALK_MISSING_WEIGHT_POLICY="fail"` when you want a strict diagnostic run that stops if selected FTZ weights have zero/undefined support.

## Four-specification crosswalk sensitivity run

To run M5 + fallback M2, M6 + fallback M2, pure M2, and M4 + fallback M2 in one pass and create the appendix comparison plots:

```r
Sys.setenv(RUN_CROSSWALK_SENSITIVITY = "true")
source(here::here("src", "run_pipeline.R"))
```

The sensitivity plan is:

| Label | `CROSSWALK_WEIGHT` | Missing-weight policy |
|---|---|---|
| M5 + fallback M2 | `m5_weight` | `fallback_m2` |
| M6 + fallback M2 | `m6_weight` | `fallback_m2` |
| Pure M2 | `m2_weight` | `fallback_m2` |
| M4 + fallback M2 | `m4_weight` | `fallback_m2` |

The M4 sensitivity specification uses M2 fallback so all four appendix series use the same balanced ADH-mainland CZ panel. The pure M2 run has `m2_weight` selected; its fallback policy only affects non-analysis/unavailable cases and does not change the selected M2 weights.

The comparison outputs are written to:

- `output/diagnostics/crosswalk_specification_comparison_coefficients.csv`
- `output/diagnostics/crosswalk_specification_comparison_deltas_vs_pure_m2.csv`
- `output/figures/fig_crosswalk_specification_comparison.png`
- `output/figures/fig_crosswalk_specification_comparison.pdf`
- `output/figures/fig_crosswalk_specification_comparison_deltas_vs_pure_m2.png`
- `output/figures/fig_crosswalk_specification_comparison_deltas_vs_pure_m2.pdf`

The level plot uses colors, line widths, transparencies, and line types to make nearly overlapping lines visible. The delta plot shows differences relative to Pure M2, which is often more informative when crosswalk estimates almost perfectly overlap.

## Demonstration maps

The demo maps use **1990 county geography** and flag ADH-mainland 1990 counties/CZs that receive **positive bridged vote mass** from source counties where M2, M4, M5, or M6 support is zero/undefined. This simplified demonstration does not require 1970/1980/2000/2010/2020 shapefiles.

Required runtime inputs:

1. the full FTZ county-to-county crosswalk ZIP at
   `ftz2024/crosswalks/CountyToCounty/1990/1990_csv.zip`;
2. a 1990 county shapefile under `spatial-data/counties-1990` (for example
   the Stanford/BTAA `co1990p020.shp` and companion files) or `COUNTY1990_SHAPEFILE` set to a `.shp` file or directory containing one;
3. package `sf` installed.

The map exporter writes full-resolution and simplified-geometry versions:

- `fig_crosswalk_unavailable_weights_1990_counties.*`
- `fig_crosswalk_unavailable_weights_1990_counties_simplified.*`
- `fig_crosswalk_unavailable_weights_1990_czs.*`
- `fig_crosswalk_unavailable_weights_1990_czs_simplified.*`

All PNGs are saved with a white background for portable previewing.

## Large model-output files and GitHub
````

````
GitHub blocks regular repository files larger than 100 MiB. The pipeline therefore defaults to `save_single_rds = FALSE` and writes large model objects as compressed chunk directories such as:

```text
output/intermediate/event_study_results_all_specs_aa2021_rep_margin_m5_rds_chunks/
```

Each chunk is capped by `rds_chunk_size_mib` (default: 45 MiB). The single `.rds` files are ignored by `.gitignore`; the chunk directories are intended to be GitHub-safe. Set `SAVE_SINGLE_RDS=true` only for local debugging or if you use Git LFS.

If a large `.rds` has already been committed, remove it from Git history before pushing; adding it to `.gitignore` only prevents future additions.

## Dependency policy

The runner does not install packages. Install the core dependencies listed in `src/dependencies.json`. The `sf` package is required for routine diagnostic maps; `ipumsr` is optional for NHGIS shapefile retrieval.


## Crosswalk sensitivity appendix run

Set `RUN_CROSSWALK_SENSITIVITY=true` to run four appendix specifications in one pass:

- M5 + fallback M2
- M6 + fallback M2
- Pure M2
- M4 + fallback M2

The runner writes `output/diagnostics/crosswalk_sensitivity_run_status.csv` and archives per-spec diagnostics under `output/diagnostics/by_crosswalk/<slug>/`. The M4 specification intentionally uses M2 fallback so that all four sensitivity series use the same balanced ADH mainland CZ panel. The all-spec coefficient CSVs now include all Conley cutoffs/specifications that can be computed, even when the VCOV is flagged as repair-required; use the `vcov_repair_required` and `vcov_repaired` columns to avoid treating those rows as publication-ready inference.

Diagnostic support-map PNGs are saved with full, simplified, and small-raster versions.

## Latest validation/output manifest conventions

The pipeline now distinguishes true validation failures from handled/reported diagnostic conditions. In `pipeline_validation_checks.csv`, rows with `status == "reported"` are nonfatal conditions that have been explicitly documented (for example, global/nonmainland source-county support rows outside the ADH-mainland estimating sample). Rows with `status == "fail"` are the rows that trigger failure logic when fatal.

The table/figure export no longer labels outputs as provisional solely because handled `reported` conditions exist. It still uses provisional labeling if an unhandled validation failure or main-VCOV numerical repair is recorded.

Every run writes an output inventory:

- `output/diagnostics/output_manifest.csv`
- `output/diagnostics/output_manifest.json`

These files list generated outputs, file sizes, MD5 checksums, inferred crosswalk slugs, and broad output categories. They are intended as the final reproducibility inventory for the generated `output/` tree.

Conley rows in `event_study_se_warning_diagnostics.csv` now carry explicit diagnostic notes when the unrepaired Conley VCOV is non-positive-definite or requires numerical repair. Such Conley estimates remain diagnostic-only unless separately justified.

## Sixth-section extensions

The runner now defaults to `run_sixth_extensions = TRUE`. These extensions are run once for the primary design, even when `RUN_CROSSWALK_SENSITIVITY=true`, so mechanism and identification outputs are not multiplied by crosswalk specification unless explicitly requested in future work. The primary extension design defaults to M5 + fallback M2 and can be overridden with:

```r
Sys.setenv(EXTENSION_CROSSWALK_WEIGHT = "m5_weight")
Sys.setenv(EXTENSION_CROSSWALK_MISSING_WEIGHT_POLICY = "fallback_m2")
```

The extension modules write:

- `output/diagnostics/bartik_first_stage_diagnostics.csv`
- `output/diagnostics/bartik_balance_correlations.csv`
- `output/diagnostics/bartik_pretrend_placebos.csv`
- `output/diagnostics/bartik_rotemberg_data_availability.csv`
- `output/diagnostics/bartik_identification_memo.md`
- `output/diagnostics/alternative_political_outcome_event_studies.csv`
- `output/figures/fig_alternative_political_outcomes.*`
- `output/intermediate/nanda_2004_2022_cz_panel.csv`
- `output/diagnostics/nanda_outcome_event_studies.csv`
- `output/figures/fig_nanda_outcome_event_studies.*`
- `output/diagnostics/adhm2020_public_data_summary.csv`
- `output/diagnostics/adhm2020_mechanism_regressions.csv`
- `output/diagnostics/adhm2020_outcome_dictionary.csv`
- `output/intermediate/adh_subperiod_exposure_by_cz.csv`
- `output/diagnostics/subperiod_exposure_event_study_coefficients.csv`
- `output/figures/fig_subperiod_exposure_event_study.*`
````

```
Exact GPSS Rotemberg weights are not computed unless a CZ-by-industry baseline share matrix is added or reconstructed. The current Bartik module therefore reports first-stage, balance, and pretrend diagnostics and writes a data-availability note explaining what is missing for exact Rotemberg weights.

## Sixth-section extension diagnostics

`06_sixth_extensions.R` now writes status/validation files for each extension module. Key additions include:

- `bartik_first_stage_diagnostics.csv`, `fig_bartik_first_stage.*`, `bartik_preperiod_workfile_diagnostics.csv`, and `bartik_industry_shift_summary.csv` for shift-share identification diagnostics.
- `alternative_political_outcome_status.csv` plus `fig_alternative_political_outcome_decomposition.*` for vote-share/vote-volume/swing decomposition.
- `nanda_county_year_validation.csv`, `nanda_cz_year_validation.csv`, `nanda_outlier_rates.csv`, and `nanda_outcome_event_study_status.csv`; NaNDA rate outcomes with impossible values outside `[0, 1.5]` are set to missing before estimation.
- `adhm2020_mechanism_regression_status.csv`, `adhm2020_heterogeneity_data_availability.csv`, and `fig_adhm2020_mechanism_regressions.*` for ADHM public political-supply diagnostics across minimal, core, and full-control specifications.
- `subperiod_exposure_first_stage.csv` and `subperiod_exposure_equality_tests.csv` for 1990-2000 versus 2000-2007 exposure diagnostics.

These extensions are intentionally estimated once under the preferred M5 + fallback-M2 design, even when `RUN_CROSSWALK_SENSITIVITY=true`, unless the preferred panel has not yet been generated.
```